

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس رباتیک و بینایی ماشین

استاد: دکتر حمیدرضا تقی راد

استاد: دکتر محمدجواد احمدی

تمرین سری سوم

گروه آزادی

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	هستی محمدی
شماره دانشجویی	۴۰۲۲۱۲۴۳
تاریخ	اردیبهشت ۱۴۰۵



فهرست مطالب

۶	۱ پرسش اول: شناخت دامنه، تحلیل اکتشافی داده‌ها EDA و استراتژی‌های پیش‌پردازش
۶	۱.۱ درک سناریوی بالینی
۶	۱.۱.۱ شرح جراحی آب مروارید و فازهای اصلی آن
۷	۲.۱.۱ اهمیت آگاهی از فازهای جراحی برای سیستم‌های دستیار هوشمند
۹	۲.۱ فرمول‌بندی مسئله
۹	۱.۲.۱ طبقه‌بندی‌های Single-Label و Multi-Label و تفاوت‌های آن‌ها
۱۰	۲.۲.۱ طبقه‌بندی مناسب برای سناریو جراحی آب مروارید
۱۰	۳.۲.۱ تابع زیان مناسب در صورت انتخاب روش Multi-Label
۱۰	۴.۲.۱ محدودیت‌های تابع Cross-Entropy استاندارد و رویکرد جایگزین برای آن
۱۱	۳.۱ تحلیل داده‌ها و چالش‌های بصری
۱۱	۱.۳.۱ تحلیل آماری و توصیفی از توزیع کلاس‌ها
۲۳	۲.۳.۱ بررسی توازن کلاس‌ها (Class Imbalance)
۲۸	۳.۳.۱ روش‌های پیشنهادی افزایش داده
۳۰	۲ پرسش دوم: طراحی معماری، تنظیم فرآیند آموزش و تفسیرپذیری مدل (XAI)
۳۲	۱.۲ انتخاب معماری و یادگیری انتقال (Transfer Learning)
۳۲	۱.۱.۲ انتخاب معماری
۳۲	۲.۱.۲ چرا آموزش از صفر (From-Scratch) مناسب نیست؟
۳۳	۳.۱.۲ استفاده از وزن‌های از پیش آموزش دیده ImageNet
۳۳	۲.۲ استراتژی آموزش و بهینه‌سازی
۳۳	۱.۲.۲ جستجوی فراپارامترها با Optuna
۳۴	۲.۲.۲ مقایسه Focal Loss و Label Smoothing برای مقابله با عدم تعادل کلاس‌ها
۳۶	۳.۲.۲ دورویکرد آموزشی
۳۶	۳.۲ پیاده‌سازی رویکرد اول: یادگیری چندبرچسبی با استفاده از EfficientNet-B۳
۳۷	۱.۳.۲ ساختار خط لوله (Pipeline) پیاده‌سازی
۳۷	۲.۳.۲ نتایج بهینه‌سازی و تحلیل روند آموزش
۳۸	۳.۳.۲ ارزیابی مدل بر روی داده‌های تست
۴۰	۴.۳.۲ تحلیل فضای ویژگی با t-SNE
۴۱	۵.۳.۲ تفسیرپذیری مدل با استفاده از Grad-CAM
۴۲	۴.۲ پیاده‌سازی رویکرد دوم: حذف ساختارهای آناتومیک و موازنه ابزارها
۴۲	۱.۴.۲ تعدیل برچسب‌ها و استراتژی موازنه (Data Balancing)
۴۳	۲.۴.۲ بهینه‌سازی فراپارامترها با Optuna
۴۳	۳.۴.۲ روند آموزش و تحلیل بیش‌برازش (Overfitting)
۴۴	۴.۴.۲ ارزیابی نهایی و مقایسه با رویکرد اول



۴۸		کد	۵.۲
۸۰		پرسش سوم: اعتبارسنجی متقاطع، ارزیابی پایداری و یادگیری تجمعی	۳
۸۰		رویکردهای اعتبارسنجی متقاطع (Cross-Validation)	۱.۳
۸۰		مقایسه تطبیقی K-Fold و Stratified K-Fold:	۱.۱.۳
۸۰		ضرورت استفاده از روش Stratified در جراحی آب مروارید:	۲.۱.۳
۸۱		پایه سازی و ریفکتور کد	۲.۲
۸۱		پایه سازی اعتبارسنجی متقاطع لایه بندی شده (Multilabel Stratified K-Fold):	۱.۲.۳
۸۱		تحلیل کمی و کیفی نتایج (Results Analysis):	۲.۲.۳
۸۲		کد	۳.۲.۳
۸۸		یادگیری تجمعی (Ensemble Learning) در برابر سرعت استنتاج	۳.۳



فهرست تصاویر

۱۸	نمودار توزیع برچسب‌ها (بدون کلاس‌های آناتومی)	۱
۱۹	خروجی کد (ماتریس هم‌رخدادی)	۲
۲۰	ماتریس هم‌رخدادی (بدون کلاس‌های آناتومی)	۳
۲۲	نمودار توزیع برچسب‌ها (با کلاس‌های آناتومی)	۴
۲۳	نمودار ماتریس هم‌رخدادی (با کلاس‌های آناتومی)	۵
۲۶	فراوانی داده در هر کلاس (بدون کلاس‌های آناتومی)	۶
۲۶	منحنی لورنتز (بدون کلاس‌های آناتومی)	۷
۲۸	فراوانی داده در هر کلاس (با کلاس‌های آناتومی)	۸
۲۸	منحنی لورنتز (با کلاس‌های آناتومی)	۹
۳۱	توزیع داده‌ها در هر کلاس روی مجموعه داده های آموزش	۱۰
۳۱	توزیع داده‌ها در هر کلاس روی مجموعه داده های اعتبارسنجی	۱۱
۳۲	توزیع داده‌ها در هر کلاس روی مجموعه داده های تست	۱۲
۳۷	تحلیل روند آموزش و اعتبارسنجی (مشاهده پدیده Overfitting بعد از اپوک ۲)	۱۳
۳۹	نمایش ماتریس درهم‌ریختگی برای رویکرد اول	۱۴
۴۰	نگاشت فضای ویژگی داده های تست در دو بعد	۱۵
۴۱	تفسیرپذیری مدل با استفاده از Grad-CAM در رویکرد اول	۱۶
۴۳	توزیع داده‌ها در هر کلاس بعد از آگمنتیشن در رویکرد دوم	۱۷
۴۴	روند تغییرات خطا و صحت در رویکرد دوم (۱۰ کلاس)	۱۸
۴۶	ماتریس درهم‌ریختگی مجموعه داده تست برای رویکرد دوم	۱۹
۴۷	نگاشت فضای ویژگی داده های تست در دو بعد	۲۰
۴۸	تفسیرپذیری مدل با استفاده از Grad-CAM در رویکرد دوم	۲۱



فهرست جداول

۳۸	مقادیر PR-AUC برای هر کلاس	۱
۳۸	گزارش عملکرد مدل EfficientNet-B3 بر روی داده‌های آزمون (رویکرد اول)	۲
۴۴	گزارش عملکرد مدل EfficientNet-B3 در رویکرد دوم (۱۰ کلاس)	۳
۴۵	مقادیر PR-AUC برای ۱۰ کلاس در رویکرد دوم	۴
۸۱	نتایج اعتبارسنجی متقاطع ۵-تایی برای مدل EfficientNet-B3	۵



فهرست برنامه‌ها

۱۱	تحلیل جامع دیتاست
۱۶	بررسی توزیع برچسب‌ها
۱۸	ماتریس هم‌رخدادی
۲۳	بررسی فراوانی کلاس‌ها و توازن داده‌ها



۱ پرسش اول: شناخت دامنه، تحلیل اکتشافی داده‌ها EDA و استراتژی‌های پیش‌پردازش

۱.۱ درک سناریوی بالینی

۱.۱.۱ شرح جراحی آب مروارید و فازهای اصلی آن

در جراحی آب مروارید یا کاتاراکت (Cataract) عدسی طبیعی چشم بیمار که کدر شده است را با یک عدسی مصنوعی شفاف تعویض می‌کنند. خارج کردن عدسی کدر شده معمولاً از طریق امولسیفیکیشن (Emulsification) انجام می‌شود که در آن با استفاده از امواج فراصوت عدسی خرد می‌شود و می‌توان آن را از طریق برشی کوچک (در حدود ۳ میلی‌متر) خارج و جایگزین کرد. اما در مواردی که پیشرفت بیماری بیشتر باشد، عدسی سفت شده ممکن است با امواج فراصوت خرد نشود و لازم است آن را به صورت کامل خارج و جایگزین کرد که به برش بزرگ‌تری بین ۶ تا ۱۰ میلی‌متر نیاز دارد. واضح است که در روش دوم دوره نقاهت بیمار طولانی‌تر خواهد بود. جراحی آب مروارید به طور معمول بین ۱۰ تا ۱۵ دقیقه زمان می‌برد و با بی‌حسی موضعی انجام می‌شود. به طور کلی می‌توان ۱۰ فاز اصلی برای جراحی کاتاراکت در نظر گرفت که در ادامه به بررسی این فازها و ابزارهای مورد استفاده در هر یک می‌پردازیم.

۱. برش جانبی (Side Incision): نخستین مرحله در جراحی آب مروارید به شمار می‌رود که در آن یک یا چند برش ثانویه کوچک در محیط قرنیه ایجاد می‌شود. این برش معمولاً ابعادی در حدود ۰.۸mm تا ۱.۲mm دارد هدف اصلی آن فراهم کردن امکان ورود ابزارهای کمکی، انجام تزریق‌های لازم و دستکاری بافت‌های داخل چشم در مراحل بعدی جراحی است. ابزار اصلی مورد استفاده چاقوی لانس (Lance tip Knife) یا چاقوی الماسه (Diamond Knife) است که هر دو دارای زوایایی تیز برای ایجاد برش‌های دقیق هستند.

۲. برش اصلی (Main Incision): در این مرحله، برش اصلی برای ورود ابزارهای جراحی قرنیه ایجاد می‌شود. این برش معمولاً ابعادی در حدود ۲.۲mm تا ۳.۰mm داشته و به گونه‌ای طراحی می‌شود که پایداری ساختاری چشم را حفظ کرده و نشت مایعات پس از عمل را کاهش دهد.

ابزار اصلی مورد استفاده چاقوی کراتوم (Keratome blade) با پهنای کالیبره‌شده با توجه به ابعاد برش است.

۳. کپسولورکسیس (Capsulorhexis): یکی از حساس‌ترین بخش‌های جراحی محسوب می‌شود و در آن یک برش دایره‌ای، پیوسته و متقارن در کپسول قدامی عدسی چشم برای دسترسی مستقیم به هسته کاتاراکت ایجاد می‌شود. ابزارهای مورد استفاده شامل پنس کپسولورکسیس (Capsorhexis forceps) یا سیستیتوم (Cystotome)^۱ هستند.

۴. هیدرودیسکشن (Hydrodissection): در این مرحله با تزریق ملایم و کنترل‌شده مایع بین کپسول عدسی و هسته عدسی، چسبندگی‌های موجود آزاد می‌شوند. به طور دقیق‌تر، هسته (Nucleus) و قشر (Cortex) عدسی از کپسول محیطی جدا می‌شوند و امکان چرخش آزادانه عدسی را فراهم می‌شود. ابزارهای مورد استفاده شامل کانونلای هیدرودیسکسیون (Hydrodissection cannula) متصل به سرنگ حاوی محلول نمکی متعادل‌شده (BSS - Balanced Salt Solution).

۵. فاکوآمولسیفیکاسیون (Phacoemulsification): مرحله اصلی برداشت آب مروارید است که در آن با استفاده از امواج فراصوت با فرکانس بالا، هسته سخت کاتاراکت خرد می‌شود و همزمان ذرات خرد شده توسط سیستم مکش از چشم خارج می‌شوند.

^۱ یک سوزن ظریف و خمیده برای هدایت پارگی کپسول



هندپیس یا پروب فاکو (Phacoemulsification probe) به عنوان ابزار فعال اصلی و ابزار کمکی چاپر (Chopper) جهت شکستن مکانیکی هسته به قطعات کوچک تر استفاده می شوند.

۶. فاز حذف قشر عدسی (Cortical Removal): پس از خروج هسته، لایه های قشری نرم باقی مانده که به دیواره کپسول چسبیده اند، پاکسازی و مکش می شوند تا کیسه کپسول کاملاً شفاف و آماده پذیرش لنز شود.

در این مرحله از ابزار هندپیس شستشو و مکش (Irrigation/Aspiration handpiece - I/A) که می تواند به صورت هم محور (Coaxial) یا دو کاناله مجزا (Bimanual) باشد، استفاده می شود.

۷. کاشت عدسی (Lens Insertion): در این فاز لازم است یک لنز داخل چشمی تاشو (Intraocular Lens - IOL) درون کیسه کپسولی تزریق و جایگذاری شود. عدسی مصنوعی پس از خروج از ابزار تزریق (Injector)، به آرامی باز شده و در موقعیت مناسب قرار داده می شود.

از انژکتور لنز (IOL Injector) مجهز به کارتریج مخصوص (Cartridge system) برای متراکم و هدایت کردن لنز تاشو برای این کار استفاده می شود.

۸. حذف ماده ویسکوالاستیک (Ophthalmic Viscosurgical Device Removal): در طول جراحی از مواد ویسکوالاستیک برای حفظ فضای داخل چشم و محافظت از بافت های حساس استفاده می شود. پس از کاشت لنز، این مواد باید کاملاً خارج شوند تا از افزایش فشار داخل چشمی پس از عمل جلوگیری شود.

برای این کار مجدداً از هندپیس شستشو و مکش (I/A handpiece) برای پاکسازی کامل فضای داخل چشمی استفاده می شود.

۹. بستن زخم با هیدراتاسیون قرنیه (Wound Closure - Corneal Hydration):

در بسیاری از جراحی های مدرن، برش های کوچک به صورت خودبسته شونده طراحی می شوند. در این مرحله با تزریق مایع به استرومای قرنیه در اطراف محل برش، لبه های زخم متورم شده و به یکدیگر فشرده می شوند که موجب آب بندی برش می گردد. در این مرحله با تزریق متمرکز مایع به لبه های قرنیه در محل برش های اصلی و جانبی بافت لبه های زخم متورم شده و به یکدیگر فشرده می شوند تا بدون نیاز به بخیه، پایداری ساختاری چشم تامین شود.

ابزار مورد استفاده در این مرحله کانولای هیدراتاسیون ظریف مانند کانولای رایکرافت (Rycroft cannula) متصل به سرنگ حاوی (BSS) است.

۱۰. بستن زخم با بخیه زدن (Wound Closure - Suturing Incision): این فاز پایان رسمی جراحی را نشان می دهد که در آن در شرایطی که برش پایدار نباشد یا احتمال نشت وجود داشته باشد، از بخیه برای بستن محل برش استفاده می شود. این روش بیشتر در برش های بزرگ یا موارد پیچیده کاربرد دارد.

برای بخیه از نخ بخیه نایلونی غیر قابل جذب بسیار ظریف با سایز 10 – 0 (10-0 Nylon suture)، پنس بخیه (Suture forceps) و سوزن گیر مخصوص میکرو جراحی (Micro-needle holder) استفاده می شود.

۲.۱.۱ اهمیت آگاهی از فازهای جراحی برای سیستم های دستیار هوشمند

فرآیند جراحی کاتاراکت را می توان به عنوان یک مسئله زمان مند مدل سازی کرد که در آن فاز جراحی در زمان t به صورت تابعی از ویژگی های بصری استخراج شده از فریم میکروسکوپ جراحی و وضعیت ابزارهای جراحی شناسایی شده در محیط متغیر عمل نوشته می شود.



در معماری سیستم‌های هوشمند کمکی و بازوهای رباتیک مورد استفاده در جراحی، مدل‌سازی رفتار سیستم و اتخاذ سیاست‌های ایمنی، تابعی مستقیم از حالت پویای محیط عملیاتی است و به همین دلیل آگاهی آنی از فاز جاری جراحی اهمیت بسیاری پیدا می‌کند. شرط پایداری و ایمنی سیستم هوشمند این است که فاز تخمین شده توسط الگوریتم بینایی کامپیوتر و فاز واقعی فعلی در جراحی یکی باشند؛ در صورت بروز تاخیر زمانی، تخمین سیستم در لحظات انتقال فاز، دیگر با فاز واقعی جراحی تطابق ندارد.

دلایل ضرورت آگاهی بلادرنگ از فاز جراحی

تنظیم محدودیت‌های مجازی: در جراحی رباتیک، محدودیت‌های مجازی حرکتی برای جلوگیری از ورود ابزار به نواحی آناتومیک حساس تعریف می‌شوند. هندسه این نواحی ممنوعه در هر فاز متفاوت است. بر فرض مثال، در فاز کپسولورکسیس مرزهای حرکتی باید بازوی رباتیک را به یک مسیر مدور دو بعدی روی کپسول قدامی مقید کنند، در حالی که در فاز فاکوآمولسیفیکاسیون، این مرزها باید به صورت سه بعدی تغییر کرده و مانع از حرکت پروب به سمت کپسول خلفی شوند.

مدیریت هوشمند جریان کار و پارامترهای جانبی: سیستم هوشمند جراحی نه تنها بازوی رباتیک، بلکه تجهیزات جانبی مانند نرخ شستشو (Irrigation rate)، میزان مکش (Aspiration suction) و توان امواج اولتراسونیک را کنترل می‌کند. تنظیم دقیق فشار داخل چشمی (IOP) برای جلوگیری از فروپاشی اتاق قدامی چشم، نیازمند نداشت بلادرنگ بین فاز فعلی جراحی و تنظیمات دستگاه است.

مدیریت محدودیت‌های سینماتیکی بازوی رباتیک: پیشتر در بررسی سینماتیک رباتیک، متوجه نقص‌هایی در بیان ریاضی موقعیت ابزار نهایی (End-Effector) شدیم که Gimbal Lock نام دارد و ناشی از وجود تکینگی (Singularity) در برخی نقاط خاص است. در این حالت درجه آزادی ربات از دید سیستم یک واحد کم شده است، در حالی که از لحاظ فیزیکی هیچ مشکلی در ربات ایجاد نشده است که می‌تواند رفتار سیستم را غیرقابل پیش‌بینی و الگوریتم‌های کنترل و تخمین را سردرگم کند. در کاربردهای پزشکی خصوصاً جراحی‌های چشم مانند جراحی آب‌مروارید که محدوده حرکتی در ابعاد میلی‌متری تعریف می‌شود، این مسئله می‌تواند منجر به عواقب آناتومیک جبران‌ناپذیر شود. بنابراین، پیش از ورود به فازهایی که نیازمند دامنه حرکتی وسیع‌تر یا تعویض ابزار هستند (مانند انتقال از فاز هیدرودیسکسیون به فاکوآمولسیفیکاسیون)، سیستم باید موقعیت محل برش را نسبت به پایه ربات ارزیابی کند و فضای کاری ایمن بدون تکینگی (Singularity-free workspace) تضمین شود تا بازوی رباتیک در حین جراحی مجبور به تغییر آرایش ناگهانی نشود. اما این برنامه‌ریزی پیش از شروع فاز به دلیل پویا (Dynamic) بودن محیط، جابجایی احتمالی سر بیمار یا نیاز جراح به مانورهای پیش‌بینی نشده کافی نیست و الگوریتم کنترل ربات باید معیار تکینگی را محاسبه کند تا هنگام نزدیک شدن به نقاط تکینگی، با استفاده از روش‌های مناسب سرعت را به طور ایمن محدود یا آرایش مفاصل ربات را بدون تغییر در موقعیت ابزار داخل چشم اصلاح کند.

پیامدهای عدم تشخیص یا تشخیص نادرست فاز جراحی

اگر سیستم دچار خطای طبقه‌بندی (Misclassification) شود یا به دلیل تاخیر در پردازش فریم‌های ویدئویی، تغییر فاز را بلافاصله درک نکند، ممکن است سناریوهای زیر رخ دهند:

پارگی کپسول خلفی: سهمگین‌ترین عارضه در جراحی کاتاراکت، پارگی کپسول خلفی است. اگر ربات به اشتباه تصور کند سیستم همچنان در فاز حذف قشر عدسی است در حالی که جراح وارد فاز حذف ماده ویسکوالاستیک شده است، محدودیت‌های حرکتی لازم را اعمال نکرده و یک حرکت ناخواسته یا اعمال نیروی بیش از حد می‌تواند منجر به سوراخ شدن کپسول خلفی و خروج زجاجیه شود.



اعمال اشتباه و آسیب‌زای نیرو یا انرژی: چنانچه سیستم هوشمند فاز فاکوآمولسیفیکاسیون (که نیازمند امواج فراصوت است) را با فازهای حساس دیگر مانند هیدرو دیسکسیون یا کاشت لنز اشتباه بگیرد، ممکن است سیگنال فعال‌سازی انرژی فراصوت را در زمان یا مکان نادرست صادر کند. این امر منجر به سوختگی حرارتی شدید بافت قرنیه و تخریب سلول‌ها می‌شود که غیرقابل بازگشت است.

عدم همگام‌سازی سیستم همکار با جراح: در تعامل انسان و ربات، اگر سیستم پشتیبان فاز متفاوتی را فرض کند، رفتاری متناقض با قصد جراح نشان خواهد داد. این پدیده موجب مقاومت مکانیکی بازو در برابر حرکت دست جراح یا هدایت اشتباه ابزار شده و تعادل جراح را برهم می‌زند و ریسک خطا را به صورت افزایش می‌دهد.

۲.۱ فرمول‌بندی مسئله

در تحلیل ویدیوهای جراحی و سیستم‌های بینایی ماشین در اتاق عمل دو روش یادگیری عمیق Single-Label و چندبرچسبی Multi-Label برای طبقه‌بندی ویژگی‌ها وجود دارد. انتخاب بین این دو رویکرد مستقیماً بر معماری لایه نهایی شبکه و طراحی تابع زیان (Loss Function) تأثیرگذار است.

۱.۲.۱ طبقه‌بندی‌های Single-Label و Multi-Label و تفاوت‌های آن‌ها

۱. روش Single-Label

در این روش، کلاس‌های تعریف‌شده در مسئله همگی از نظر معنایی یکتا و دارای انحصار متقابل با یکدیگر هستند؛ به این معنی که هر نمونه ورودی (مانند یک فریم ویدیو) در لحظه تنها می‌تواند متعلق به یک کلاس باشد. در این حالت تعداد نوروهای لایه خروجی شبکه عصبی با تعداد کلاس‌ها برابر است و شبکه عصبی برای هر نمونه، برداری از لاجیت‌ها (logits) به نام z تولید می‌کند. سپس با اعمال تابع Softmax، این لاجیت‌ها به توزیع احتمالی بین کلاس‌ها تبدیل می‌شوند:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

۲. روش Multi-Label

در این روش، فرض انحصار متقابل کلاس‌ها حذف می‌شود؛ به این معنی که کلاس‌های مختلف دیگر مستقل از یکدیگر در نظر گرفته می‌شوند و یک فریم ورودی می‌تواند به طور هم‌زمان شامل هیچ، یک، یا چند برچسب باشد. در لایه نهایی این شبکه، همچنان تعداد نوروها برابر با تعداد کلاس‌ها است اما این کلاس‌ها دیگر با یکدیگر رقابت نمی‌کنند. بنابراین به جای تابع Softmax، تابع Sigmoid به صورت مستقل روی هر لاجیت اعمال می‌شود تا احتمال حضور هر برچسب به طور جداگانه بین 0 و 1 محاسبه شود. تابع فعال‌سازی Sigmoid به صورت زیر تعریف می‌شود:

$$P(y_c = 1 | x) = \frac{1}{1 + e^{-z_c}}$$

۳. تفاوت‌های اصلی دو روش

۱. انحصار متقابل کلاس‌ها (Mutual Exclusivity): در روش تک‌برچسبی، کلاس‌ها رقیب یکدیگرند و مجموع احتمالات خروجی تمام نوروها همواره برابر با 1 است. در روش چندبرچسبی، خروجی‌ها مستقل از یکدیگرند و مجموع آن‌ها محدودیتی ندارد.

۲. مکانیزم فعال‌سازی (Activation Mechanism): تابع Softmax با بزرگ شدن خروجی یک کلاس، به شدت احتمالات سایر کلاس‌ها را سرکوب می‌کند. در مقابل، تابع Sigmoid اجازه می‌دهد افزایش احتمال حضور یک ابزار هیچ اثر کاهنده‌ای بر احتمال حضور ابزار دیگر نداشته باشد.

۳. ساختار Ground Truth Vector: بردار هدف در روش Single-Label یک بردار با تنها یک مقدار 1 و مابقی 0 است (One-hot)، در حالی که در روش Multi-Label بردار هدف یک بردار چندمقداری است که می‌تواند حاوی چندین مقدار 1 به صورت هم‌زمان باشد (Multi-hot).

۴. حساسیت به عدم تعادل داده‌ها (Data Imbalance Sensitivity): تابع هزینه Multi-Label به دلیل بررسی مستقل حضور یا عدم حضور هر کلاس، به شدت در معرض چالش عدم تعادل داده‌ها قرار دارد. چرا که در بیشتر فریم‌های جراحی، اکثر ابزارها غایب هستند و مقدار پیش‌فرض برچسب آن‌ها 0 است. این امر گاهی جابجایی به توابع پیشرفته‌تر مانند زیان فوکال Focal Loss را الزامی می‌کند، در حالی که تابع استاندارد Cross-Entropy در روش Single-Label، پایداری بهتری در برابر این پدیده دارد.

۲.۲.۱ طبقه‌بندی مناسب برای سناریو جراحی آب مروارید

با توجه به اینکه ممکن است چند ابزار به صورت هم‌زمان در میدان دید وجود داشته باشند، روش Multi-Label تنها گزینه معتبر و کاربردی است. در جراحی کاتاراکت، جراح به طور مداوم از ترکیب هم‌زمان ابزارها استفاده می‌کند. به عنوان مثال در فاز فاکوآمولسیفیکاسیون، پروب فاکو و ابزار چاپر در یک فریم حضور دارند. بنابراین Single-Label فرض کردن مسئله به دلیل ترکیب‌های متنوع ابزارها، منجر به از بین رفتن کارایی مدل می‌شود.

۳.۲.۱ تابع زیان مناسب در صورت انتخاب روش Multi-Label

در روش Multi-Label تابع زیان به صورت Binary cross-entropy with logits برای تک‌تک کلاس‌ها تعریف می‌شود:

$$L = - \sum_{c=1}^C [y_c \log(P(y_c = 1 | x)) + (1 - y_c) \log(1 - P(y_c = 1 | x))]$$

از آنجا که در محیط‌های عملیاتی با کلاس‌های نامتوازن (Imbalanced Classes) مواجه هستیم، استفاده از تابع زیان Focal نیز ضروری است:

$$\mathcal{L}_{\text{focal}} = -(1 - p_y)^\gamma \log p_y$$

این تابع با اعمال ضریب $(1 - p_y)^\gamma$ ، وزن نمونه‌های ساده (easy examples) را کاهش داده و گرادینان را به سمت داده‌های سخت (Hard ones) هدایت می‌کند.

۴.۲.۱ محدودیت‌های تابع Cross-Entropy استاندارد و رویکرد جایگزین برای آن

تابع زیان در حالت Single-Label، Cross-entropy استاندارد است که خطا را برای برچسب واقعی y به صورت زیر محاسبه می‌کند:

$$\mathcal{L} = - \log p_y$$



در PyTorch، برای اعمال آن از دستور `CrossEntropyLoss.nn` استفاده می‌شود که برای حفظ پایداری عددی، لاجیت‌های خام (نه خروجی تابع Softmax) را به عنوان ورودی دریافت می‌کند.

تابع زیان Cross-Entropy با وجود کارآمد بودن در روش Single-Label دارای برخی محدودیت‌های ساختاری و عملیاتی است:

- ایجاد Overconfidence در شبکه: ساختار ریاضی این تابع به گونه‌ای است که لاجیت‌های مربوط به کلاس صحیح را به سمت مثبت بی‌نهایت و لاجیت‌های سایر کلاس‌ها را به سمت منفی بی‌نهایت میل می‌دهد تا احتمال کلاس هدف به عدد 1 برسد. این امر باعث کاهش تعمیم‌پذیری مدل و کالیبراسیون می‌شود. برای بهبود این مشکلات، از Label smoothing استفاده می‌شود که برچسب‌های one-hot به صورت $[0, 0, 1, 0, \dots]$ را به $[\frac{\epsilon}{K-1}, \frac{\epsilon}{K-1}, 1 - \frac{\epsilon}{K-1}, \frac{\epsilon}{K-1}, \dots]$ تبدیل می‌کند. در این حالت مقدار ϵ کوچک (مثلاً 0.1) انتخاب شده و اهداف تبدیل به Soft Targets می‌شوند.
 - آسیب‌پذیری در برابر عدم توازن کلاس‌ها: این تابع زیان برای تمامی نمونه‌ها وزن یکسانی در محاسبه گرایان قائل است. در کاربردهایی مانند جراحی کاتاراکت، برخی فازها مانند فاکوآمولسفیو کاسیون مدت زمان بسیار طولانی‌تری نسبت به فازهای کوتاهی مانند برش قرنیه دارند. در نتیجه مدل، فازهای اقلیت را نادیده می‌گیرد.
 - حساسیت به برچسب‌های نویزدار: به دلیل ماهیت لگاریتمی تابع، ممکن است مدل برای یک نمونه نویزدار یا با برچسب‌گذاری اشتباه، پیش‌بینی نادرستی با اطمینان بالا انجام دهد که عواقب بسیار سنگینی را در پی دارد.
- از طرف دیگر تابع زیان Cross-Entropy در روش Multi-Label غیرقابل استفاده است. این مسئله ناشی از فرضیات ریاضی در تابع فعال‌سازی Softmax است. در تابع Softmax، احتمالات خروجی به یکدیگر وابسته هستند و مجموع احتمالات برای تمامی کلاس‌ها الزاماً برابر با 1 خواهد بود؛ به عبارت دیگر افزایش احتمال حضور یک کلاس (مثلاً حضور پنس)، الزاماً باعث کاهش احتمال حضور دیگر کلاس‌ها (مثلاً حضور چاپر) در پیش‌بینی شبکه می‌شود. این ویژگی کاملاً با ماهیت سیستم Multi-Label متناقض است. در این سیستم خروجی شبکه باید برای هر کلاس یک احتمال مستقل بین 0 و 1 باشد و مجموع احتمالات کلاس‌ها می‌تواند از 1 بزرگ‌تر باشد.
- بنابراین به جای استفاده از ترکیب، تابع فعال‌سازی Sigmoid برای هر لاجیت به طور مستقل استفاده می‌شود تا احتمالات خروجی نیز مستقل باشند. سپس تابع Binary Cross-Entropy with Logits برای هر کلاس اعمال می‌شود که تک تک کلاس‌ها را به عنوان یک مسئله دسته‌بندی باینری مستقل مدیریت می‌کند.

۳.۱ تحلیل داده‌ها و چالش‌های بصری

۱.۳.۱ تحلیل آماری و توصیفی از توزیع کلاس‌ها

برای تحلیل داده‌ها، می‌توان کلاس‌های آناتومی (Pupil و Cornea) را حذف کرد و تنها ابزارهای جراحی را بررسی کرد یا همه کلاس‌ها را همزمان بررسی کرد. در ادامه داده‌های مربوط به هر دو رویکرد آمده‌اند، اما رویکرد اول داده‌های معنادارتری در اختیار ما می‌گذارد چرا که دو کلاس آناتومی مذکور تقریباً در همه فریم‌ها وجود دارند.

رویکرد اول (بدون کلاس‌های آناتومی)

اطلاعات جامع در مورد داده‌ها

در بخش اول، اطلاعات کلی و جامع در رابطه با داده‌های موجود با استفاده از کد زیر در پایتون استخراج می‌کنیم.

لازم به ذکر است، در همه تحلیل‌ها، کلاس‌های آناتومیک Pupil و Cornea کنار گذاشته شده‌اند (چرا که تقریباً در همه فریم‌ها وجود دارند) و تنها ابزارهای جراحی مورد بررسی‌اند.

```
1 # SECTION 1
2 # libraries
3 import warnings
4 from pathlib import Path
5 from itertools import combinations
6 from collections import Counter
7
8 import numpy as np
9 import pandas as pd
10 import matplotlib.pyplot as plt
11 import matplotlib.patches as mpatches
12 import matplotlib.gridspec as gridspec
13 import seaborn as sns
14
15 warnings.filterwarnings("ignore")
16
17 # constants
18 DATASET_ROOT = Path(r"C:\Users\ASUS\Desktop\New folder
19 ↪ (2)\Cataract_dataset_classification.vli.multiclass")
20 SPLITS = ["train", "valid", "test"]
21 CSV_NAME = "_classes.csv"
22 OUTPUT_DIR = Path(r"C:\Users\ASUS\Desktop\New folder (2)\analysis_outputs")
23 OUTPUT_DIR.mkdir(exist_ok=True)
24
25 # shortened labels for display
26 SHORT = {
27     "Cannula" : "Cannula",
28     "Cap Cystotome" : "Cap Cyst.",
29     "Cap Forceps" : "Cap Forc.",
30     "Cornea" : "Cornea",
31     "Forceps" : "Forceps",
32     "I-A Handpiece" : "I-A Hand.",
33     "Lens Injector" : "Lens Inj.",
34     "Phaco Handpiece" : "Phaco H.",
35     "Primary Knife" : "Prim. Kn.",
36     "Pupil" : "Pupil",
37     "Second Instrument" : "Sec. Inst.",
38     "Secondary Knife" : "Sec. Kn.",
39 }
```



```
۳۹
۴۰ # step 1: load the data
۴۱ def load_split(split: str) -> pd.DataFrame:
۴۲     csv_path = DATASET_ROOT / split / CSV_NAME
۴۳
۴۴     # double check if file exists
۴۵     if not csv_path.exists():
۴۶         raise FileNotFoundError(f"Missing CSV file at: {csv_path.resolve()}")
۴۷
۴۸     # load csv as pandas dataframe
۴۹     df = pd.read_csv(csv_path)
۵۰
۵۱     # remove extra spaces
۵۲     df.columns = [c.strip() for c in df.columns]
۵۳
۵۴     df.rename(columns={df.columns[0]: "filename"}, inplace=True)
۵۵
۵۶     # new column for separating data into train, valid & test
۵۷     df["split"] = split
۵۸
۵۹     return df
۶۰
۶۱ # step 2: merge data
۶۲ print("-> loading data")
۶۳
۶۴ # store each split dataframe into a dict
۶۵ dfs = {s: load_split(s) for s in SPLITS}
۶۶
۶۷ # stack the 3 splits into one dataframe
۶۸ df_all = pd.concat(dfs.values(), ignore_index=True)
۶۹
۷۰ # set of the non-tool columns (like cornea & pupil)
۷۱ META_COLS = {"filename", "split"}
۷۲
۷۳ # list of the tool columns
۷۴ CLASS_COLS = [c for c in df_all.columns if c not in META_COLS]
۷۵ # count of all tool classes
۷۶ N = len(CLASS_COLS)
۷۷ # use the name in SHORT dictionary
۷۸ SLABELS = [SHORT.get(c, c) for c in CLASS_COLS]
```



```
۷۹
۸۰ # count the num of tools in each image
۸۱ df_all["n_labels"] = df_all[CLASS_COLS].sum(axis=1).astype(int)
۸۲ # if no tools in the image = true (neg frame)
۸۳ df_all["is_negative"] = df_all["n_labels"] == 0
۸۴
۸۵ # remove this
۸۶ #df_all["source_type"] =
→ df_all["filename"].str.extract(r'^(Video\d+/case\d+)'[0].fillna("other")
۸۷
۸۸ # reseed df_all into dictionaries for each split
۸۹ for s in SPLITS:
۹۰     dfs[s] = df_all[df_all["split"] == s].copy()
۹۱
۹۲ # colors for plots
۹۳ CMAP = plt.cm.get_cmap("tab20", N)
۹۴ BAR_COLS = [CMAP(i) for i in range(N)]
۹۵ SP_COLORS = {"train": "#4C72B0", "valid": "#DD8452", "test": "#55A868"}
۹۶
۹۷ # SECTION 2*: TEXT
۹۸ # SURGICAL TOOLS ONLY (CORNEA AND PUPIL EXCLUDED)
۹۹
۱۰۰ # update class list
۱۰۱ CLASS_COLS = [c for c in CLASS_COLS if c not in ["Cornea", "Pupil"]]
۱۰۲ N = len(CLASS_COLS)
۱۰۳
۱۰۴ # recalculate num of imgs and neg frames
۱۰۵ df_all["n_labels"] = df_all[CLASS_COLS].sum(axis=1).astype(int)
۱۰۶ df_all["is_negative"] = df_all["n_labels"] == 0
۱۰۷
۱۰۸ # new values for each split
۱۰۹ for s in SPLITS:
۱۱۰     dfs[s] = df_all[df_all["split"] == s].copy()
۱۱۱
۱۱۲ # visual
۱۱۳ DIV = " " * 65
۱۱۴
۱۱۵ print(DIV)
۱۱۶ print(f"      (A) OVERVIEW:  ")
۱۱۷ print(DIV)
```



```

۱۱۸
۱۱۹ # count total images
۱۲۰ total = len(df_all)
۱۲۱ print(f"total images: {total:,}")
۱۲۲
۱۲۳ # num of classes
۱۲۴ print(f"class columns: {N}")
۱۲۵ print(f"name of classes: {CLASS_COLS}")
۱۲۶ print()
۱۲۷ print("split | images | percentage | negative frames | avg labels")
۱۲۸ print(DIV)
۱۲۹
۱۳۰ # loop that repeats 3 times for train, valid & test
۱۳۱ for s in SPLITS:
۱۳۲     d = dfs[s]
۱۳۳     n = len(d)
۱۳۴
۱۳۵     # num of neg frames
۱۳۶     neg = d["is_negative"].sum()
۱۳۷
۱۳۸     # avg num of tools in images
۱۳۹     avg = d["n_labels"].mean()
۱۴۰
۱۴۱     # calculate percentages
۱۴۲     pct = (n / total) * 100
۱۴۳     neg_pct = (neg / n) * 100
۱۴۴
۱۴۵     print(f"{s} | {n} | {pct:.1f}% | {neg} ({neg_pct:.0f}%) | {avg:.2f}")
۱۴۶
۱۴۷ print()

```

این کد پس از خواندن داده‌ها، مجموع تعداد عکس‌ها، تعداد و اسم کلاس‌ها را محاسبه کرده و سپس تعداد تصاویر بدون ابزار (Negative Frames)، سهم هر کدام از داده‌های test، validation و train را به نسبت کل داده‌ها تعیین می‌کند. در نهایت میانگین تعداد برچسب در فریم‌های هر بخش داده را محاسبه می‌کند. خروجی کد به صورت زیر است

(A) OVERVIEW:

total images: 6,962


```
class columns: 10
name of classes: ['Cannula', 'Cap Cystotome', 'Cap Forceps', 'Forceps', 'I-A Handpiece', 'Lens Injector']

split | images | percentage | negative frames | avg labels

train | 5136 | 73.8% | 527 (10%) | 1.12
valid | 991 | 14.2% | 100 (10%) | 1.16
test | 835 | 12.0% | 121 (14%) | 1.07
```

و نشان می‌دهد، مجموعه داده مورد بررسی شامل ۶۹۶۲ تصویر از جراحی کاتاراکت است. این داده‌ها به سه بخش train (73.8%)، validate (14.2%) و test (12.0%) تقسیم شده‌اند. نکته آماری حائز اهمیت در این فایل، وجود فریم‌های منفی (Negative Frames) است که در آن‌ها هیچ ابزاری دیده نمی‌شود. این فریم‌ها 10% از بخش آموزش، 10% از بخش ارزیابی و 14% از بخش تست را به خود اختصاص داده‌اند. در بینایی ماشین، وجود فریم‌های منفی برای کاهش خطای مثبت کاذب (False Positive Rate) بسیار حیاتی است، زیرا به مدل آموزش می‌دهد که در غیاب ابزار، برچسب اشتباهی صادر نکند.

بررسی توزیع برچسب‌ها

در قسمت بعدی، توزیع برچسب‌ها یا به عبارتی توزیع تعداد ابزارها در هر فریم با کد زیر بررسی شده است.

```
1 print(DIV)
2 print(f"      (B) LABEL-COUNT DISTRIBUTION:  ")
3 print(DIV)
4 for s in SPLITS:
5     lc = dfs[s]["n_labels"].value_counts().sort_index()
6     print(f"\n  [{s}]")
7
8     # print raw num and percentage
9     for k, v in lc.items():
10         pct = (v / len(dfs[s])) * 100
11         print(f"  {k} tool(s): {v} ({pct:.1f}%)")
12
13 print()
```

خروجی کد به صورت زیر است

(B) LABEL-COUNT DISTRIBUTION:

[train]

0 tool(s): 527 (10.3%)
1 tool(s): 3463 (67.4%)
2 tool(s): 1146 (22.3%)

[valid]

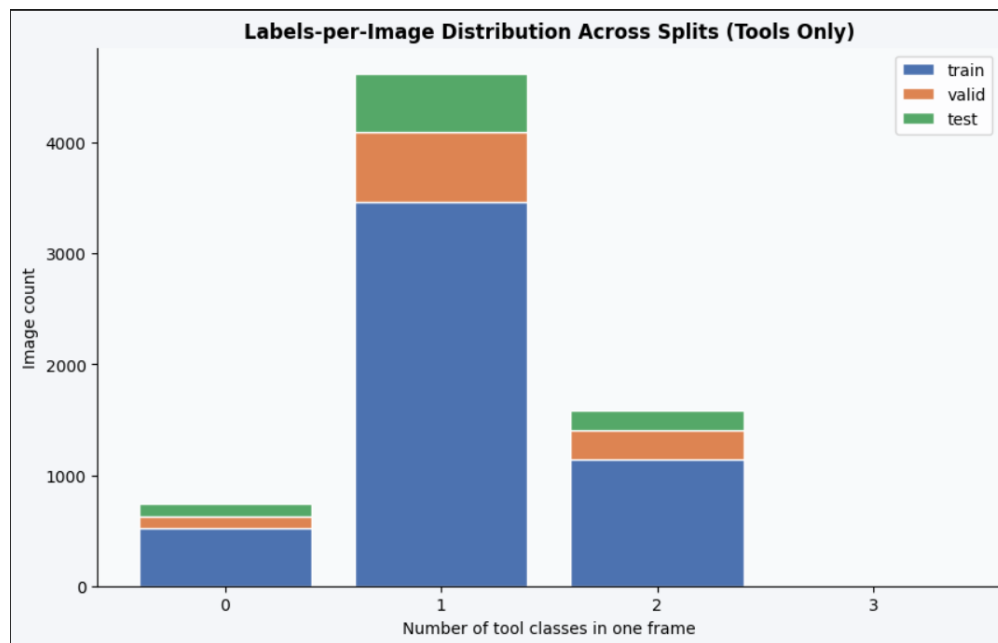
0 tool(s): 100 (10.1%)
1 tool(s): 629 (63.5%)
2 tool(s): 262 (26.4%)

[test]

0 tool(s): 121 (14.5%)
1 tool(s): 532 (63.7%)
2 tool(s): 181 (21.7%)
3 tool(s): 1 (0.1%)

و نشان‌دهنده چگالی کم برچسب‌ها در تصاویر است. از نظر آماری:

- 0 tool(s): فریم‌های منفی که بین 10.1% تا 14.5% داده‌ها را در بخش‌های مختلف تشکیل می‌دهند.
 - 1 tool(s): بالاترین فراوانی آماری را دارد و حدود 63.5% تا 67.4% تصاویر را شامل می‌شود. این وضعیت نشان می‌دهد که در بیشتر مراحل جراحی، تنها یک ابزار به طور فعال در میدان دید دوربین قرار دارد.
 - 2 tool(s): بین 21.7% تا 26.4% تصاویر را شامل می‌شود که بازتاب‌دهنده جراحی دو دستی است.
 - 3 tool(s): بسیار نادر است و تنها یک فریم (0.1%) در مجموعه تست آن را ثبت کرده است.
- نمودار این داده‌ها در شکل ۴ قابل مشاهده است.



شکل ۱: نمودار توزیع برچسب‌ها (بدون کلاس‌های آناتومی)

ماتریس هم‌رخدادی Co-occurrence Matrix

با استفاده از کد زیر محاسبه می‌شود:

```

۱ # co-occurrence matrix
۲ # how often tools are used together
۳ co = np.zeros((N, N), dtype=int)
۴ label_mat = dfs["train"][CLASS_COLS].values
۵
۶ # loop that repeats for all imgs (rows of the .csv)
۷ for row in label_mat:
۸     # finds all 1s
۹     idxs = np.where(row == 1)[0]
۱۰    # finds all possible combos (intersection points)
۱۱    for a, b in combinations(idxs, 2):
۱۲        co[a, b] += 1; co[b, a] += 1
۱۳
۱۴ # final dataframe
۱۵ co_df = pd.DataFrame(co, index=CLASS_COLS, columns=CLASS_COLS)
۱۶
۱۷ print(DIV)
۱۸ print(co_df.to_string())
۱۹

```



۲۰ print (DIV)

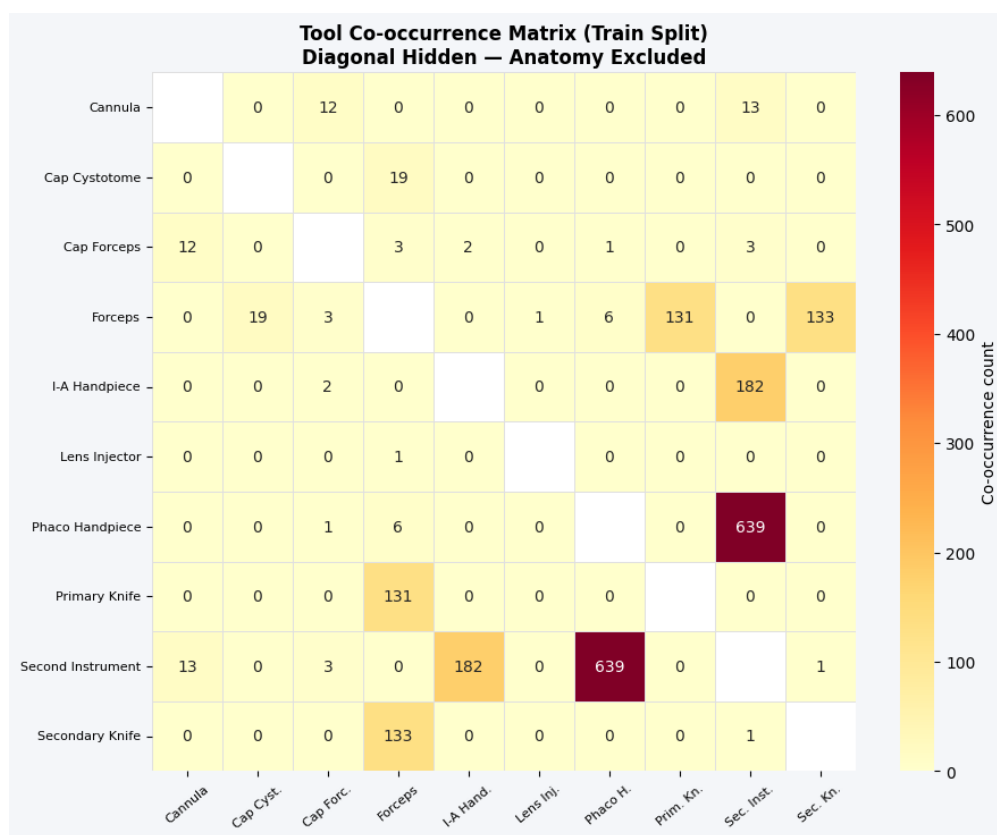
خروجی آن به صورت زیر است:

	Cannula	Cap Cystotome	Cap Forceps	Forceps	I-A Handpiece	Lens Injector	Phaco Handpiece	Primary Knife	Second Instrument	Secondary Knife
Cannula	0	0	12	0	0	0	0	0	13	0
Cap Cystotome	0	0	0	19	0	0	0	0	0	0
Cap Forceps	12	0	0	3	2	0	1	0	3	0
Forceps	0	19	3	0	0	1	6	131	0	133
I-A Handpiece	0	0	2	0	0	0	0	0	182	0
Lens Injector	0	0	0	1	0	0	0	0	0	0
Phaco Handpiece	0	0	1	6	0	0	0	0	639	0
Primary Knife	0	0	0	131	0	0	0	0	0	0
Second Instrument	13	0	3	0	182	0	639	0	0	1
Secondary Knife	0	0	0	133	0	0	0	0	1	0

شکل ۲: خروجی کد (ماتریس هم‌رخدادی)

ماتریس هم‌رخدادی در این ساختار، الگوهای تعامل ابزارها با یکدیگر را بدون در نظر گرفتن ساختار چشم نشان می‌دهد:

- هم‌رخدادی غالب: بیشترین هم‌زمانی مربوط به جفت Phaco Handpiece و Second Instrument با 639 مرتبه تکرار است. این داده بازتاب‌دهنده فاز Phacoemulsification در جراحی کاتاراکت است که در آن هر دو دست جراح فعال هستند.
 - هم‌رخدادی‌های ثانویه: جفت‌های Second Instrument و I-A Handpiece با 182 بار تکرار و همچنین Forceps و Secondary Knife با 133 بار تکرار پیوندهای فرآیندی مهمی را نشان می‌دهند.
 - وابستگی‌های صفر (Zero Co-occurrences): بسیاری از ابزارها دارای نرخ هم‌رخدادی صفر با یکدیگر هستند. به عنوان مثال، Pri-mary Knife و Phaco Handpiece هرگز با هم ظاهر نشده‌اند. این ویژگی آماری به مدل اجازه می‌دهد تا در لایه‌های تصمیم‌گیری نهایی، احتمال حضور هم‌زمان دو ابزار غیرمرتبط را فیلتر کند.
- این ماتریس به صورت نمودار (با حذف کردن قطر اصلی) در شکل ۳ قابل مشاهده است.



شکل ۳: ماتریس هم‌رخدادی (بدون کلاس‌های آناتومی)

رویکرد دوم (با کلاس‌های آناتومی)

کدهای این بخش مشابه بخش قبل هستند و تنها تفاوت‌هایی جزئی دارند. بنابراین تنها خروجی کدها در این بخش مورد بررسی قرار گرفته است.

اطلاعات جامع در مورد داده‌ها
خروجی به شکل زیر می‌باشد:

(A) OVERVIEW:

total images: 6,962

class columns: 12

name of classes: ['Cannula', 'Cap Cystotome', 'Cap Forceps', 'Cornea', 'Forceps', 'I-A Handpiece', 'Lens Injector', 'Phaco Handpiece', 'Primary Knife', 'Second Instrument', 'Secondary Knife', 'Tissue']

split | images | percentage | negative frames | avg labels



```
train | 5136 | 73.8% | 0 (0%) | 3.07
valid | 991 | 14.2% | 0 (0%) | 3.13
test | 835 | 12.0% | 0 (0%) | 3.06
```

این مجموعه داده با اضافه شدن دو کلاس آناتومیک کلیدی چشم، یعنی Pupil و Cornea، به مجموع 12 کلاس مقیاس پذیری یافته است. تعداد فریم‌ها کاملاً مشابه فایل اول (6,962) است، اما تغییرات آماری در توزیع برچسب‌ها رخ داده است:

- Train: شامل 5136 تصویر (73.8%) با میانگین 3.07 برچسب در هر فریم.
- Validation: شامل 991 تصویر (14.2%) با میانگین 3.13 برچسب در هر فریم.
- Test: شامل 835 تصویر (12.0%) با میانگین 3.06 برچسب در هر فریم.

تفاوت بنیادین در این فایل، کاهش درصد فریم‌های منفی به 0% در تمامی بخش‌ها است. از منظر آمار و بینایی ماشین، وقتی دو جزء ثابت آناتومیک که تقریباً در تمام فریم‌های ویدیو جراحی حضور دارند به سیستم معرفی می‌شوند، دیگر هیچ فریم خالی یا منفی وجود نخواهد داشت و میانگین تعداد برچسب‌ها از حدود 1 به بالای 3 افزایش می‌یابد.

بررسی توزیع برچسب‌ها
خروجی به صورت زیر است:

(B) LABEL-COUNT DISTRIBUTION:

```
[train]
1 class(es): 115 (2.2%)
2 class(es): 556 (10.8%)
3 class(es): 3343 (65.1%)
4 class(es): 1122 (21.8%)
```

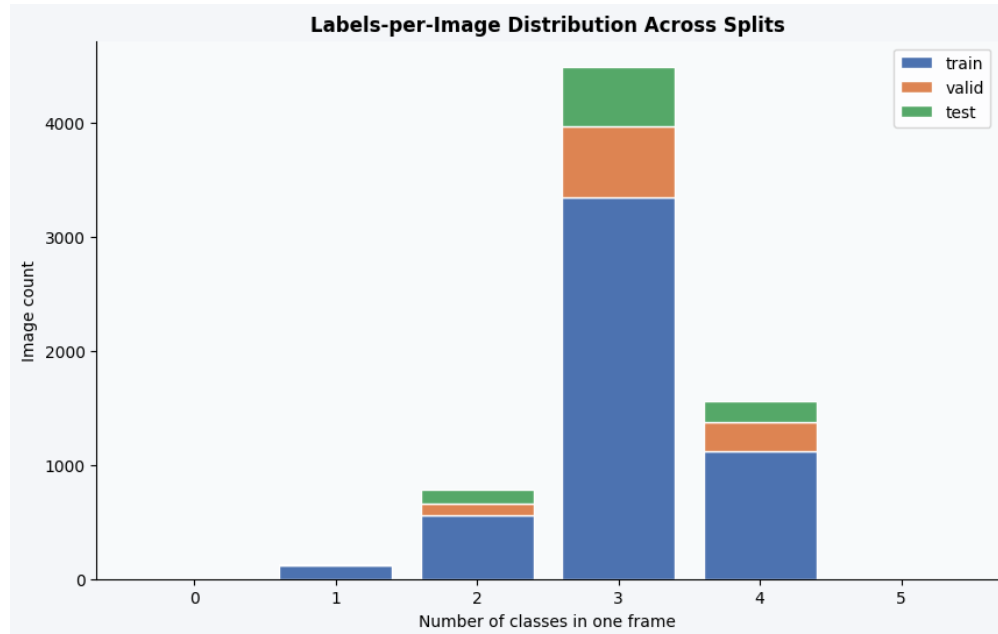
```
[valid]
1 class(es): 11 (1.1%)
2 class(es): 104 (10.5%)
3 class(es): 619 (62.5%)
4 class(es): 257 (25.9%)
```

```
[test]
1 class(es): 5 (0.6%)
```

```
2 class(es): 124 (14.9%)
3 class(es): 526 (63.0%)
4 class(es): 179 (21.4%)
5 class(es): 1 (0.1%)
```

واضح است که توزیع کلاسی در هر فریم به شدت تحت تاثیر کلاس‌های آناتومیک قرار گرفته است:

- **class(es): 1:** سهم بسیار ناچیزی دارد (0.6% تا 2.2%). این فریم‌ها نشان‌دهنده موقعیت‌هایی هستند که به دلیل زوم شدید یا جابجایی دوربین، تنها یکی از ساختارهای قرنیه یا مردمک بدون هیچ ابزاری دیده شده است.
 - **class(es): 2:** بین 10.5% تا 14.9% داده‌ها را شامل می‌شود. این فریم‌ها فاقد ابزار بوده و فقط شامل Cornea و Pupil هستند (معادل همان فریم‌های منفی فایل اول).
 - **class(es): 3:** بالاترین فراوانی را با سهمی حدود 62.5% تا 65.1% دارد. این مقدار نمایانگر حالت استاندارد جراحی است: قرنیه + مردمک + یک ابزار جراحی فعال.
 - **class(es): 4:** بین 21.4% تا 25.9% فریم‌ها را تشکیل می‌دهد که نشان‌دهنده حضور هم‌زمان دو ابزار در کنار قرنیه و مردمک است.
- نمودار این داده‌ها در شکل ۴ قابل مشاهده است.



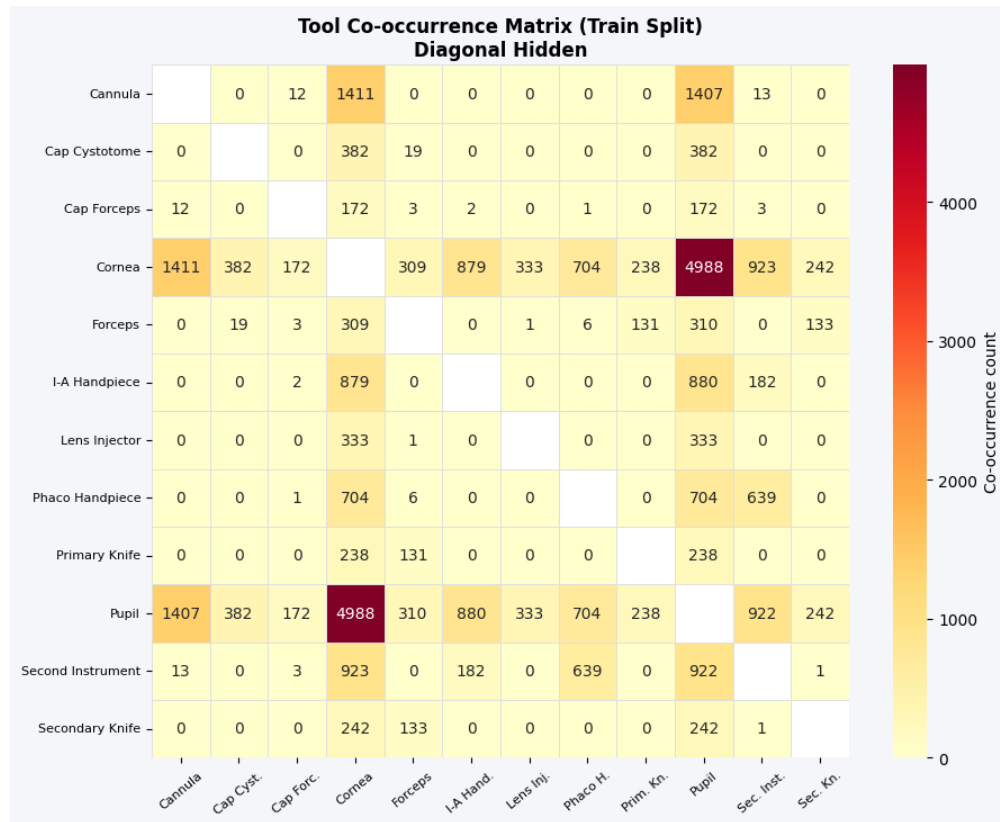
شکل ۴: نمودار توزیع برچسب‌ها (با کلاس‌های آناتومی)

ماتریس هم‌رخدادی **Co-occurrence Matrix**

این ماتریس (در شکل ۵) اطلاعات زیر را در اختیار ما می‌گذارد:



- تعاملات آناتومیک: هم‌رخدادی بین دو کلاس Cornea و Pupil برابر با 4,988 فریم در داده‌های آموزش است. این همبستگی بسیار بالا مبرهن می‌سازد که این دو ساختار فضا-زمانی کاملاً به یکدیگر وابسته هستند.
- نگاشت ابزار به آناتومی: تمام ابزارهای جراحی نرخ هم‌رخدادی تقریباً کاملی با قرنیه و مردمک دارند. به عنوان مثال، کلاس Cannula از مجموع 1457 حضور خود در فاز آموزش، در 1411 فریم با Cornea و در 1407 فریم با Pupil هم‌رخداد بوده است. این به مدل هوش مصنوعی یاد می‌دهد که ابزارها همواره در بستر آناتومیک چشم کار می‌کنند.
- حفظ روابط ابزار-ابزار: الگوهای هم‌رخدادی بین ابزارها بدون تغییر باقی مانده است؛ جفت Phaco Handpiece و Second Instrument کماکان با 639 رخداد هم‌زمانی، فعال‌ترین زوج ابزاری جراحی هستند.



شکل ۵: نمودار ماتریس هم‌رخدادی (با کلاس‌های آناتومی)

۲.۳.۱ بررسی توازن کلاس‌ها (Class Imbalance)

بدون کلاس‌های آناتومی

برای بررسی توازن کلاس‌ها، درصد فراوانی هر یک را محاسبه کرده و به صورت مقابل طبقه‌بندی می‌کنیم: بالای ۱۵ درصد خوب (OK)، بین ۱۰ تا ۱۵ درصد متوسط (moderate) و زیر ۱۰ درصد شدید (severe).
با استفاده از کد زیر فراوانی کلاس‌ها را به دست آورده و آن‌ها را طبقه‌بندی می‌کنیم:

```
print()
```




```
۲ print(DIV)
۳ print(f"      (C) PER-CLASS FREQUENCY:      ")
۴ print(DIV)
۵
۶ print("Class | Train | Val | Test | Total | % Total | Severity")
۷ print(DIV)
۸
۹ # loop that repeats for each class
۱۰ for cls in CLASS_COLS:
۱۱
۱۲     # per split values
۱۳     train_num = int(dfs["train"][cls].sum())
۱۴     valid_num = int(dfs["valid"][cls].sum())
۱۵     test_num = int(dfs["test"][cls].sum())
۱۶
۱۷     # adds per split values
۱۸     tot = train_num + valid_num + test_num
۱۹
۲۰     # (data set that has this tool/ total data set)%
۲۱     pct = tot / total * 100
۲۲
۲۳     # imbalance label
۲۴     # > 15 (ok), 10 < < 15 (mod.), < 10 (sev.)
۲۵     if pct >= 15:
۲۶         sev = "OK"
۲۷     elif pct >= 10:
۲۸         sev = "moderate"
۲۹     else:
۳۰         sev = "severe"
۳۱
۳۲     print(f"{cls} | {train_num} | {valid_num} | {test_num} | {tot} | {pct:.1f}% | {sev}")
۳۳
۳۴ # num of appearances of each class in train
۳۵ train_counts = np.array([dfs["train"][c].sum() for c in CLASS_COLS], dtype=float)
۳۶
۳۷ # imbalance ratio (most freq/least freq)
۳۸ max_val = train_counts.max()
۳۹ min_val = max(train_counts.min(), 1) # prevents dividing by zero if a class has 0 counts
۴۰ ratio = max_val / min_val
۴۱
```



```
print(f"\nmax-to-min imbalance ratio (in train): {max_val:.0f} / {min_val:.0f} = {ratio:.1f}x")
```

خروجی کد به صورت زیر خواهد بود:

(C) PER-CLASS FREQUENCY:

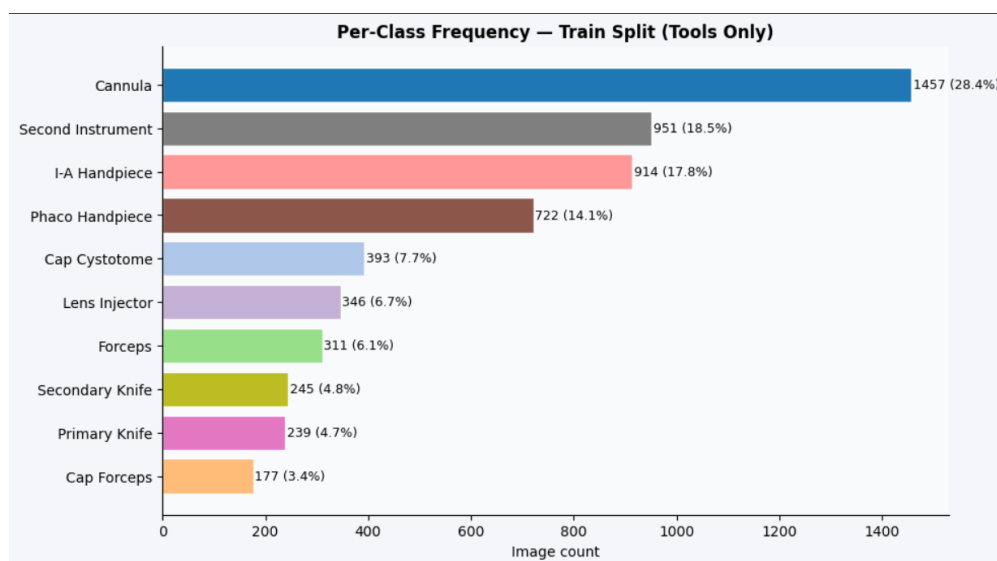
Class	Train	Val	Test	Total	% Total	Severity
Cannula	1457	237	226	1920	27.6%	OK
Cap Cystotome	393	72	59	524	7.5%	severe
Cap Forceps	177	30	21	228	3.3%	severe
Forceps	311	36	66	413	5.9%	severe
I-A Handpiece	914	213	105	1232	17.7%	OK
Lens Injector	346	64	64	474	6.8%	severe
Phaco Handpiece	722	181	100	1003	14.4%	moderate
Primary Knife	239	37	60	336	4.8%	severe
Second Instrument	951	244	149	1344	19.3%	OK
Secondary Knife	245	39	47	331	4.8%	severe

max-to-min imbalance ratio (in train): 1457 / 177 = 8.2x

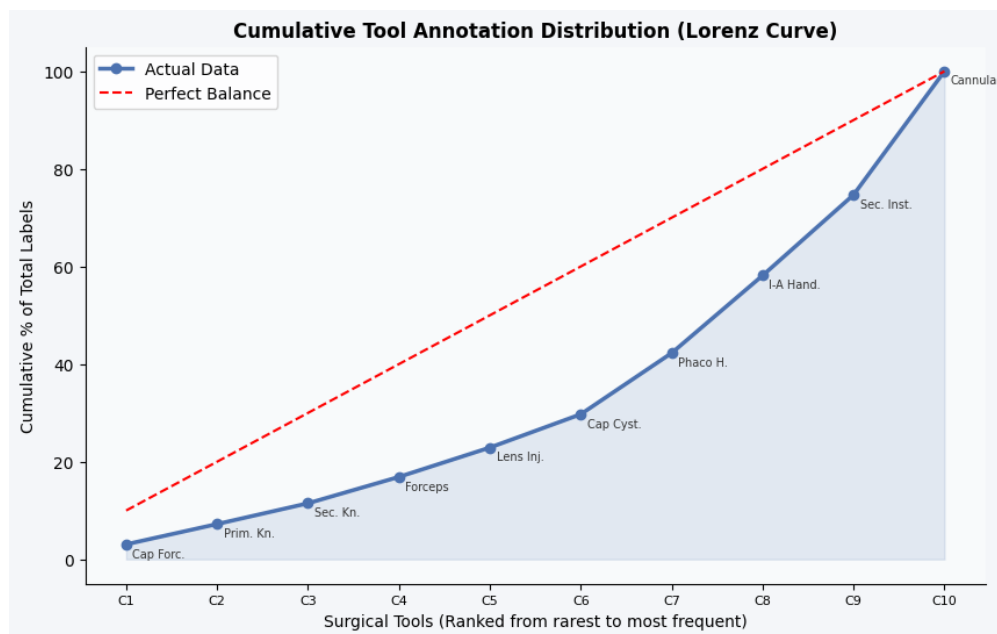
سه کلاس اکثریت یعنی Cannula (با 27.6%)، Second Instrument (با 19.3%) و I-A Handpiece (با 17.7%) بخش اعظم داده‌ها را به خود اختصاص داده‌اند. کلاس‌های کم‌یاب شامل ابزارهایی مانند Cap Forceps (با 3.3%) و Primary Knife (با 4.8%) در انتهای نمودار قرار دارند. این موضوع یک هشدار آماری برای مدل محسوب می‌شود؛ زیرا شبکه ممکن است در تشخیص این ابزارهای حیاتی دچار سوگیری منفی شود. نسبت حداکثر به حداقل فراوانی (Imbalance Ratio) در داده‌های train برابر با $8.2x = \frac{1457}{177}$ است. این ناهمگونی 8.2 برابری بدین معناست که مدل در حین بهینه‌سازی توابع هدف، به سمت کلاس‌های اکثریت (Cannula) متمایل خواهد شد. برای جبران این پدیده آماری، استفاده از روش‌های مناسب افزودن داده الزامی است.

برای این بخش دو نمودار شکل ۸ و ۹ رسم شده‌اند که عدم توازن داده‌ها را به خوبی نمایش می‌دهند.

در منحنی لورنتز، خط چین قرمز نشان‌دهنده یک الگوی ایده‌آل توزیع یکنواخت داده است. انحراف شدید منحنی آبی (داده‌های فعلی ما) از این خط، شدت نابرابری را تایید می‌کند. همان‌طور که در نمودار مشهود است، ۶ کلاس اول جراحی (از Cap Cyst. تا Cap Forc.) روی هم‌رفته تنها حدود ۳۰ درصد از کل برچسب‌ها را تشکیل می‌دهند. در مقابل، بازه تند منحنی نشان می‌دهد که تنها ۴ کلاس برتر (Cannula، Sec. Inst.، I-A Hand.، Phaco H.) ۷۰ درصد باقی‌مانده از حجم کل برچسب‌ها را تشکیل می‌دهند. این منحنی اثبات می‌کند که در این سناریو استفاده از توابع زیان استاندارد مانند Cross-Entropy بدون اعمال وزن کلاسی، قطعاً شکست مدل را به همراه خواهد داشت.



شکل ۶: فراوانی داده در هر کلاس (بدون کلاس‌های آناتومی)



شکل ۷: منحنی لورنتز (بدون کلاس‌های آناتومی)

با کلاس‌های آناتومی
خروجی کد به صورت زیر است:

(C) PER-CLASS FREQUENCY:

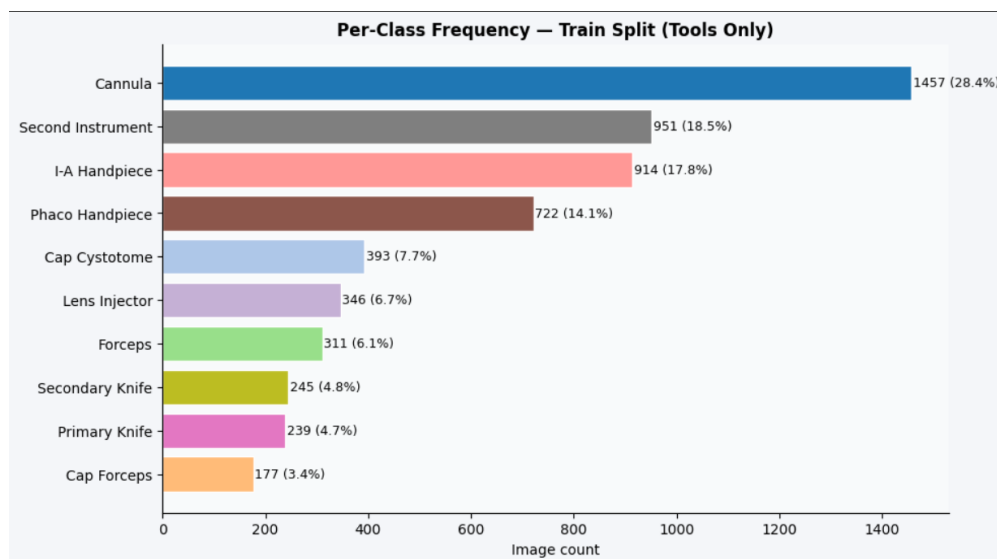


Class | Train | Val | Test | Total | % Total | Severity

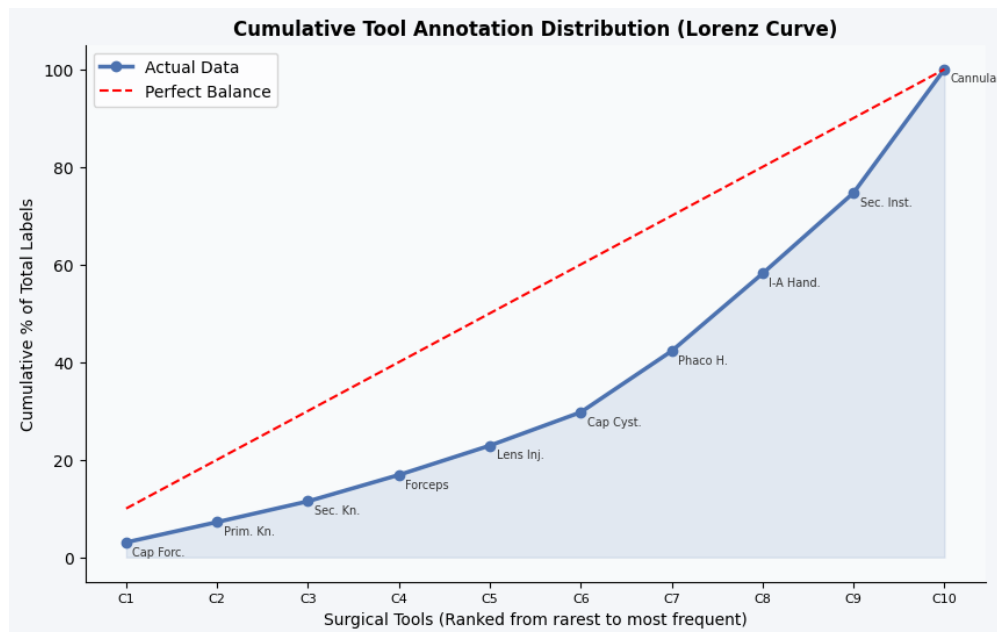
Cannula | 1457 | 237 | 226 | 1920 | 27.6% | moderate
 Cap Cystotome | 393 | 72 | 59 | 524 | 7.5% | severe
 Cap Forceps | 177 | 30 | 21 | 228 | 3.3% | severe
 Cornea | 4997 | 976 | 828 | 6801 | 97.7% | OK
 Forceps | 311 | 36 | 66 | 413 | 5.9% | severe
 I-A Handpiece | 914 | 213 | 105 | 1232 | 17.7% | moderate
 Lens Injector | 346 | 64 | 64 | 474 | 6.8% | severe
 Phaco Handpiece | 722 | 181 | 100 | 1003 | 14.4% | moderate
 Primary Knife | 239 | 37 | 60 | 336 | 4.8% | severe
 Pupil | 4992 | 975 | 827 | 6794 | 97.6% | OK
 Second Instrument | 951 | 244 | 149 | 1344 | 19.3% | moderate
 Secondary Knife | 245 | 39 | 47 | 331 | 4.8% | severe

max-to-min imbalance ratio (in train): $4997 / 177 = 28.2x$

مشاهده میشود که ورود کلاس‌های آناتومیک، توزیع فراوانی را به طور کامل تغییر داده است: دو کلاس Cornea (97.7%) و Pupil (97.6%) در بالاترین سطح فراوانی قرار دارند و وضعیت آن‌ها OK است. فراوانی ابزارهای جراحی در این فایل دقیقاً مشابه فایل اول است (برای مثال Cannula با 1457 فریم آموزش)، اما وضعیت شدت نابرابری ابزارها به دلیل مقایسه با کلاس‌های آناتومیک تغییر کرده و کلاس‌هایی مانند Cannula از وضعیت OK به moderate تبدیل شده‌اند. Imbalance Ratio: نسبت حداکثر به حداقل فراوانی در داده‌های آموزش به شدت جهش یافته و به $\frac{4997}{177} = 28.2x$ تغییر پیدا کرده است. این نابرابری 28.2 برابری یک چالش آماری بسیار بزرگ برای مدل ایجاد می‌کند. لایه‌های کانولوشنی شبکه متمایل به پیش‌بینی دائمی Cornea و Pupil خواهند بود. در اینجا نیز دو نمودار لورنتز (شکل؟؟) و نمودار توزیع داده (؟؟) رسم شده‌اند.



شکل ۸: فراوانی داده در هر کلاس (با کلاس‌های آناتومی)



شکل ۹: منحنی لورنتز (با کلاس‌های آناتومی)

۳.۳.۱ روش‌های پیشنهادی افزایش داده

در سناریوی جراحی آب مروارید، وظیفه‌ی اصلی تشخیص ابزارهای جراحی با طبقه‌بندی multi-label در فریم‌های ویدئویی عمل آب مروارید است. مجموعه‌داده شامل ۶۹۶۲ تصویر با ابعاد 640×640 پیکسل و ۱۲ کلاس ابزار است. تحلیل آماری این مجموعه از داده دو چالش اساسی را آشکار کرد:



- عدم تعادل شدید کلاسی: نسبت فراوانی بیشینه به کمینه در مجموعه‌ی آموزشی از مرز $10 \times$ فراتر می‌رود؛ کلاس‌هایی نظیر Cannula و Secondary Knife در کمتر از ۱۰٪ از تصاویر حضور دارند.
- ماهیت بصری محیط جراحی: حضور بافت‌های زنده، خون، بازتاب نور تجهیزات، و انسداد جزئی (occlusion) ابزارها توسط یکدیگر، توزیع پیکسلی تصاویر را به شدت متغیر می‌کند.
- بنابراین انتخاب روش افزایش داده باید از دو نظر چالش‌های بصری و ساختار داده بررسی شده و محدودیت‌های لازم در جراحی آب‌مروارید نیز در نظر گرفته شود.

۱. چالش‌های بصری و روش بهبود آن‌ها

- انسداد (Occlusion): در فریم‌های جراحی، ابزارها غالباً به صورت جزئی توسط بافت، خون، یا سایر ابزارها پوشانده می‌شوند. ماتریس هم‌حضور کلاس‌ها نشان می‌دهد که Phaco Handpiece و I-A Handpiece اغلب همزمان در صحنه حضور دارند و فضای دید یکدیگر را محدود می‌کنند. بنابراین مدل باید قادر باشد یک ابزار را از روی بخشی از شکل آن شناسایی کند. روش CoarseDropout (معادل Cutout) که در آن مجموعه‌ای از نواحی مستطیلی به صورت تصادفی با مقدار صفر (سیاه) پوشانده می‌شوند، می‌تواند برای بهبود این مسئله استفاده شود. این روش مدل را مجبور می‌کند به ویژگی‌های باقیمانده‌ی ابزار متکی باشد، نه الگوی کامل آن. روش دیگری که می‌تواند مورد استفاده قرار گیرد GridMask است که الگوی شبکه‌ای منظمی از پیکسل‌ها را پنهان می‌کند و نوعی انسداد ریز و متناوب را شبیه‌سازی می‌کند.
- تغییرپذیری مقیاس و موضع ابزار: ابزارها در فریم‌های مختلف در مقیاس‌ها و موضع‌های متفاوتی ظاهر می‌شوند و در حین جراحی گاهی به دوربین نزدیک‌ترند (بزرگ‌تر دیده می‌شوند) و گاهی دورترند (کوچک‌تر دیده می‌شوند).
- روش RandomResizedCrop با پارامتر مقیاس مناسب به مدل یاد می‌دهد که اندازه ابزار در تشخیص آن اثری نداشته باشد و جایگاه‌های مختلف ابزار در صفحه را بشناسد. این روش ابتدا یک بخش تصادفی از تصویر برش خورده و سپس بخش برش خورده را دوباره بزرگ می‌کند تا به اندازه اصلی تصویر (640×640) برسد.
- تغییرات نوری و بازتاب: نور تجهیزات بازتاب‌های شدیدی روی سطح ابزارهای فلزی ایجاد می‌کند و روشنایی صحنه در طول عمل تغییر می‌کند. روش ColorJitter با پارامترهای محدود مدل را در برابر تغییرات نوری مقاوم می‌کند. این روش مقادیر روشنایی (brightness)، کنتراست (contrast)، اشباع (saturation) تصویر را به صورت تصادفی و اندک تغییر می‌دهد. اگر لازم باشد در رنگ (hue) نیز تغییرات اعمال شود، دامنه تغییرات باید بسیار محدود انتخاب شوند؛ چرا که رنگ بافت و خون اهمیت تشخیصی بالایی دارد و نباید تغییر غیر واقعی کند.
- تاری حرکتی و خارج از فوکوس: ماهیت ویدئویی داده‌ها به این معناست که برخی فریم‌ها دارای تاری ناشی از حرکت دست جراح یا تنظیم فوکوس دوربین هستند. می‌توان با اعمال GaussianBlur مات‌شدگی را شبیه‌سازی کرد تا مدل در مواجهه با فریم‌های تاری ویدئو ابزارها را گم نکند.
- تقارن افقی: در جراحی چشم، جراح ممکن است از سمت چپ یا راست کار کند و ساختار آناتومیک چشم متقارن است. این کار بدون اینکه معنای تصویر را خراب کند یا برچسب‌ها را تغییر دهد، تنوع جهات ورود ابزار را دو برابر می‌کند. بنابراین با استفاده از HorizontalFlip و آینه کردن تصویر این مشکل حل می‌شود.



• چرخش‌های کوچک و محدود: دوربین اندوسکوپی معمولاً در زاویه‌ای ثابت نصب شده است؛ بنابراین چرخش‌های بزرگ در یک جراحی واقعی دیده نمی‌شود. اما چرخش‌های کوچک و لرزش دوربین ممکن است. بنابراین با روش Rotate به اندازه چند درجه محدود، می‌توان این مسئله را شبیه‌سازی کرد تا مدل آن‌ها را یاد بگیرد.

۲. چالش‌های مربوط به ساختار داده و عدم تعادل کلاس‌ها

با توجه به ماهیت چندبرچسبی مسئله و هم‌حضور بالایی برخی کلاس‌ها، MixUp می‌تواند مرزهای تصمیم‌گیری را نرم‌تر (Label Smoothing) کند و مانع از این می‌شود که مدل بر کلاس‌های اکثریت بیش از حد اطمینان کند. این کار به شناخت کلاس‌های نادر کمک زیادی می‌کند. این روش، دو تصویر مختلف از مجموعه داده را برمی‌دارد و آن‌ها را به صورت شفاف روی هم می‌اندازد (Linear Combination) و برچسب‌های آن‌ها را هم با همان نسبت با هم ترکیب می‌کند.

۳. بررسی محدودیت‌های جراحی

محدودیت‌های موجود در حوزه پزشکی و سناریو جراحی آب‌مروارید، استفاده از برخی روش‌ها را غیرمجاز می‌کند. چهار مورد از این روش‌ها به شرح زیر می‌باشند:

- چرخش بزرگ (بیش از ۱۵ درجه): میدان جراحی از منظر دوربینی با زاویه‌ی ثابت تصویربرداری می‌شود؛ چرخش زیاد داده‌های غیرواقعی تولید می‌کند.
- وارونگی عمودی (Vertical Flip): از نظر آناتومیک معتبر نیست؛ ساختار چشم و جایگاه ابزارها در صحنه‌ی جراحی دارای جهت‌گیری طبیعی هستند.
- تغییر شدید hue: رنگ ابزارهای جراحی و بافت‌های بیولوژیک دارای معنای تشخیصی است؛ تغییر رنگ زیاد این اطلاعات را از بین می‌برد.
- تبدیل به grayscale: تمایز بین خون، بافت، و ابزار فلزی به اطلاعات رنگی وابسته است.

۲ پرسش دوم: طراحی معماری، تنظیم فرآیند آموزش و تفسیرپذیری مدل (XAI)

در این تمرین، مسئله ما یک طبقه‌بندی Multi-label شامل ۱۲ کلاس مختلف از ابزارهای جراحی آب‌مروارید است. هر تصویر می‌تواند به‌طور همزمان شامل چند ابزار باشد. بررسی نمودارهای توزیع داده در مجموعه‌های Train، Validation و Test نشان می‌دهد که دیتاست به‌شدت نامتوازن (Severely Imbalanced) است.

به‌طور مشخص، کلاس‌های Cornea و Pupil تقریباً در اکثر تصاویر حضور دارند، در حالی که ابزارهایی مانند Cap Forceps، Lens، Injector یا Secondary Knife تعداد نمونه بسیار کمتری دارند. این عدم تعادل باعث می‌شود مدل به سمت یادگیری کلاس‌های پرتکرار گرایش پیدا کند.

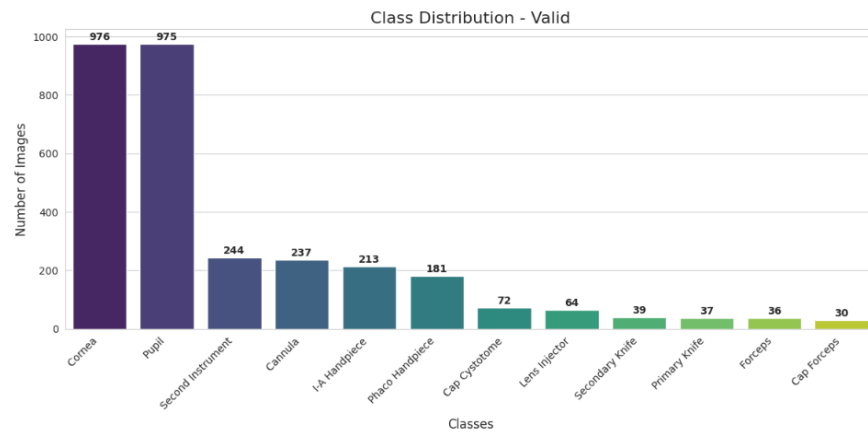
در چنین شرایطی، اگر تنها معیار ما Accuracy باشد، مدل می‌تواند با یادگیری بسیار خوب کلاس‌های Cornea و Pupil به دقت بالایی برسد، در حالی که در تشخیص ابزارهای کم‌نمونه عملکرد ضعیفی داشته باشد. این مسئله به‌خصوص در تنظیم Subset Accuracy (که نیازمند پیش‌بینی صحیح تمام برچسب‌های یک تصویر است) می‌تواند گمراه‌کننده باشد.



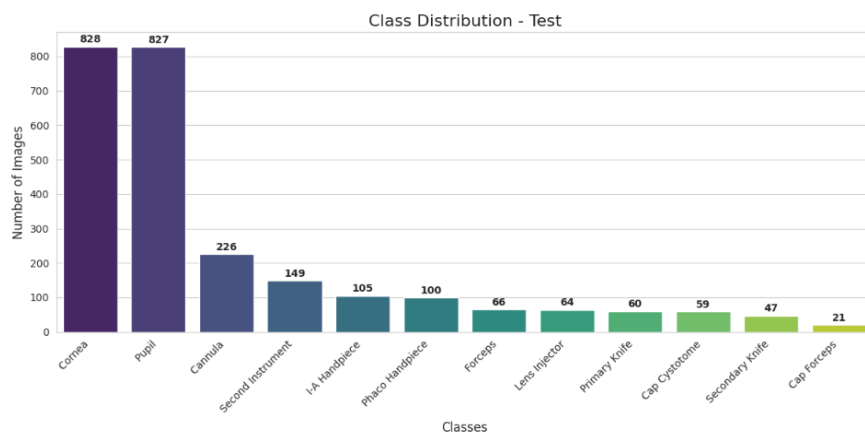
بنابراین چالش اصلی این پروژه، مقابله با عدم تعادل کلاس‌ها در یک مسئله چندبرچسبی پزشکی است. در زیر نمودار توزیع دیتاها برای هر کلاس برای مجموعه داده های train، validation و test آورده شده است:



شکل ۱۰: توزیع داده ها در هر کلاس روی مجموعه داده های آموزش



شکل ۱۱: توزیع داده ها در هر کلاس روی مجموعه داده های اعتبارسنجی



شکل ۱۲: توزیع داده ها در هر کلاس روی مجموعه داده های تست

۱.۲ انتخاب معماری و یادگیری انتقال (Transfer Learning)

۱.۱.۲ انتخاب معماری

از میان معماری های مدرن بینایی ماشین مانند ResNet، ConvNeXt و EfficientNet، مدل EfficientNet-B3 انتخاب شد. دلیل انتخاب این معماری:

- تعادل مناسب بین دقت و پیچیدگی محاسباتی
- استفاده از Compound Scaling (مقیاس دهی همزمان عمق، عرض و رزولوشن)
- عملکرد بسیار قوی در مسائل طبقه بندی تصویر پزشکی
- مصرف حافظه کمتر نسبت به مدل های بسیار عمیق تر

در این پروژه، لایه نهایی مدل به صورت زیر تغییر داده شد تا با مسئله ۱۲ کلاسه ما سازگار شود:

```
model = models.efficientnet_b3(
    weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1
)

num_fttrs = model.classifier[1].in_features
model.classifier[1] = nn.Linear(num_fttrs, 12)

=====
```

۲.۱.۲ چرا آموزش از صفر (From-Scratch) مناسب نیست؟

در داده های پزشکی معمولاً حجم دیتاست محدود است. آموزش یک شبکه عمیق از ابتدا باعث:



- بیش‌برازش شدید (Overfitting)

- ناپایداری در همگرایی

- نیاز به داده بسیار زیاد

- زمان آموزش طولانی‌تر

شبکه‌های عمیق میلیون‌ها پارامتر دارند و بدون داده کافی قادر به یادگیری نمایش‌های پایدار نیستند.

۳.۱.۲ استفاده از وزن‌های از پیش آموزش دیده ImageNet

در این پروژه از وزن‌های آموزش دیده روی دیتاست ImageNet استفاده شد.

مزایا:

- استخراج ویژگی‌های عمومی مانند لبه‌ها، بافت‌ها و الگوهای پایه

- تسریع همگرایی

- کاهش نیاز به داده زیاد

- کاهش ریسک بیش‌برازش

نرمال‌سازی ورودی‌ها نیز مطابق استاندارد ImageNet انجام شد:

```
transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225]
)
```

این کار تضمین می‌کند که توزیع داده‌های ورودی با توزیع داده‌های زمان پیش‌آموزش هم‌راستا باشد.

۲.۲ استراتژی آموزش و بهینه‌سازی

۱.۲.۲ جستجوی فرآپارامترها با Optuna

برای یافتن بهترین تنظیمات آموزشی، از چارچوب Optuna استفاده شد. پارامترهای زیر جستجو شدند:

- نرخ یادگیری (Learning Rate)

- نوع بهینه‌ساز (AdamW یا SGD with Momentum)

• Weight Decay

نمونه‌ای از فضای جستجو:

```
lr = trial.suggest_float("lr", 1e5-, 1e3-, log=True)
optimizer_name = trial.suggest_categorical(
    "optimizer", ["AdamW", "SGD"]
)
weight_decay = trial.suggest_float(
    "weight_decay", 1e6-, 1e3-
```

مقایسه AdamW و SGD

• AdamW: همگرایی سریع‌تر و مناسب برای دیتاست‌های کوچک

• SGD + Momentum: تعمیم‌پذیری بهتر در برخی موارد

۲.۲.۲ مقایسه Focal Loss و Label Smoothing برای مقابله با عدم تعادل کلاس‌ها

در مسائل طبقه‌بندی چندبرچسبی، به‌ویژه در داده‌های پزشکی، یکی از چالش‌های مهم عدم تعادل شدید بین کلاس‌ها است. در دیتاست مورد استفاده در این تمرین نیز چنین شرایطی وجود دارد؛ به‌طوری که برخی کلاس‌ها مانند Pupil و Cornea در اکثر تصاویر حضور دارند، در حالی که ابزارهایی مانند Cap Forceps یا Lens Injector نمونه‌های بسیار کمتری دارند.

در چنین شرایطی، اگر از تابع زیان استاندارد مانند Binary Cross Entropy استفاده شود، مدل تمایل دارد بیشتر بر روی کلاس‌های پرتکرار تمرکز کند و در نتیجه عملکرد ضعیفی روی کلاس‌های کم‌نمونه داشته باشد. برای کاهش اثر این مشکل، دو رویکرد متداول در طراحی تابع زیان مطرح می‌شود: Label Smoothing و Focal Loss.

Label Smoothing در روش Label Smoothing، برچسب‌های قطعی (۰ و ۱) کمی نرم‌تر می‌شوند. به عبارت دیگر، به جای استفاده از برچسب‌های کاملاً قطعی، مقدار احتمال اندکی به سایر کلاس‌ها اختصاص داده می‌شود. برای مثال:

$$y = 1 \rightarrow 0.9, \quad y = 0 \rightarrow 0.1$$

هدف اصلی این روش جلوگیری از اطمینان بیش از حد (Overconfidence) مدل است. این تکنیک معمولاً باعث بهبود تعمیم‌پذیری مدل می‌شود و از بیش‌برازش جلوگیری می‌کند.

با این حال، Label Smoothing مستقیماً مشکل عدم تعادل کلاس‌ها را هدف قرار نمی‌دهد، بلکه بیشتر بر تنظیم توزیع احتمالات خروجی تمرکز دارد. در نتیجه در دیتاست‌هایی که عدم تعادل شدید دارند، اثر آن محدودتر خواهد بود.



Focal Loss تابع زیان Focal Loss به طور خاص برای مقابله با عدم تعادل داده‌ها طراحی شده است. ایده اصلی آن کاهش وزن نمونه‌های آسان و افزایش اهمیت نمونه‌های سخت در فرآیند یادگیری است. فرمول آن به صورت زیر است:

$$FL(p_t) = \alpha(1 - p_t)^\gamma \cdot BCE(p_t)$$

که در آن:

- p_t احتمال پیش‌بینی صحیح نمونه است
 - γ پارامتر تمرکز (focusing parameter) است که میزان کاهش وزن نمونه‌های آسان را کنترل می‌کند
 - α وزن‌دهی به کلاس‌ها را تنظیم می‌کند
- وقتی مقدار γ افزایش می‌یابد، نمونه‌هایی که به راحتی طبقه‌بندی می‌شوند (مانند کلاس‌های پرتکرار) وزن کمتری در تابع زیان دریافت می‌کنند. در مقابل، نمونه‌های دشوار یا کم‌نمونه تأثیر بیشتری در فرآیند آموزش خواهند داشت.

دلیل انتخاب **Focal Loss** در این تمرین در این تمرین به دلیل وجود عدم تعادل شدید بین کلاس‌ها، استفاده از Focal Loss گزینه مناسب‌تری نسبت به Label Smoothing محسوب می‌شود. زیرا:

- تمرکز آموزش را از کلاس‌های بسیار پرتکرار به سمت نمونه‌های دشوارتر هدایت می‌کند.
 - باعث می‌شود مدل نسبت به ابزارهای کم‌نمونه حساسیت بیشتری پیدا کند.
 - در مسائل Multi-label با داده‌های نامتوازن عملکرد بهتری نسبت به زیان استاندارد دارد.
- در این پروژه مقدار $\gamma = 2$ برای تابع Focal Loss انتخاب شد که یکی از مقادیر متداول در ادبیات یادگیری عمیق است و تعادل مناسبی بین تمرکز بر نمونه‌های سخت و پایداری آموزش ایجاد می‌کند.
- در نتیجه، استفاده از Focal Loss در کنار تنظیم فرامترها با استفاده از Optuna به بهبود یادگیری کلاس‌های کم‌نمونه و افزایش تعادل عملکرد مدل در میان کلاس‌های مختلف کمک می‌کند.
- پیاده‌سازی:

```
class FocalLoss(nn.Module):
    def __init__(self, alpha=1, gamma=2):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma

    def forward(self, inputs, targets):
        bce_loss = F.binary_cross_entropy_with_logits(
            inputs, targets, reduction='none'
```



```

)
pt = torch.exp(-bce_loss)
focal_loss = self.alpha * (1 - pt)**self.gamma * bce_loss
return focal_loss.mean()

```

استفاده از $\gamma = 2$ باعث شد مدل تمرکز بیشتری بر ابزارهای کم‌نمونه داشته باشد.

۳.۲.۲ دورویکرد آموزشی

رویکرد اول:

- حفظ تمام ۱۲ کلاس (شامل Pupil و Cornea)
- بدون اعمال Augmentation
- استفاده از Transfer Learning
- استفاده از Focal Loss
- تنظیم فرآپارامترها با Optuna

رویکرد دوم:

- حذف کلاس‌های بسیار پرتکرار (Pupil و Cornea)
- اعمال Augmentation برای متعادل‌سازی
- آموزش مجدد مدل

در این حالت تمرکز مدل بر تشخیص ابزارها افزایش یافت و ارزیابی واقعی‌تری از عملکرد آن به‌دست آمد. در رویکرد دوم نیز فرآپارامترها تنظیم شد و از Transfer Learning و Focal Loss برای یادگیری بهتر استفاده گردید.

۳.۲ پیاده‌سازی رویکرد اول: یادگیری چندبرچسبی با استفاده از EfficientNet-B۳

در رویکرد اول، هدف آموزش یک مدل طبقه‌بندی چندبرچسبی (Multi-label Classification) برای تشخیص هم‌زمان ۱۲ کلاس (شامل ابزارهای جراحی و ساختارهای آناتومیک مانند قرنیه و مردمک) بود. برای این منظور از معماری پیشرفته EfficientNet-B3 استفاده شد که توازن مناسبی میان دقت و هزینه محاسباتی برقرار می‌کند.

۱.۳.۲ ساختار خط لوله (Pipeline) پیاده‌سازی

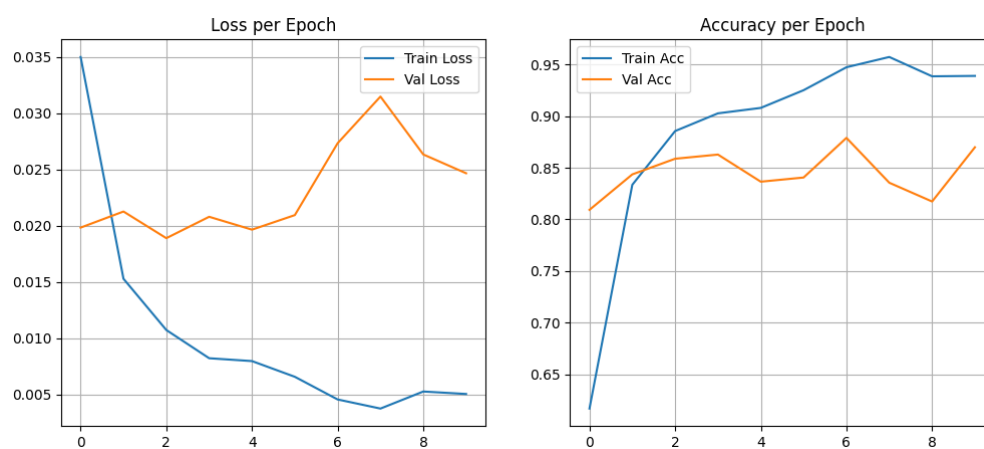
فرآیند آموزش شامل مراحل زیر بود:

- پیش‌پردازش و نرمال‌سازی: تصاویر به ابعاد 300×300 تغییر سایز داده شده و با میانگین و انحراف معیار مجموعه داده ImageNet استانداردسازی شدند.
- بهینه‌سازی فرآیندها با **Optuna**: برای یافتن بهترین نرخ یادگیری (Learning Rate)، نوع بهینه‌ساز (AdamW یا SGD) و ضریب جریمه (Weight Decay)، از چارچوب Optuna با ۵ تکرار استفاده شد.
- تابع زیان (Loss Function): به منظور مقابله با عدم تعادل کلاس‌ها، تابع زیان Focal Loss با پارامتر $\gamma = 2$ جایگزین BCE استاندارد شد تا تمرکز مدل بر روی نمونه‌های سخت (ابزارهای کم‌تکرار) افزایش یابد.

۲.۳.۲ نتایج بهینه‌سازی و تحلیل روند آموزش

بر اساس جستجوی Optuna، بهترین فرآیندها به شرح زیر استخراج شدند:

- بهینه‌ساز: AdamW
 - نرخ یادگیری: 6.047×10^{-4}
 - وزن جریمه (Weight Decay): 0.0009
- روند آموزش در ۱۰ اپوک انجام شد. مطابق با مقادیر ثبت شده، در اپوک دوم مدل به بهترین مقدار Validation Loss (0.0189) دست یافت. با این حال، با ادامه آموزش، شکاف میان مقدار خطای آموزش (0.0051) و خطای اعتبارسنجی شروع به افزایش کرد که نشان‌دهنده وقوع بیش‌برازش (Overfitting) است.



شکل ۱۳: تحلیل روند آموزش و اعتبارسنجی (مشاهده پدیده Overfitting بعد از اپوک ۲)

۳.۳.۲ ارزیابی مدل بر روی داده‌های تست

برای ارزیابی دقیق‌تر در مسائل نامتوازن، معیار PR-AUC محاسبه گردید:

جدول ۱: مقادیر PR-AUC برای هر کلاس

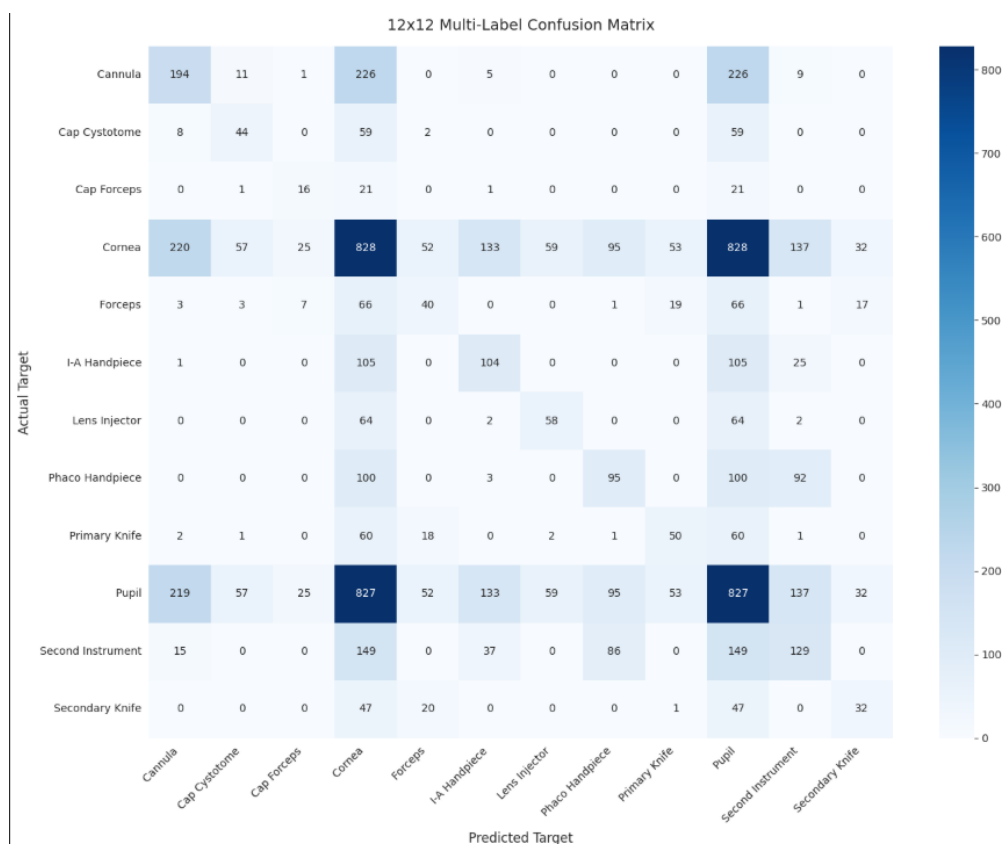
Class	PR-AUC	Class	PR-AUC
Cannula	0.9306	Lens Injector	0.9773
Cap Cystotome	0.8795	Phaco Handpiece	0.9905
Cap Forceps	0.8376	Primary Knife	0.8894
Cornea	0.9923	Pupil	0.9919
Forceps	0.7448	Second Instrument	0.9719
I-A Handpiece	0.9564	Secondary Knife	0.9486

نتایج ارزیابی نهایی بر روی داده‌های آزمون در جدول زیر ارائه شده است.

جدول ۲: گزارش عملکرد مدل EfficientNet-B3 بر روی داده‌های آزمون (رویکرد اول)

Class	Precision	Recall	F1-Score	Support
Cannula	0.87	0.86	0.86	226
Cap Cystotome	0.77	0.75	0.76	59
Cap Forceps	0.64	0.76	0.70	21
Cornea	0.99	1.00	1.00	828
Forceps	0.77	0.61	0.68	66
I-A Handpiece	0.78	0.99	0.87	105
Lens Injector	0.95	0.91	0.93	64
Phaco Handpiece	1.00	0.95	0.97	100
Primary Knife	0.94	0.83	0.88	60
Pupil	0.99	1.00	1.00	827
Second Instrument	0.93	0.87	0.90	149
Secondary Knife	1.00	0.68	0.81	47
Micro Avg	0.95	0.95	0.95	2552
Macro Avg	0.89	0.85	0.86	2552
Weighted Avg	0.95	0.95	0.95	2552
Samples Avg	0.95	0.95	0.94	2552

ماتریس درهم‌ریختگی حاصل نیز به صورت زیر است:



شکل ۱۴: نمایش ماتریس درهم‌ریختگی برای رویکرد اول

تحلیل غلبه آماری کلاس‌های آناتومیک: مطابق ماتریس درهم‌ریختگی، دو کلاس Cornea و Pupil با مقادیر ۸۲۸ و ۸۲۷ در قطر اصلی، بالاترین دقت تشخیص را دارند. نکته قابل توجه در این ماتریس، حضور ستون‌های پررنگ برای این دو کلاس در تمامی ردیف‌های دیگر است. این موضوع تأیید می‌کند که در یک محیط جراحی چندبرچسبی، این دو ساختار تقریباً در تمام فریم‌ها در کنار ابزارهای دیگر حضور دارند (Co-occurrence). مدل به خوبی یاد گرفته است که حضور ابزار جراحی به معنای نفی حضور قرنیه یا مردمک نیست.

عملکرد تخصصی ابزارهای جراحی: ابزارهایی مانند Phaco Handpiece (۹۵ تشخیص درست از ۱۰۰)، Lens Injector (۵۸ تشخیص درست از ۶۴) و Cannula (۱۹۴ تشخیص درست از ۲۲۶) عملکرد بسیار مطلوبی در قطر اصلی نشان می‌دهند. این نشان‌دهنده آن است که مدل EfficientNet-B3 ویژگی‌های بصری منحصربه‌فرد این ابزارها، مانند شکل بدنه و بازتاب نور از سطح فلزی آن‌ها را به خوبی استخراج کرده است.

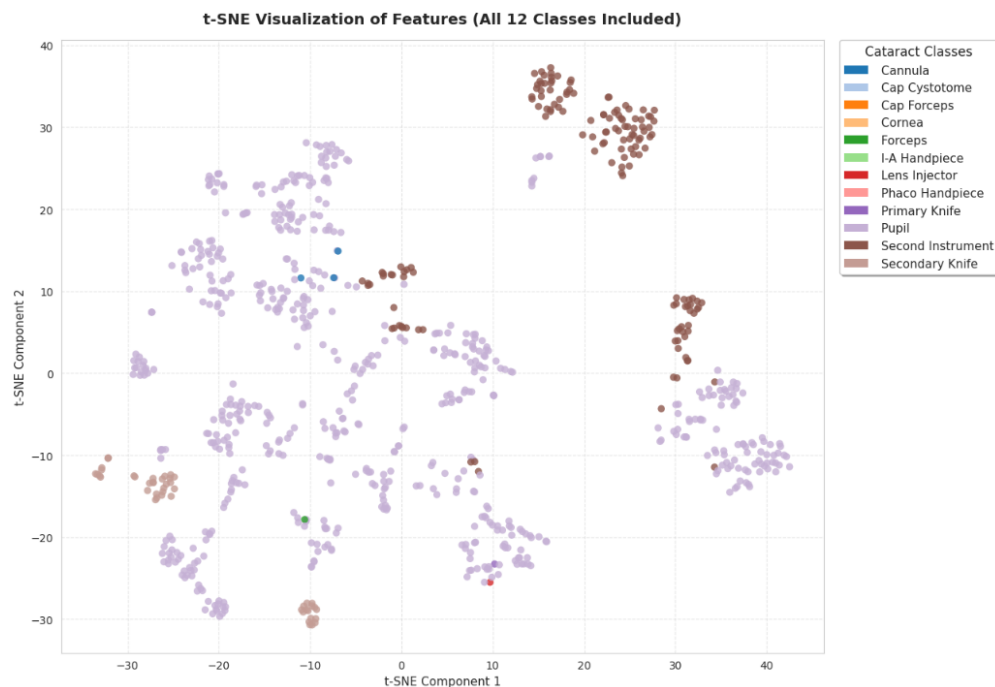
تحلیل خطاهای طبقه‌بندی و هم‌پوشانی ویژگی‌ها: برخی ابزارها دارای درهم‌ریختگی نسبی هستند؛ به عنوان مثال، کلاس Secondary Knife در ۲۰ مورد به اشتباه به عنوان Forceps تشخیص داده شده است. این خطا می‌تواند ناشی از شباهت هندسی بخش‌های انتهایی (دسته) این دو ابزار یا نحوه به دست گرفتن آن‌ها توسط جراح باشد. همچنین در ردیف Second Instrument، هم‌پوشانی قابل توجهی با Phaco Handpiece (۸۶ مورد) دیده می‌شود که نشان‌دهنده حضور هم‌زمان این دو ابزار در مراحل خاصی از جراحی (مانند مرحله فیکو) است که مدل را در تفکیک دقیق مرزهای برچسب‌زنی دچار چالش می‌کند.

تأثیر عدم تعادل داده‌ها (**Data Imbalance**): کلاس‌هایی با پشتیبانی (Support) کمتر، مانند Cap Forceps (۲۱ نمونه)، اگرچه در قطر اصلی ۱۶ مورد درست دارند، اما نسبت به کلاس‌های پرجمعیت‌تر، حساسیت بیشتری به نویز نشان می‌دهند. با این حال، غلبه رنگ آبی در اکثر عناصر قطر اصلی نشان‌دهنده این است که استراتژی استفاده از Focal Loss موفقیت‌آمیز بوده و مدل اجازه نداده است کلاس‌های پرتکرار، فرآیند یادگیری ابزارهای کمیاب را کاملاً سرکوب کنند.

تحلیل نتایج: همان‌طور که مشاهده می‌شود، کلاس‌های Cornea و Pupil به Precision و Recall نزدیک به ۱۰۰٪ رسیده‌اند. دلیل این امر حضور دائمی این دو کلاس در تصاویر آموزشی است. این موضوع باعث شده مدل بخش بزرگی از ظرفیت یادگیری خود را صرف این دو کلاس پایدار کند. در مقابل، ابزارهایی مانند Forceps به دلیل تعداد نمونه کمتر و تنوع ظاهری، مقادیر کمتری در F1-score و PR-AUC برابر با 0.75 ثبت کرده‌اند. اگرچه Focal Loss تا حدی این نابرابری را جبران کرده، اما غلبه آماری کلاس‌های آناتومیک همچنان مشهود است. همچنین چون Cornea و Pupil بزرگ هستند، عملاً بخش بزرگی از فضای ویژگی را به خود اختصاص می‌دهند و این باعث می‌شود که مدل بیشتر آنها را ببیند درحالی‌که برخی ابزارهای ما به نسبت نازکتر و کوچکترند.

۴.۳.۲ تحلیل فضای ویژگی با t-SNE

الگوریتم t-SNE یک روش کاهش ابعاد غیرخطی است که برای تجسم داده‌های با ابعاد بالا در فضای دو یا سه بعدی استفاده می‌شود. این روش با حفظ فاصله‌های محلی، خوشه‌های داده را مشخص می‌کند. در خروجی حاصل از مدل ما بر روی داده‌های تست، مشاهده شد که خوشه‌های مربوط به هر کلاس به خوبی از یکدیگر تفکیک شده‌اند. این امر نشان‌دهنده توانایی بالای استخراج ویژگی (Feature Extraction) توسط هسته EfficientNet-B3 است که توانسته نگاهی متمایز برای هر ابزار جراحی در فضای ویژگی ایجاد کند. در تصویر زیر این نتیجه قابل مشاهده است:



شکل ۱۵: نگاشت فضای ویژگی داده‌های تست در دو بعد

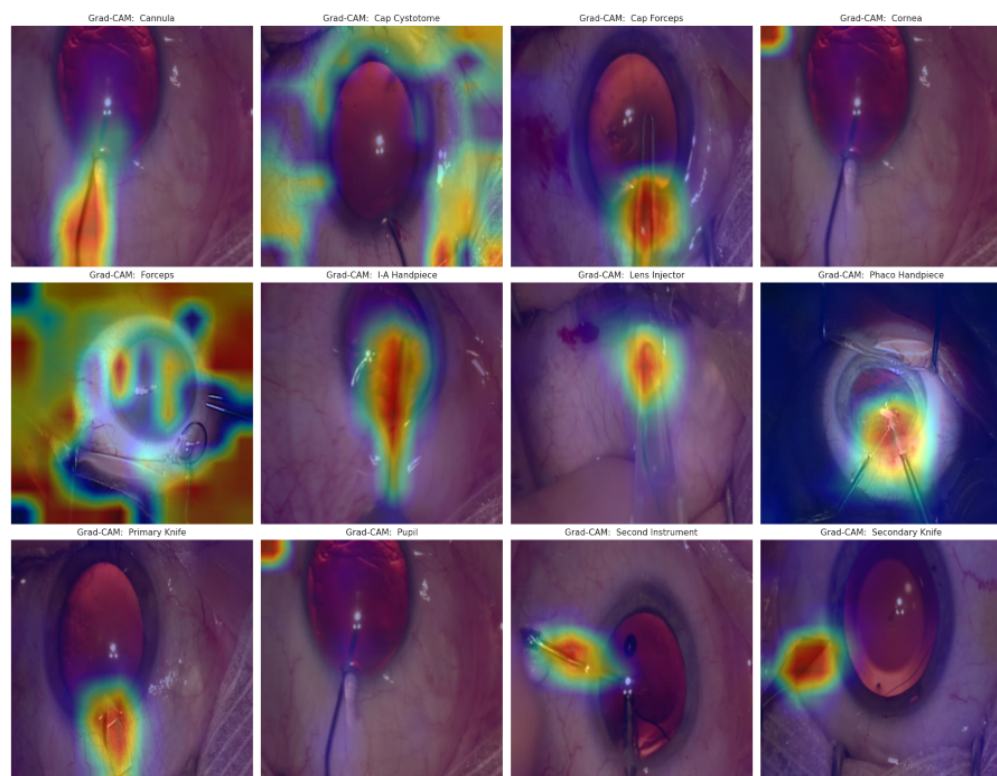
۵.۳.۲ تفسیرپذیری مدل با استفاده از Grad-CAM

روش Grad-CAM با استفاده از گرادیان‌های آخرین لایه کانولوشنی، نقشه‌های حرارتی ایجاد می‌کند که نشان می‌دهد مدل برای پیش‌بینی هر کلاس به کدام نواحی تصویر توجه کرده است.

- در مورد ابزارهای جراحی: نقشه‌های حرارتی به‌دقت بر روی بدنه ابزار و نوک آن (جایگاه اثرگذاری در جراحی) متمرکز شده‌اند. این نشان می‌دهد مدل ویژگی‌های هندسی و بافتی صحیح ابزارها را فرا گرفته است.

- در مورد **Pupil** و **Cornea**: برخلاف ابزارها، در این دو کلاس مدل تمایل دارد کل ناحیه چشم را به‌عنوان ناحیه فعال در نظر بگیرد. این رفتار تأیید می‌کند که به دلیل تکرار بیش از حد این کلاس‌ها در تمام تصاویر، مدل به جای یادگیری مرزهای دقیق، کل پس‌زمینه جراحی را به این دو کلاس نسبت داده است.

نمونه تصاویری که بر روی آنها Grad-CAM زده شده است به صورت زیر می‌باشند (از تمامی کلاس‌ها یک تصویر آورده شده است):



شکل ۱۶: تفسیرپذیری مدل با استفاده از Grad-CAM در رویکرد اول

تفسیر بصری تمرکز مدل بر روی ابزارها: نقشه‌های حرارتی حاصل از Grad-CAM نشان می‌دهند که مدل در تشخیص اکثر ابزارهای جراحی مانند Cannula، I-A Handpiece، Phaco Handpiece، به‌درستی بر روی بدنه فلزی و ناحیه عملیاتی ابزار (نوک ابزار در محل تماس با بافت چشم) متمرکز شده است. این موضوع تأیید می‌کند که مدل ویژگی‌های ریخت‌شناختی (Morphological) ابزارها را به جای نویزهای پس‌زمینه یاد گرفته است.

تحلیل نقاط داغ در ابزارهای خاص: در مواردی مانند Lens Injector و Primary Knife، تمرکز اصلی نقشه حرارتی دقیقاً در مرکز تصویر و بر روی محل ورود ابزار قرار دارد. این دقت بالا نشان‌دهنده توانایی شبکه در تفکیک ابزارهای باریک از بافت‌های مشابه در محیط مرطوب جراحی است. در کلاس‌هایی مانند Second Instrument، مدل علاوه بر ابزار، به نواحی اطراف آن نیز حساس است که نشان‌دهنده یادگیری وابستگی‌های مکانی ابزار نسبت به ساختار چشم است.

ارزیابی اعتبار تصمیم‌گیری مدل: به طور کلی، انطباق بالای نقاط داغ (Hotspots) با مکان واقعی ابزارها در تصاویر خروجی، گواهی بر این است که نرخ دقت بالای مدل ناشی از یادگیری صحیح ویژگی‌های بصری است و نه میان‌برهای آماری یا بیش‌برازش بر روی المان‌های نامرتبط در محیط اتاق عمل.

۴.۲ پیاده‌سازی رویکرد دوم: حذف ساختارهای آناتومیک و موازنه ابزارها

در رویکرد دوم، با هدف متمرکز کردن مدل بر روی ویژگی‌های ظریف ابزارهای جراحی و جلوگیری از غلبه آماری ساختارهای دائمی، تغییرات بنیادینی در خط لوله (Pipeline) داده‌ها ایجاد شد.

۱.۴.۲ تعدیل برچسب‌ها و استراتژی موازنه (Data Balancing)

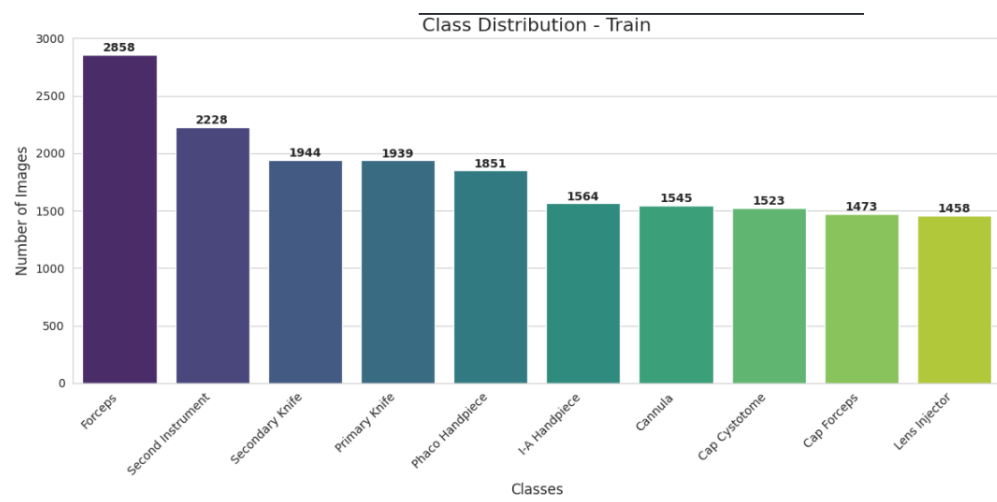
در این مرحله، کلاس‌های Cornea و Pupil که تقریباً در اکثر تصاویر حضور داشتند، از مجموعه برچسب‌ها حذف شدند.

- دلیل حذف: حضور دائمی این دو کلاس فضای ویژگی (Feature Space) را اشغال کرده و به دلیل ابعاد بزرگشان نسبت به ابزارها، مدل را به سمت یادگیری ویژگی‌های درشت‌دانه سوق می‌دادند. حذف آن‌ها مدل را مجبور می‌کند تا ظرفیت یادگیری خود را صرفاً روی ۱۰ ابزار جراحی متمرکز کند.

- تولید داده‌های افزایشی (Augmentation): برای مقابله با عدم تعادل میان ابزارها، کلاس Cannula با ۱۴۵۷ نمونه به عنوان هدف (Target) در نظر گرفته شد. سایر کلاس‌ها با استفاده از تکنیک‌های Rotate، Horizontal Flip و Brightness Contrast به این سطح رسانده شدند.

- نتیجه نهایی: حجم مجموعه داده آموزش پس از حذف تصاویر فاقد ابزار و انجام آگمنتیشن به ۱۳،۴۲۴ تصویر رسید که توزیع یکنواخت‌تری میان ۱۰ کلاس باقی‌مانده برقرار کرد.

تصویر زیر توزیع داده‌ها را پس از آگمنتیشن نشان می‌دهد:



شکل ۱۷: توزیع داده ها در هر کلاس بعد از آگمنتیشن در رویکرد دوم

۲.۴.۲ بهینه‌سازی فرآپارامترها با Optuna

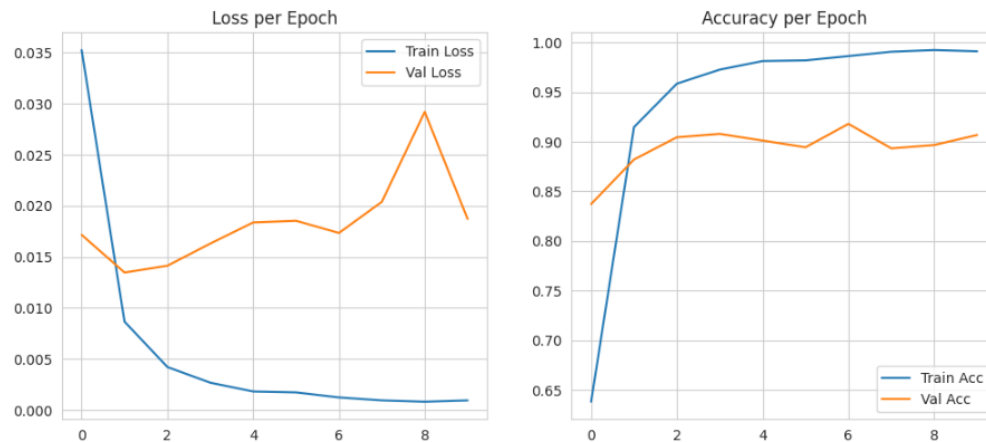
برای این رویکرد جدید، جستجوی فرآپارامترها مجدداً توسط Optuna انجام شد. نتایج ۵ تکرار (Trial) نشان داد که بهینه‌ساز AdamW به طور معناداری بهتر از SGD عمل می‌کند.

- بهترین مقادیر یافت شده: نرخ یادگیری 4.91×10^{-5} و وزن جریمه 0.00057.
- تحلیل: انتخاب نرخ یادگیری پایین‌تر توسط Optuna نشان‌دهنده نیاز مدل به یادگیری دقیق‌تر و آرام‌تر بر روی داده‌های متوازن شده است تا از پرش‌های تند در فضای بهینه جلوگیری شود.

۳.۴.۲ روند آموزش و تحلیل بیش‌برازش (Overfitting)

مدل با تابع زیان Focal Loss ($\gamma = 2$) آموزش داده شد. مطابق خروجی‌ها، بهترین مدل در اپوک دوم با Val Loss: 0.0135 ذخیره شد.

تحلیل نمودارهای Loss و Accuracy: در این رویکرد، اگرچه مقدار مطلق Loss نسبت به رویکرد اول کاهش یافته و مدل سریع‌تر به دقت بالای ۹۰٪ رسیده است، اما پدیده بیش‌برازش (Overfitting) همچنان مشهود است. شکاف میان دقت آموزش (99.1%) و اعتبارسنجی (90.6%) در اپوک‌های پایانی نشان می‌دهد که مدل علی‌رغم آگمنتیشن سنگین، تمایل به حفظ کردن جزئیات تصاویر آموزشی دارد. با این حال، کاهش مقدار Validation Loss نسبت به حالت قبل نشان‌دهنده موفقیت نسبی در متمرکز کردن مدل بر روی ابزارهاست.



شکل ۱۸: روند تغییرات خطا و صحت در رویکرد دوم (۱۰ کلاس)

۴.۴.۲ ارزیابی نهایی و مقایسه با رویکرد اول

نتایج حاصل از تست بر روی ۱۰ کلاس در جدول زیر ارائه شده است.

جدول ۳: گزارش عملکرد مدل EfficientNet-B3 در رویکرد دوم (۱۰ کلاس)

Class	Precision	Recall	F1-Score	Support
Cannula	0.83	0.91	0.87	226
Cap Cystotome	0.82	0.69	0.75	59
Cap Forceps	0.90	0.90	0.90	21
Forceps	0.61	0.67	0.64	66
I-A Handpiece	0.80	0.99	0.89	105
Lens Injector	0.94	0.94	0.94	64
Phaco Handpiece	0.97	0.98	0.98	100
Primary Knife	1.00	0.78	0.88	60
Second Instrument	0.95	0.87	0.91	149
Secondary Knife	0.94	1.00	0.97	47
Micro Avg	0.86	0.89	0.88	897
Macro Avg	0.88	0.87	0.87	897

تحلیل مقایسه‌ای: با حذف کلاس‌های غالب (Cornea/Pupil)، میانگین دقت کل (Micro Avg) از ۹۵٪ به ۸۸٪ کاهش یافته است. این کاهش کاملاً قابل انتظار بود، زیرا در رویکرد اول، تشخیص بسیار آسان دو کلاس حجیم، میانگین دقت را به طور کاذب بالا می‌برد. در رویکرد دوم، بهبودهای مهمی در کلاس‌های سخت مشاهده می‌شود؛ برای مثال دقت کلاس Cap Forceps از ۷۰.۰ به ۹۰.۰ ارتقا یافته است. این امر نشان‌دهنده قدرت رویکرد دوم در استخراج ویژگی‌های تخصصی ابزارها پس از حذف بخش آناتومیک است.

جدول ۴: مقادیر PR-AUC برای ۱۰ کلاس در رویکرد دوم

Class	PR-AUC	Class	PR-AUC
Cannula	0.8834	Lens Injector	0.9403
Cap Cystotome	0.7701	Phaco Handpiece	0.9765
Cap Forceps	0.9062	Primary Knife	0.9008
Forceps	0.6543	Second Instrument	0.9240
I-A Handpiece	0.8959	Secondary Knife	0.9700

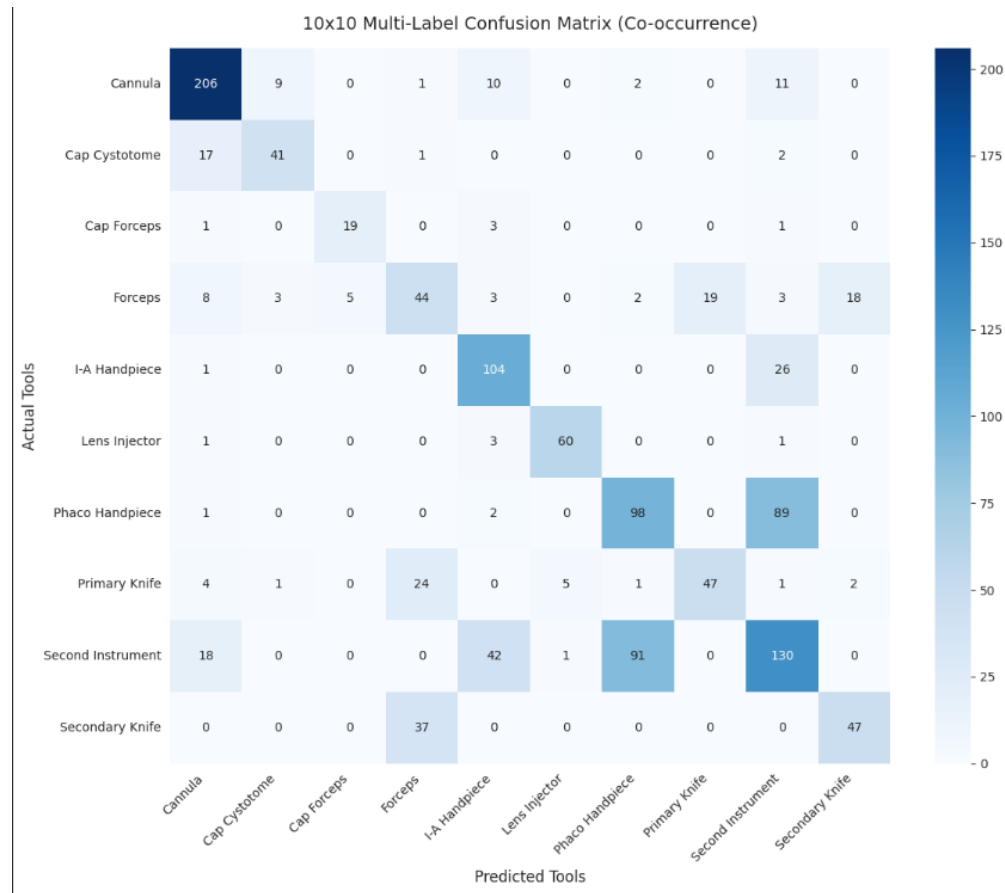
بصری‌سازی و تفسیر: در ادامه، با استفاده از تحلیل‌های t-SNE، Confusion Matrix و Grad-CAM، کیفیت تفکیک ابزارها در فضای ویژگی و تمرکز مدل بر روی لبه‌های تیز ابزارها مورد بررسی قرار می‌گیرد که تأییدکننده بهبود قدرت تفکیک مدل در غیاب کلاس‌های غالب است.

تحلیل ماتریس درهم‌ریختگی و الگوهای هم‌رخدادی (Co-occurrence): با بررسی ماتریس درهم‌ریختگی رویکرد دوم، مشخص می‌شود که مدل در تشخیص کلاس‌های منفرد و متمایز مانند Phaco Handpiece (۹۸ تشخیص درست) و Cannula (۲۰۶ تشخیص درست) بسیار دقیق عمل می‌کند. با این حال، به دلیل ماهیت چندبرچسبی (Multi-label) مسئله و ماهیت جراحی آب‌مروارید، الگوهای خاصی از تداخل در ماتریس مشاهده می‌شود:

- تداخل ابزارهای کمکی: بیشترین میزان تداخل بین Second Instrument با کلاس‌های Phaco Handpiece (۹۱ مورد) و I-A Handpiece (۴۲ مورد) دیده می‌شود. این موضوع به دلیل هم‌رخدادی همیشگی ابزار ثانویه برای نگه داشتن لنز یا تثبیت چشم هنگام کار با هندپیس‌های اصلی است. در واقع، مدل یاد گرفته است که این ابزارها معمولاً به صورت جفت در تصویر ظاهر می‌شوند.

- دقت در کلاس‌های خاص: کلاس‌هایی مانند Lens Injector و Cap Forceps کمترین میزان خطا و تداخل را با سایر ابزارها دارند که نشان‌دهنده ویژگی‌های بصری منحصربه‌فرد آن‌ها (مانند بدنه ضخیم‌تر اینجکتور) است که توسط مدل EfficientNet-B3 به خوبی استخراج شده است.

به طور کلی، ماتریس درهم‌ریختگی تأیید می‌کند که حذف کلاس‌های غالب (Cornea/Pupil) نه تنها باعث سردرگمی مدل نشده، بلکه باعث شده است مدل بر روی روابط منطقی و هم‌رخدادی ابزارهای جراحی در محیط عملیاتی تمرکز بهتری داشته باشد.

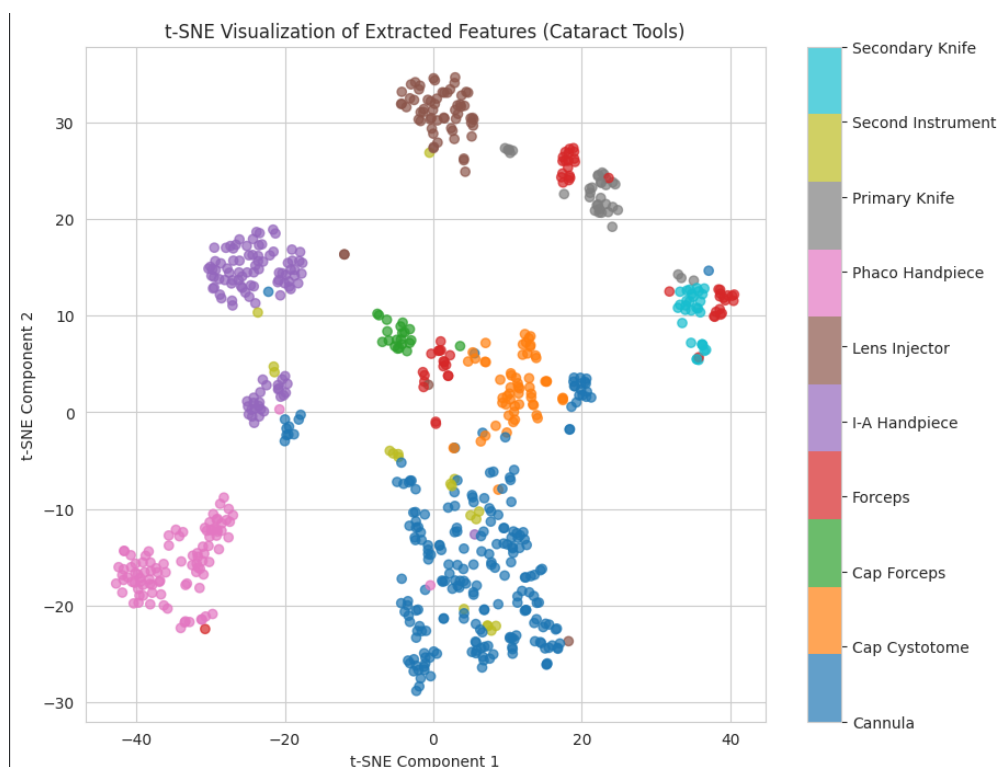


شکل ۱۹: ماتریس درهم‌ریختگی مجموعه داده تست برای رویکرد دوم

تحلیل بصری فضای ویژگی با استفاده از t-SNE: نمودار t-SNE حاصل از استخراج ویژگی‌های لایه پیش‌آخر شبکه EfficientNet-B3، گویای موفقیت مدل در تفکیک خوشه‌های مربوط به هر ابزار است. تحلیل این نگاهت دو بعدی نتایج زیر را به همراه دارد:

- تشکیل خوشه‌های متراکم و مجزا: اکثر کلاس‌ها مانند Phaco Handpiece (رنگ صورتی)، I-A Handpiece (رنگ بنفش) و Lens Injector (رنگ قهوه‌ای) خوشه‌های کاملاً مجزایی را تشکیل داده‌اند. این موضوع نشان می‌دهد که حذف کلاس‌های آناتومیک (Cornea/Pupil) فضا را برای یادگیری ویژگی‌های تمایزدهنده (Discriminative Features) ابزارهای جراحی باز کرده است.
- تفسیر نزدیکی خوشه‌ها: نزدیکی و درهم‌تنیدگی جزئی در مرکز نمودار میان کلاس‌هایی همچون Cannula (آبی) و Cap Cystotome (نارنجی) مشاهده می‌شود. این نزدیکی در فضای ویژگی کاملاً منطقی است، زیرا این ابزارها از نظر ساختار فیزیکی (بدنه فلزی لوله‌ای) شباهت بسیار زیادی به یکدیگر دارند.
- توزیع کلاس‌های چندگانه: پراکندگی کلاس Second Instrument (زرد/سبز لیمویی) در میان سایر خوشه‌ها نشان‌دهنده حضور همزمان این ابزار با سایر ابزارهای جراحی است. از آنجا که تصویر حاوی هر دو ابزار است، بردار ویژگی استخراج شده مقادیر مشترکی از هر دو کلاس را در خود دارد که باعث می‌شود این نقاط در مرزهای خوشه‌های دیگر قرار گیرند.

در مجموع، شفافیت خوشه‌ها در این نمودار ثابت می‌کند که مدل علی‌رغم پیچیدگی محیط جراحی و ابعاد کوچک ابزارها، توانسته است بازنمایی‌های (Representations) بسیار قدرتمند و تفکیک‌پذیری را برای هر کلاس یاد بگیرد.



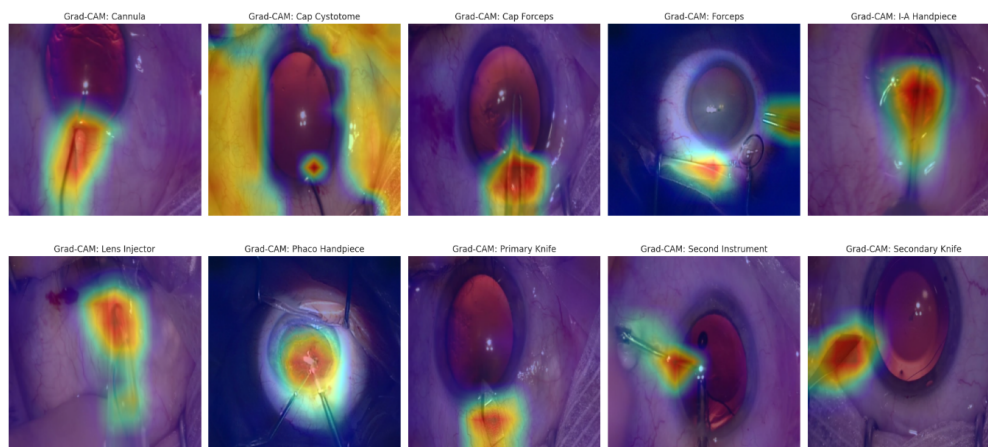
شکل ۲۰: نگاشت فضای ویژگی داده های تست در دو بعد

تفسیر مدل و بصری سازی نواحی تصمیم ساز با **Grad-CAM**: به منظور ارزیابی میزان انطباق تصمیمات مدل با واقعیت های جراحی، از تکنیک Grad-CAM برای بصری سازی نقشه های حرارتی (Heatmaps) استفاده شد. این نقشه ها نشان می دهند که نوروں های لایه های نهایی مدل بر روی کدام پیکسل ها متمرکز شده اند:

- تمرکز بر نوک ابزار (**Functional Ends**): در اکثر تصاویر، به ویژه کلاس های Cannula، Primary Knife و Cap Cystotome، مدل به طور دقیق بر روی بخش عملیاتی ابزار (نوک تیز یا محلی که وارد قرنیه می شود) متمرکز شده است. این نشان می دهد که مدل به جای تکیه بر پس زمینه، ویژگی های ریخت شناسختی خود ابزار را یاد گرفته است.
- تفکیک در حضور چندین ابزار: در تصویر مربوط به Second Instrument، مشاهده می شود که نقشه حرارتی دقیقاً بر روی ابزار جانبی (سمت چپ تصویر) قرار گرفته و از ابزار اصلی در مرکز چشم صرف نظر کرده است. این موضوع دقت مدل در مسئله چندبرچسبی را اثبات می کند که می تواند همزمان چندین ناحیه را برای کلاس های مختلف پایش کند.
- تشخیص هندپیس های حجیم: در کلاس هایی مانند Phaco Handpiece و I-A Handpiece، مدل علاوه بر نوک ابزار، بدنه و زاویه ورود ابزار به تصویر را نیز در نظر می گیرد. گستردگی ناحیه قرمز در این تصاویر نشان دهنده یادگیری ویژگی های مقیاس بزرگ (**Global Features**) توسط معماری EfficientNet-B3 است.
- عدم حساسیت به بافت های آناتومیک: نکته بسیار مهم این است که نواحی قرمز رنگ (بیشترین وزن در تصمیم گیری) به ندرت بر روی نواحی خونی یا بافت های محیطی چشم قرار گرفته اند. این امر تایید می کند که حذف کلاس های Cornea و Pupil در رویکرد دوم، باعث شده است مدل نسبت به نویزهای بیولوژیکی مقاوم شده و فقط به هندسه ابزارهای فلزی حساس باشد.



در مجموع، نتایج Grad-CAM تایید می‌کند که مدل توسعه داده شده نه تنها از نظر آماری دقیق است، بلکه از نظر بالینی نیز به ویژگی‌های درست و منطقی برای تشخیص ابزارهای جراحی استناد می‌کند.



شکل ۲۱: تفسیرپذیری مدل با استفاده از Grad-CAM در رویکرد دوم

۵.۲ کد

```
1 # -*- coding: utf-8 -*-
2 """RC_Hw3_Q2.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/1k3MMxVFSkgeWcn4PEeb8MldNzIKsr_Hw
8
9 #Import Data From Drive
10 """
11
12 from google.colab import drive
13 drive.mount('/content/drive')
14
15 import os
16
17 folder_path = "/content/drive/MyDrive/Cataract_dataset_classification"
18
19 files = os.listdir(folder_path)
20 print("Files in folder:")
21 for f in files:
22     print(f)
```



```
۲۳
۲۴ import zipfile
۲۵ import os
۲۶
۲۷ zip_path1 = "/content/drive/MyDrive/Cataract_dataset_classification/Cataract_dataset_classification.vli.multiclass.zip"
↪ ion.vli.multiclass.zip"
۲۸ extract_path1 = "/content/cataract_dataset1"
۲۹
۳۰
۳۱ os.makedirs(extract_path1, exist_ok=True)
۳۲
۳۳ # Extract
۳۴ with zipfile.ZipFile(zip_path1, 'r') as zip_ref:
۳۵     zip_ref.extractall(extract_path1)
۳۶
۳۷ print(" Extracted to:", extract_path1)
۳۸
۳۹ import os
۴۰
۴۱ base_path = "/content/cataract_dataset1"
۴۲
۴۳ for split in ["train", "valid", "test"]:
۴۴     split_path = os.path.join(base_path, split)
۴۵     print(f"\n {split}:")
۴۶     if os.path.exists(split_path):
۴۷         subfolders = [f for f in os.listdir(split_path) if os.path.isdir(os.path.join(split_path,
↪ f))]
۴۸         for sub in sorted(subfolders):
۴۹             print(" -", sub)
۵۰     else:
۵۱         print("Dont find anything")
۵۲
۵۳ import os
۵۴ import shutil
۵۵
۵۶ base_path = "/content/cataract_dataset1"
۵۷ output_folder = "/content/collected_csv"
۵۸
۵۹
۶۰ os.makedirs(output_folder, exist_ok=True)
```



```
۶۱
۶۲ splits = ["train", "valid", "test"]
۶۳
۶۴ for split in splits:
۶۵     split_path = os.path.join(base_path, split)
۶۶
۶۷     if not os.path.exists(split_path):
۶۸         print(f" Dont find:{split_path} ")
۶۹         continue
۷۰
۷۱
۷۲     for file in os.listdir(split_path):
۷۳         if file.endswith(".csv"):
۷۴             src = os.path.join(split_path, file)
۷۵             dst = os.path.join(output_folder, f"{split}_{file}")
۷۶             shutil.copy(src, dst)
۷۷             print(f" Finding csv: {src} → {dst}")
۷۸
۷۹ print("\n Saving All CSVs on Folder: /content/collected_csv ")
۸۰
۸۱ """#Data Distribution per Class"""
۸۲
۸۳ import pandas as pd
۸۴ import matplotlib.pyplot as plt
۸۵ import seaborn as sns
۸۶
۸۷
۸۸ paths = {
۸۹     "Train": "/content/collected_csv/train__classes.csv",
۹۰     "Valid": "/content/collected_csv/valid__classes.csv",
۹۱     "Test": "/content/collected_csv/test__classes.csv"
۹۲ }
۹۳
۹۴ sns.set_style("whitegrid")
۹۵
۹۶ for split_name, path in paths.items():
۹۷
۹۸
۹۹     df = pd.read_csv(path)
۱۰۰
```



```
101 label_df = df.drop(columns=["filename"])
102
103
104
105 class_counts = label_df.sum().sort_values(ascending=False)
106
107
108 plt.figure(figsize=(12, 6))
109 ax = sns.barplot(
110     x=class_counts.index,
111     y=class_counts.values,
112     palette="viridis"
113 )
114
115 plt.title(f"Class Distribution - {split_name}", fontsize=16)
116 plt.xlabel("Classes", fontsize=12)
117 plt.ylabel("Number of Images", fontsize=12)
118 plt.xticks(rotation=45, ha="right")
119
120
121 for i, value in enumerate(class_counts.values):
122     ax.text(
123         i,
124         value + 5,
125         str(int(value)),
126         ha='center',
127         va='bottom',
128         fontsize=10,
129         fontweight='bold'
130     )
131
132 plt.tight_layout()
133 plt.show()
134
135 """#Data Preprocessing"""
136
137 import torch
138 import torch.nn as nn
139 import torch.optim as optim
140 from torch.utils.data import Dataset, DataLoader
```

```
۱۴۱ from torchvision import transforms, models
۱۴۲ from PIL import Image
۱۴۳ import pandas as pd
۱۴۴ import os
۱۴۵
۱۴۶
۱۴۷ device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
۱۴۸ print(f" Using device: {device}")
۱۴۹
۱۵۰ class CataractDataset(Dataset):
۱۵۱     def __init__(self, csv_file, img_dir, transform=None):
۱۵۲         self.df = pd.read_csv(csv_file)
۱۵۳         self.img_dir = img_dir
۱۵۴         self.transform = transform
۱۵۵
۱۵۶         self.classes = [c for c in self.df.columns if c != 'filename']
۱۵۷
۱۵۸     def __len__(self):
۱۵۹         return len(self.df)
۱۶۰
۱۶۱     def __getitem__(self, idx):
۱۶۲         img_name = os.path.join(self.img_dir, self.df.iloc[idx]['filename'])
۱۶۳         image = Image.open(img_name).convert('RGB')
۱۶۴
۱۶۵
۱۶۶         labels = torch.tensor(self.df.iloc[idx][self.classes].values.astype(float),
۱۶۷                               dtype=torch.float32)
۱۶۸
۱۶۹         if self.transform:
۱۷۰             image = self.transform(image)
۱۷۱
۱۷۲         return image, labels
۱۷۳
۱۷۴ transform = transforms.Compose([
۱۷۵     transforms.Resize((300, 300)),
۱۷۶     transforms.ToTensor(),
۱۷۷     transforms.Normalize(mean=[485.0, 456.0, 406.0], std=[229.0, 224.0, 225.0])
۱۷۸ ])
۱۷۹
```



```
180
181
182 train_dataset = CataractDataset('/content/collected_csv/train__classes.csv',
    ↳ '/content/cataract_dataset1/train', transform=transform)
183 val_dataset = CataractDataset('/content/collected_csv/valid__classes.csv',
    ↳ '/content/cataract_dataset1/valid', transform=transform)
184 test_dataset = CataractDataset('/content/collected_csv/test__classes.csv',
    ↳ '/content/cataract_dataset1/test', transform=transform)
185
186 """#The first Approach"""
187
188 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True, num_workers=2)
189 val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False, num_workers=2)
190 test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False, num_workers=2)
191
192 !pip install optuna
193
194 """#Hyperparameter Optimization Using Optuna Framework"""
195
196 import optuna
197 import gc
198 def objective(trial):
199     gc.collect()
200     torch.cuda.empty_cache()
201
202     lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
203     optimizer_name = trial.suggest_categorical("optimizer", ["AdamW", "SGD"])
204     weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3)
205
206
207     model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
208     num_fters = model.classifier[1].in_features
209     model.classifier[1] = nn.Linear(num_fters, 12)
210     model = model.to(device)
211
212
213     if optimizer_name == "AdamW":
214         optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
215     else:
216         optimizer = optim.SGD(model.parameters(), lr=lr, momentum=9.0)
```



```
۲۱۷
۲۱۸     criterion = nn.BCEWithLogitsLoss()
۲۱۹
۲۲۰
۲۲۱     model.train()
۲۲۲
۲۲۳     for batch_idx, (data, target) in enumerate(train_loader):
۲۲۴         if batch_idx > 30: break
۲۲۵         data, target = data.to(device), target.to(device)
۲۲۶         optimizer.zero_grad()
۲۲۷         output = model(data)
۲۲۸         loss = criterion(output, target)
۲۲۹         loss.backward()
۲۳۰         optimizer.step()
۲۳۱
۲۳۲
۲۳۳     model.eval()
۲۳۴     val_loss = 0
۲۳۵     val_loader = DataLoader(val_dataset, batch_size=32)
۲۳۶     with torch.no_grad():
۲۳۷         for data, target in val_loader:
۲۳۸             data, target = data.to(device), target.to(device)
۲۳۹             output = model(data)
۲۴۰             val_loss += criterion(output, target).item()
۲۴۱
۲۴۲     return val_loss / len(val_loader)
۲۴۳
۲۴۴
۲۴۵ study = optuna.create_study(direction="minimize")
۲۴۶ study.optimize(objective, n_trials=5)
۲۴۷
۲۴۸ print("Best hyperParameters:", study.best_params)
۲۴۹
۲۵۰ best_params = study.best_params
۲۵۱ model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
۲۵۲ num_fts = model.classifier[1].in_features
۲۵۳ model.classifier[1] = nn.Linear(num_fts, 12)
۲۵۴ model = model.to(device)
۲۵۵
۲۵۶ if best_params['optimizer'] == "AdamW":
```



```
257 optimizer = optim.AdamW(model.parameters(), lr=best_params['lr'],
    ↳ weight_decay=best_params['weight_decay'])
258 else:
259     optimizer = optim.SGD(model.parameters(), lr=best_params['lr'], momentum=9.0)
260
261 """#Focal Loss Class"""
262
263 import torch.nn.functional as F
264
265 class FocalLoss(nn.Module):
266     def __init__(self, alpha=1, gamma=2, reduction='mean'):
267         super(FocalLoss, self).__init__()
268         self.alpha = alpha
269         self.gamma = gamma
270         self.reduction = reduction
271
272     def forward(self, inputs, targets):
273
274         probs = torch.sigmoid(inputs)
275
276         bce_loss = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')
277         pt = torch.exp(-bce_loss)
278         focal_loss = self.alpha * (1 - pt)**self.gamma * bce_loss
279
280         if self.reduction == 'mean':
281             return focal_loss.mean()
282         else:
283             return focal_loss.sum()
284
285 """#Training model"""
286
287 drive_save_dir = "/content/drive/MyDrive/EfficientNet_Weights_RC_NOA"
288 os.makedirs(drive_save_dir, exist_ok=True)
289
290
291 save_path = os.path.join(drive_save_dir, "best_efficientnet_b3.pth")
292 criterion = FocalLoss(gamma=2)
293 best_val_loss = float('inf')
294
295
```




```
۲۹۶ history = {
۲۹۷     'train_loss': [], 'val_loss': [],
۲۹۸     'train_acc': [], 'val_acc': []
۲۹۹ }
۳۰۰
۳۰۱ epochs = 10
۳۰۲
۳۰۳ print(" Start Training:")
۳۰۴
۳۰۵ for epoch in range(epochs):
۳۰۶     # ---Train---
۳۰۷     model.train()
۳۰۸     running_loss, train_correct = 0.0, 0
۳۰۹
۳۱۰     for inputs, labels in train_loader:
۳۱۱         inputs, labels = inputs.to(device), labels.to(device)
۳۱۲
۳۱۳         optimizer.zero_grad()
۳۱۴         outputs = model(inputs)
۳۱۵         loss = criterion(outputs, labels)
۳۱۶         loss.backward()
۳۱۷         optimizer.step()
۳۱۸
۳۱۹         running_loss += loss.item() * inputs.size(0)
۳۲۰
۳۲۱
۳۲۲         preds = (torch.sigmoid(outputs) > 0.5).float()
۳۲۳         train_correct += (preds == labels).all(dim=1).sum().item()
۳۲۴
۳۲۵     epoch_train_loss = running_loss / len(train_dataset)
۳۲۶     epoch_train_acc = train_correct / len(train_dataset)
۳۲۷
۳۲۸
۳۲۹     model.eval()
۳۳۰     val_running_loss, val_correct = 0.0, 0
۳۳۱
۳۳۲
۳۳۳     with torch.no_grad():
۳۳۴         for inputs, labels in val_loader:
۳۳۵             inputs, labels = inputs.to(device), labels.to(device)
```



```
۳۳۶         outputs = model(inputs)
۳۳۷         loss = criterion(outputs, labels)
۳۳۸
۳۳۹         val_running_loss += loss.item() * inputs.size(0)
۳۴۰         preds = (torch.sigmoid(outputs) > 5.0).float()
۳۴۱         val_correct += (preds == labels).all(dim=1).sum().item()
۳۴۲
۳۴۳     epoch_val_loss = val_running_loss / len(val_dataset)
۳۴۴     epoch_val_acc = val_correct / len(val_dataset)
۳۴۵
۳۴۶
۳۴۷     history['train_loss'].append(epoch_train_loss)
۳۴۸     history['val_loss'].append(epoch_val_loss)
۳۴۹     history['train_acc'].append(epoch_train_acc)
۳۵۰     history['val_acc'].append(epoch_val_acc)
۳۵۱
۳۵۲     print(f"Epoch {epoch+1}/{epochs} | Train Loss: {epoch_train_loss:.4f} | Train Acc:
↪ {epoch_train_acc:.4f} | Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f}")
۳۵۳
۳۵۴
۳۵۵     if epoch_val_loss < best_val_loss:
۳۵۶         best_val_loss = epoch_val_loss
۳۵۷         torch.save({
۳۵۸             'epoch': epoch + 1,
۳۵۹             'model_state_dict': model.state_dict(),
۳۶۰             'optimizer_state_dict': optimizer.state_dict(),
۳۶۱             'loss': best_val_loss,
۳۶۲             'history': history
۳۶۳         }, save_path)
۳۶۴         print(f" Save Best model to Google Drive! (Val Loss: {best_val_loss:.4f})")
۳۶۵
۳۶۶     """#Plotting Accuracy and Loss vs. Epochs"""
۳۶۷
۳۶۸     import matplotlib.pyplot as plt
۳۶۹
۳۷۰     plt.figure(figsize=(12, 5))
۳۷۱
۳۷۲
۳۷۳     plt.subplot(1, 2, 1)
۳۷۴     plt.plot(history['train_loss'], label='Train Loss')
```



```
۳۷۵ plt.plot(history['val_loss'], label='Val Loss')
۳۷۶ plt.title('Loss per Epoch')
۳۷۷ plt.grid("True")
۳۷۸ plt.legend()
۳۷۹
۳۸۰
۳۸۱ plt.subplot(1, 2, 2)
۳۸۲ plt.plot(history['train_acc'], label='Train Acc')
۳۸۳ plt.plot(history['val_acc'], label='Val Acc')
۳۸۴ plt.title('Accuracy per Epoch')
۳۸۵ plt.legend()
۳۸۶ plt.grid("True")
۳۸۷ plt.show()
۳۸۸
۳۸۹ """#Evaluation on test set"""
۳۹۰
۳۹۱ import os
۳۹۲ import torch
۳۹۳ import torch.nn as nn
۳۹۴ import numpy as np
۳۹۵ import matplotlib.pyplot as plt
۳۹۶ import seaborn as sns
۳۹۷ from torchvision import models
۳۹۸ from sklearn.metrics import classification_report, precision_recall_curve, auc
۳۹۹ from sklearn.manifold import TSNE
۴۰۰
۴۰۱
۴۰۲ model = models.efficientnet_b3(weights=None)
۴۰۳ num_fters = model.classifier[1].in_features
۴۰۴ model.classifier[1] = nn.Linear(num_fters, 12)
۴۰۵
۴۰۶
۴۰۷ save_path = "/content/drive/MyDrive/EfficientNet_Weights_RC_NOA/best_efficientnet_b3.pth"
۴۰۸ if os.path.exists(save_path):
۴۰۹     checkpoint = torch.load(save_path, map_location=device)
۴۱۰     model.load_state_dict(checkpoint['model_state_dict'])
۴۱۱     print(" Model weights loaded successfully from Google Drive!")
۴۱۲ else:
۴۱۳     print(f" Error: Checkpoint not found at {save_path}")
۴۱۴
```



```
۴۱۵ model = model.to(device)
۴۱۶ model.eval()
۴۱۷
۴۱۸
۴۱۹ class_names = test_dataset.classes
۴۲۰ print(f" Class names detected (12 classes): {class_names}")
۴۲۱
۴۲۲ all_preds = []
۴۲۳ all_labels = []
۴۲۴ all_probs = []
۴۲۵ all_features = []
۴۲۶
۴۲۷
۴۲۸ features_flatten = None
۴۲۹ def hook_fn(module, input, output):
۴۳۰     global features_flatten
۴۳۱     features_flatten = torch.mean(output, dim=[2, 3]).detach().cpu().numpy()
۴۳۲
۴۳۳ hook_handle = model.features.register_forward_hook(hook_fn)
۴۳۴
۴۳۵ print(" Evaluating on Test Dataset (12 Classes)...")
۴۳۶ with torch.no_grad():
۴۳۷     for inputs, labels in test_loader: #         test_loader
۴۳۸         inputs, labels = inputs.to(device), labels.to(device)
۴۳۹         outputs = model(inputs)
۴۴۰
۴۴۱         all_features.append(features_flatten)
۴۴۲         probs = torch.sigmoid(outputs)
۴۴۳         preds = (probs > 5.0).float()
۴۴۴
۴۴۵         all_probs.append(probs.cpu().numpy())
۴۴۶         all_preds.append(preds.cpu().numpy())
۴۴۷         all_labels.append(labels.cpu().numpy())
۴۴۸
۴۴۹ hook_handle.remove()
۴۵۰
۴۵۱ all_probs = np.vstack(all_probs)
۴۵۲ all_preds = np.vstack(all_preds)
۴۵۳ all_labels = np.vstack(all_labels)
۴۵۴ all_features = np.vstack(all_features)
```



```
۴۵۵
۴۵۶
۴۵۷ print("\n --- Classification Report (12 Classes) ---")
۴۵۸ print(classification_report(all_labels, all_preds, target_names=class_names, zero_division=0))
۴۵۹
۴۶۰ print(" --- PR-AUC Per Class ---")
۴۶۱ for i, name in enumerate(class_names):
۴۶۲     precision, recall, _ = precision_recall_curve(all_labels[:, i], all_probs[:, i])
۴۶۳     pr_auc = auc(recall, precision)
۴۶۴     print(f"Class {name} | PR-AUC: {pr_auc:4f}")
۴۶۵
۴۶۶ num_classes = len(class_names)
۴۶۷ cm_12x12 = np.zeros((num_classes, num_classes), dtype=int)
۴۶۸ for i in range(num_classes):
۴۶۹     for j in range(num_classes):
۴۷۰         cm_12x12[i, j] = np.sum((all_labels[:, i] == 1) & (all_preds[:, j] == 1))
۴۷۱
۴۷۲ plt.figure(figsize=(14, 11))
۴۷۳ sns.heatmap(cm_12x12, annot=True, fmt='d', cmap='Blues', xticklabels=class_names,
۴۷۴             ↳ yticklabels=class_names)
۴۷۵ plt.title('12x12 Multi-Label Confusion Matrix', fontsize=14, pad=15)
۴۷۶ plt.xlabel('Predicted Target', fontsize=12)
۴۷۷ plt.ylabel('Actual Target', fontsize=12)
۴۷۸ plt.xticks(rotation=45, ha='right')
۴۷۹ plt.tight_layout()
۴۸۰ plt.show()
۴۸۱
۴۸۱ import matplotlib.pyplot as plt
۴۸۲ from matplotlib.patches import Patch
۴۸۳ import seaborn as sns
۴۸۴ import numpy as np
۴۸۵
۴۸۶ print("\n Generating 100% Complete t-SNE Plot for all 12 Classes...")
۴۸۷
۴۸۸ plt.figure(figsize=(13, 9))
۴۸۹
۴۹۰
۴۹۱ palette = sns.color_palette("tab20", n_colors=12)
۴۹۲
۴۹۳
```



```
۴۹۴ color_indices = np.zeros(len(all_labels), dtype=int)
۴۹۵ for class_idx in range(12):
۴۹۶     color_indices[all_labels[:, class_idx] == 1] = class_idx
۴۹۷
۴۹۸
۴۹۹ scatter = plt.scatter(
۵۰۰     features_2d[:, 0],
۵۰۱     features_2d[:, 1],
۵۰۲     c=[palette[i] for i in color_indices],
۵۰۳     alpha=8.0,
۵۰۴     s=45,
۵۰۵     edgecolors='none'
۵۰۶ )
۵۰۷
۵۰۸
۵۰۹ plt.title("t-SNE Visualization of Features (All 12 Classes Included)", fontsize=14, pad=15,
۵۱۰ ↪ fontweight='bold')
۵۱۱ plt.xlabel("t-SNE Component 1", fontsize=11)
۵۱۲ plt.ylabel("t-SNE Component 2", fontsize=11)
۵۱۳ plt.grid(True, linestyle='--', alpha=5.0)
۵۱۴
۵۱۵
۵۱۶ legend_elements = [
۵۱۷     Patch(facecolor=palette[i], edgecolor='none', label=class_names[i])
۵۱۸     for i in range(12)
۵۱۹ ]
۵۲۰
۵۲۱ plt.legend(
۵۲۲     handles=legend_elements,
۵۲۳     title="Cataract Classes",
۵۲۴     title_fontsize='12',
۵۲۵     loc='upper left',
۵۲۶     bbox_to_anchor=(0.2, 1),
۵۲۷     borderaxespad=0.,
۵۲۸     fontsize=11,
۵۲۹     frameon=True,
۵۳۰     shadow=True
۵۳۱ )
۵۳۲
```



```
۵۳۳ plt.tight_layout()
۵۳۴ plt.show()
۵۳۵
۵۳۶ import cv2
۵۳۷
۵۳۸ class GradCAM:
۵۳۹     def __init__(self, model, target_layer):
۵۴۰         self.model = model
۵۴۱         self.target_layer = target_layer
۵۴۲         self.gradients = None
۵۴۳         self.activations = None
۵۴۴
۵۴۵         self.target_layer.register_forward_hook(self.save_activation)
۵۴۶         self.target_layer.register_full_backward_hook(self.save_gradient)
۵۴۷
۵۴۸     def save_activation(self, module, input, output):
۵۴۹         self.activations = output.detach()
۵۵۰
۵۵۱     def save_gradient(self, module, grad_input, grad_output):
۵۵۲         self.gradients = grad_output[0].detach()
۵۵۳
۵۵۴     def generate_heatmap(self, input_image, class_idx):
۵۵۵         self.model.zero_grad()
۵۵۶         output = self.model(input_image)
۵۵۷         target = output[0, class_idx]
۵۵۸         target.backward()
۵۵۹
۵۶۰         weights = torch.mean(self.gradients, dim=[2, 3])[0]
۵۶۱         heatmap = torch.zeros(self.activations.shape[2:], dtype=torch.float32,
۵۶۲             ↪ device=input_image.device)
۵۶۳         for i, w in enumerate(weights):
۵۶۴             heatmap += w * self.activations[0, i, :, :]
۵۶۵
۵۶۶         heatmap = torch.clamp(heatmap, min=0)
۵۶۷         heatmap /= torch.max(heatmap) if torch.max(heatmap) != 0 else 1
۵۶۸         return heatmap.cpu().numpy()
۵۶۹
۵۷۰ target_layer = model.features[-1]
۵۷۱ cam = GradCAM(model, target_layer)
```



```
۵۷۲
۵۷۳
۵۷۴ plt.figure(figsize=(18, 14))
۵۷۵ cols = 4
۵۷۶ rows = 3
۵۷۷
۵۷۸ for class_idx, class_name in enumerate(class_names):
۵۷۹     found_sample = False
۵۸۰     for inputs, labels in test_loader:
۵۸۱         for idx in range(inputs.size(0)):
۵۸۲             if labels[idx][class_idx] == 1:
۵۸۳                 img_tensor = inputs[idx].unsqueeze(0).to(device)
۵۸۴                 found_sample = True
۵۸۵                 break
۵۸۶     if found_sample:
۵۸۷         break
۵۸۸
۵۸۹     if not found_sample:
۵۹۰         print(f" No sample found containing class: {class_name} in test set.")
۵۹۱         continue
۵۹۲
۵۹۳
۵۹۴ heatmap = cam.generate_heatmap(img_tensor, class_idx)
۵۹۵
۵۹۶
۵۹۷ img_show = img_tensor.squeeze(0).cpu().numpy().transpose(1, 2, 0)
۵۹۸ mean = np.array([485.0, 456.0, 406.0])
۵۹۹ std = np.array([229.0, 224.0, 225.0])
۶۰۰ img_show = std * img_show + mean
۶۰۱ img_show = np.clip(img_show, 0, 1)
۶۰۲
۶۰۳
۶۰۴ heatmap_resized = cv2.resize(heatmap, (img_show.shape[1], img_show.shape[0]))
۶۰۵ heatmap_colored = cv2.applyColorMap(np.uint8(255 * heatmap_resized), cv2.COLORMAP_JET)
۶۰۶ heatmap_colored = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB) / 0.255
۶۰۷ overlapped_img = heatmap_colored * 4.0 + img_show * 6.0
۶۰۸
۶۰۹
۶۱۰ ax = plt.subplot(rows, cols, class_idx + 1)
۶۱۱ ax.imshow(overlapped_img)
```




```
۶۱۲     ax.set_title(f"Grad-CAM: {class_name}", fontsize=11)
۶۱۳     ax.axis('off')
۶۱۴
۶۱۵ plt.tight_layout()
۶۱۶ plt.show()
۶۱۷
۶۱۸ """#The Second Approach
۶۱۹ Augmentation
۶۲۰ """
۶۲۱
۶۲۲ import os
۶۲۳ import cv2
۶۲۴ import albumentations as A
۶۲۵ import pandas as pd
۶۲۶ import numpy as np
۶۲۷ from tqdm.auto import tqdm
۶۲۸
۶۲۹
۶۳۰ augmenter = A.Compose([
۶۳۱     A.HorizontalFlip(p=5.0),
۶۳۲     A.VerticalFlip(p=2.0),
۶۳۳     A.Rotate(limit=30, p=7.0, border_mode=cv2.BORDER_CONSTANT),
۶۳۴     A.RandomBrightnessContrast(brightness_limit=3.0, contrast_limit=3.0, p=7.0),
۶۳۵     A.HueSaturationValue(hue_shift_limit=10, sat_shift_limit=20, val_shift_limit=10, p=5.0),
۶۳۶     A.Affine(scale=(8.0, 2.1), translate_percent=(1.0, 1.0), rotate=(-15, 15), p=5.0),
۶۳۷     A.Resize(300, 300)
۶۳۸ ], p=0.1)
۶۳۹
۶۴۰ def augment_and_remove_majors(input_dir, output_dir, csv_path, output_csv_path):
۶۴۱     if not os.path.exists(csv_path):
۶۴۲         print(f" Error: CSV not found at {csv_path}")
۶۴۳         return
۶۴۴
۶۴۵
۶۴۶     df = pd.read_csv(csv_path)
۶۴۷     df.columns = df.columns.str.strip()
۶۴۸
۶۴۹
۶۵۰     cols_to_remove = ['Cornea', 'Pupil']
۶۵۱     df = df.drop(columns=[col for col in cols_to_remove if col in df.columns])
```



```
۶۵۲
۶۵۳
۶۵۴ tool_classes = [c for c in df.columns if c != 'filename']
۶۵۵
۶۵۶
۶۵۷ class_counts = df[tool_classes].sum()
۶۵۸ target_count = int(class_counts.max())
۶۵۹ print(f" New Target Count per class: {target_count} (based on {class_counts.idxmax()})")
۶۶۰
۶۶۱ os.makedirs(output_dir, exist_ok=True)
۶۶۲ final_rows = []
۶۶۳
۶۶۴
۶۶۵ print("Step 1: Copying original images with reduced labels...")
۶۶۶ for idx, row in tqdm(df.iterrows(), total=len(df), desc="Originals"):
۶۶۷     src_path = os.path.join(input_dir, row["filename"])
۶۶۸     if os.path.exists(src_path):
۶۶۹         img = cv2.imread(src_path)
۶۷۰         if img is not None:
۶۷۱
۶۷۲             if row[tool_classes].sum() > 0:
۶۷۳                 cv2.imwrite(os.path.join(output_dir, row["filename"]), img)
۶۷۴                 final_rows.append(row.copy())
۶۷۵
۶۷۶
۶۷۷ print("Step 2: Balancing tools via augmentation...")
۶۷۸ current_df = pd.DataFrame(final_rows)
۶۷۹ current_counts = current_df[tool_classes].sum()
۶۸۰
۶۸۱ for cls in tool_classes:
۶۸۲     needed = target_count - int(current_counts[cls])
۶۸۳     if needed <= 0: continue
۶۸۴
۶۸۵     print(f" Augmenting {cls}: Need {needed}")
۶۸۶     class_samples = current_df[current_df[cls] == 1]
۶۸۷     if len(class_samples) == 0: continue
۶۸۸
۶۸۹     count_added = 0
۶۹۰     while count_added < needed:
۶۹۱         sample = class_samples.sample(n=1).iloc[0]
```



```
۶۹۲     src_path = os.path.join(output_dir, sample["filename"])
۶۹۳     image = cv2.imread(src_path)
۶۹۴     if image is None: continue
۶۹۵
۶۹۶     augmented = augmenter(image=image)["image"]
۶۹۷     new_filename = f"aug_{cls.replace(' ', '_')}_{count_added}_{sample['filename']}"
۶۹۸     cv2.imwrite(os.path.join(output_dir, new_filename), augmented)
۶۹۹
۷۰۰     new_row = sample.copy()
۷۰۱     new_row["filename"] = new_filename
۷۰۲     final_rows.append(new_row)
۷۰۳     count_added += 1
۷۰۴
۷۰۵
۷۰۶ df_final = pd.DataFrame(final_rows).sample(frac=1).reset_index(drop=True)
۷۰۷ df_final.to_csv(output_csv_path, index=False)
۷۰۸
۷۰۹ print(f"\n Done! Dataset is now balanced without Cornea/Pupil.")
۷۱۰ print(f" Total images: {len(df_final)}")
۷۱۱ print(f" Images saved to: {output_dir}")
۷۱۲ print(f" CSV saved to: {output_csv_path}")
۷۱۳
۷۱۴
۷۱۵ augment_and_remove_majors(
۷۱۶     input_dir="/content/cataract_dataset1/train",
۷۱۷     output_dir="/content/train_no_majors_augmented",
۷۱۸     csv_path="/content/collected_csv/train_classes.csv",
۷۱۹     output_csv_path="/content/train_no_majors_final.csv"
۷۲۰ )
۷۲۱
۷۲۲ """#Data Distribution per Class after Augmentation"""
۷۲۳
۷۲۴ import pandas as pd
۷۲۵ import matplotlib.pyplot as plt
۷۲۶ import seaborn as sns
۷۲۷
۷۲۸
۷۲۹ paths = {"Train": "/content/train_no_majors_final.csv"}
۷۳۰
۷۳۱ sns.set_style("whitegrid")
```



```
۷۳۲
۷۳۳ for split_name, path in paths.items():
۷۳۴
۷۳۵
۷۳۶     df = pd.read_csv(path)
۷۳۷
۷۳۸
۷۳۹     label_df = df.drop(columns=["filename"])
۷۴۰
۷۴۱
۷۴۲     class_counts = label_df.sum().sort_values(ascending=False)
۷۴۳
۷۴۴
۷۴۵     plt.figure(figsize=(12, 6))
۷۴۶     ax = sns.barplot(
۷۴۷         x=class_counts.index,
۷۴۸         y=class_counts.values,
۷۴۹         palette="viridis"
۷۵۰     )
۷۵۱
۷۵۲     plt.title(f"Class Distribution - {split_name}", fontsize=16)
۷۵۳     plt.xlabel("Classes", fontsize=12)
۷۵۴     plt.ylabel("Number of Images", fontsize=12)
۷۵۵     plt.xticks(rotation=45, ha="right")
۷۵۶
۷۵۷
۷۵۸     for i, value in enumerate(class_counts.values):
۷۵۹         ax.text(
۷۶۰             i,
۷۶۱             value + 5,
۷۶۲             str(int(value)),
۷۶۳             ha='center',
۷۶۴             va='bottom',
۷۶۵             fontsize=10,
۷۶۶             fontweight='bold'
۷۶۷         )
۷۶۸
۷۶۹     plt.tight_layout()
۷۷۰     plt.show()
۷۷۱
```



```
۷۷۲ """#Removing Cornea and Pupil from Labels"""
۷۷۳
۷۷۴ import os
۷۷۵ import pandas as pd
۷۷۶ from tqdm.auto import tqdm
۷۷۷
۷۷۸ def remove_majors_and_save_csv(csv_path, output_csv_path):
۷۷۹     if not os.path.exists(csv_path):
۷۸۰         print(f" Error: CSV not found at {csv_path}")
۷۸۱         return
۷۸۲
۷۸۳
۷۸۴     df = pd.read_csv(csv_path)
۷۸۵     df.columns = df.columns.str.strip()
۷۸۶
۷۸۷     print(f"Processing {os.path.basename(csv_path)}...")
۷۸۸     print(f"Original shape: {df.shape}")
۷۸۹
۷۹۰
۷۹۱     cols_to_remove = ['Cornea', 'Pupil']
۷۹۲     df_filtered = df.drop(columns=[col for col in cols_to_remove if col in df.columns])
۷۹۳
۷۹۴
۷۹۵     tool_classes = [c for c in df_filtered.columns if c != 'filename']
۷۹۶
۷۹۷
۷۹۸     df_filtered = df_filtered[df_filtered[tool_classes].sum(axis=1) > 0]
۷۹۹
۸۰۰
۸۰۱     df_filtered = df_filtered.reset_index(drop=True)
۸۰۲
۸۰۳
۸۰۴     df_filtered.to_csv(output_csv_path, index=False)
۸۰۵
۸۰۶     print(f" Done! Cleaned data saved.")
۸۰۷     print(f" New shape: {df_filtered.shape}")
۸۰۸     print(f" CSV saved to: {output_csv_path}\n" + "-"*50)
۸۰۹
۸۱۰
۸۱۱
```



```
۸۱۲ remove_majors_and_save_csv(  
۸۱۳     csv_path="/content/collected_csv/valid__classes.csv",  
۸۱۴     output_csv_path="/content/val_no_majors_final.csv"  
۸۱۵ )  
۸۱۶  
۸۱۷  
۸۱۸ remove_majors_and_save_csv(  
۸۱۹     csv_path="/content/collected_csv/test__classes.csv",  
۸۲۰     output_csv_path="/content/test_no_majors_final.csv"  
۸۲۱ )  
۸۲۲  
۸۲۳ train_dataset1 = CataractDataset('/content/train_no_majors_final.csv',  
→   '/content/train_no_majors_augmented', transform=transform)  
۸۲۴ val_dataset1 = CataractDataset('/content/val_no_majors_final.csv',  
→   '/content/cataract_dataset1/valid', transform=transform)  
۸۲۵ test_dataset1 = CataractDataset('/content/test_no_majors_final.csv',  
→   '/content/cataract_dataset1/test', transform=transform)  
۸۲۶  
۸۲۷ train_loader1 = DataLoader(train_dataset1, batch_size=16, shuffle=True, num_workers=2)  
۸۲۸ val_loader1 = DataLoader(val_dataset1, batch_size=16, shuffle=False, num_workers=2)  
۸۲۹ test_loader1 = DataLoader(test_dataset1, batch_size=16, shuffle=False, num_workers=2)  
۸۳۰  
۸۳۱ """#Hyperparameter Optimization Using Optuna Framework"""  
۸۳۲  
۸۳۳ import optuna  
۸۳۴ import gc  
۸۳۵ import torch  
۸۳۶ import torch.nn as nn  
۸۳۷ import torch.optim as optim  
۸۳۸ from torchvision import models  
۸۳۹  
۸۴۰  
۸۴۱ num_classes = len(train_dataset1.classes)  
۸۴۲ print(f" Dynamic Number of Classes detected: {num_classes}")  
۸۴۳  
۸۴۴ def objective(trial):  
۸۴۵     gc.collect()  
۸۴۶     torch.cuda.empty_cache()  
۸۴۷  
۸۴۸
```



```
۸۴۹ lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
۸۵۰ optimizer_name = trial.suggest_categorical("optimizer", ["AdamW", "SGD"])
۸۵۱ weight_decay = trial.suggest_float("weight_decay", 1e-6, 1e-3)
۸۵۲
۸۵۳
۸۵۴ model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
۸۵۵ num_fters = model.classifier[1].in_features
۸۵۶
۸۵۷
۸۵۸ model.classifier[1] = nn.Linear(num_fters, num_classes)
۸۵۹ model = model.to(device)
۸۶۰
۸۶۱
۸۶۲ if optimizer_name == "AdamW":
۸۶۳     optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)
۸۶۴ else:
۸۶۵     optimizer = optim.SGD(model.parameters(), lr=lr, momentum=9.0)
۸۶۶
۸۶۷ criterion = nn.BCEWithLogitsLoss()
۸۶۸
۸۶۹
۸۷۰ model.train()
۸۷۱ for batch_idx, (data, target) in enumerate(train_loader1):
۸۷۲     if batch_idx > 30:
۸۷۳         break
۸۷۴     data, target = data.to(device), target.to(device)
۸۷۵
۸۷۶     optimizer.zero_grad()
۸۷۷     output = model(data)
۸۷۸     loss = criterion(output, target)
۸۷۹     loss.backward()
۸۸۰     optimizer.step()
۸۸۱
۸۸۲
۸۸۳ model.eval()
۸۸۴ val_loss = 0
۸۸۵ with torch.no_grad():
۸۸۶     for data, target in val_loader1:
۸۸۷         data, target = data.to(device), target.to(device)
۸۸۸         output = model(data)
```



```
889         val_loss += criterion(output, target).item()
890
891     return val_loss / len(val_loader1)
892
893
894 study = optuna.create_study(direction="minimize")
895 study.optimize(objective, n_trials=5)
896
897 print("\n Best Hyperparameters found:", study.best_params)
898
899 """#Focal Loss Class"""
900
901 import os
902 import torch
903 import torch.nn as nn
904 import torch.optim as optim
905 from torchvision import models
906 import torch.nn.functional as F
907
908
909 class FocalLoss(nn.Module):
910     def __init__(self, alpha=1, gamma=2, reduction='mean'):
911         super(FocalLoss, self).__init__()
912         self.alpha = alpha
913         self.gamma = gamma
914         self.reduction = reduction
915
916     def forward(self, inputs, targets):
917         probs = torch.sigmoid(inputs)
918         bce_loss = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')
919         pt = torch.exp(-bce_loss)
920         focal_loss = self.alpha * (1 - pt)**self.gamma * bce_loss
921
922         if self.reduction == 'mean':
923             return focal_loss.mean()
924         else:
925             return focal_loss.sum()
926
927 """#Training model"""
928
```




```
۹۲۹ num_classes = len(train_dataset1.classes)
۹۳۰ print(f" Running training with {num_classes} classes.")
۹۳۱
۹۳۲
۹۳۳ model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
۹۳۴ num_fters = model.classifier[1].in_features
۹۳۵
۹۳۶
۹۳۷ model.classifier[1] = nn.Linear(num_fters, num_classes)
۹۳۸ model = model.to(device)
۹۳۹
۹۴۰ optimizer = optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-5)
۹۴۱
۹۴۲
۹۴۳ drive_save_dir = "/content/drive/MyDrive/EfficientNet_Weights_RC_A"
۹۴۴ os.makedirs(drive_save_dir, exist_ok=True)
۹۴۵
۹۴۶ save_path = os.path.join(drive_save_dir, "best_efficientnet_b3.pth")
۹۴۷ criterion = FocalLoss(gamma=2)
۹۴۸ best_val_loss = float('inf')
۹۴۹
۹۵۰ history = {
۹۵۱     'train_loss': [], 'val_loss': [],
۹۵۲     'train_acc': [], 'val_acc': []
۹۵۳ }
۹۵۴
۹۵۵ epochs = 10
۹۵۶
۹۵۷ print(" Start Training:")
۹۵۸
۹۵۹ for epoch in range(epochs):
۹۶۰     # ---Train---
۹۶۱     model.train()
۹۶۲     running_loss, train_correct = 0.0, 0
۹۶۳
۹۶۴     for inputs, labels in train_loader1:
۹۶۵         inputs, labels = inputs.to(device), labels.to(device)
۹۶۶
۹۶۷         optimizer.zero_grad()
۹۶۸         outputs = model(inputs)
```



```
۹۶۹
۹۷۰
۹۷۱     loss = criterion(outputs, labels)
۹۷۲     loss.backward()
۹۷۳     optimizer.step()
۹۷۴
۹۷۵     running_loss += loss.item() * inputs.size(0)
۹۷۶
۹۷۷     preds = (torch.sigmoid(outputs) > 5.0).float()
۹۷۸     train_correct += (preds == labels).all(dim=1).sum().item()
۹۷۹
۹۸۰     epoch_train_loss = running_loss / len(train_dataset1)
۹۸۱     epoch_train_acc = train_correct / len(train_dataset1)
۹۸۲
۹۸۳     # ---Validation---
۹۸۴     model.eval()
۹۸۵     val_running_loss, val_correct = 0.0, 0
۹۸۶
۹۸۷     with torch.no_grad():
۹۸۸         for inputs, labels in val_loader1:
۹۸۹             inputs, labels = inputs.to(device), labels.to(device)
۹۹۰             outputs = model(inputs)
۹۹۱             loss = criterion(outputs, labels)
۹۹۲
۹۹۳             val_running_loss += loss.item() * inputs.size(0)
۹۹۴             preds = (torch.sigmoid(outputs) > 5.0).float()
۹۹۵             val_correct += (preds == labels).all(dim=1).sum().item()
۹۹۶
۹۹۷     epoch_val_loss = val_running_loss / len(val_dataset1)
۹۹۸     epoch_val_acc = val_correct / len(val_dataset1)
۹۹۹
۱۰۰۰     history['train_loss'].append(epoch_train_loss)
۱۰۰۱     history['val_loss'].append(epoch_val_loss)
۱۰۰۲     history['train_acc'].append(epoch_train_acc)
۱۰۰۳     history['val_acc'].append(epoch_val_acc)
۱۰۰۴
۱۰۰۵     print(f"Epoch {epoch+1}/{epochs} | Train Loss: {epoch_train_loss:.4f} | Train Acc:
↪ {epoch_train_acc:.4f} | Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f}")
۱۰۰۶
۱۰۰۷     if epoch_val_loss < best_val_loss:
```



```
1008     best_val_loss = epoch_val_loss
1009     torch.save({
1010         'epoch': epoch + 1,
1011         'model_state_dict': model.state_dict(),
1012         'optimizer_state_dict': optimizer.state_dict(),
1013         'loss': best_val_loss,
1014         'history': history
1015     }, save_path)
1016     print(f" Save Best model to Google Drive! (Val Loss: {best_val_loss:4f.})")
1017
1018     """#Plotting Accuracy and Loss vs. Epochs"""
1019
1020     import matplotlib.pyplot as plt
1021
1022     plt.figure(figsize=(12, 5))
1023
1024
1025     plt.subplot(1, 2, 1)
1026     plt.plot(history['train_loss'], label='Train Loss')
1027     plt.plot(history['val_loss'], label='Val Loss')
1028     plt.title('Loss per Epoch')
1029     plt.grid("True")
1030     plt.legend()
1031
1032
1033     plt.subplot(1, 2, 2)
1034     plt.plot(history['train_acc'], label='Train Acc')
1035     plt.plot(history['val_acc'], label='Val Acc')
1036     plt.title('Accuracy per Epoch')
1037     plt.legend()
1038     plt.grid("True")
1039     plt.show()
1040
1041     """#Evaluation"""
1042
1043     import numpy as np
1044     import matplotlib.pyplot as plt
1045     import seaborn as sns
1046     from sklearn.metrics import classification_report, precision_recall_curve, auc,
1047     => multilabel_confusion_matrix
```



```
1047 from sklearn.manifold import TSNE
1048
1049
1050 model.eval()
1051
1052 all_preds = []
1053 all_labels = []
1054 all_features = []
1055
1056
1057 features_flatten = None
1058 def hook_fn(module, input, output):
1059     global features_flatten
1060
1061     features_flatten = torch.mean(output, dim=[2, 3]).detach().cpu().numpy()
1062
1063
1064 hook_handle = model.features.register_forward_hook(hook_fn)
1065
1066 print(" Evaluating on Test Dataset and extracting features...")
1067 with torch.no_grad():
1068     for inputs, labels in test_loader1:
1069         inputs, labels = inputs.to(device), labels.to(device)
1070         outputs = model(inputs)
1071
1072
1073         all_features.append(features_flatten)
1074
1075
1076         probs = torch.sigmoid(outputs)
1077         preds = (probs > 5.0).float()
1078
1079         all_preds.append(preds.cpu().numpy())
1080         all_labels.append(labels.cpu().numpy())
1081
1082
1083 hook_handle.remove()
1084
1085 all_preds = np.vstack(all_preds)
1086 all_labels = np.vstack(all_labels)
```



```
1087 all_features = np.vstack(all_features)
1088 class_names = train_dataset1.classes
1089
1090
1091 print("\n --- Classification Report (Per Class) ---")
1092 print(classification_report(all_labels, all_preds, target_names=class_names, zero_division=0))
1093
1094 print(" --- PR-AUC Per Class ---")
1095 for i, name in enumerate(class_names):
1096     precision, recall, _ = precision_recall_curve(all_labels[:, i], all_preds[:, i])
1097     pr_auc = auc(recall, precision)
1098     print(f"Class {name} | PR-AUC: {pr_auc:4f}")
1099
1100 import numpy as np
1101 import matplotlib.pyplot as plt
1102 import seaborn as sns
1103
1104
1105 num_classes = len(class_names)
1106 cm_10x10 = np.zeros((num_classes, num_classes), dtype=int)
1107
1108
1109 for i in range(num_classes):
1110     for j in range(num_classes):
1111
1112         cm_10x10[i, j] = np.sum((all_labels[:, i] == 1) & (all_preds[:, j] == 1))
1113
1114
1115 plt.figure(figsize=(12, 10))
1116 sns.heatmap(cm_10x10, annot=True, fmt='d', cmap='Blues',
1117             xticklabels=class_names, yticklabels=class_names)
1118
1119 plt.title('10x10 Multi-Label Confusion Matrix (Co-occurrence)', fontsize=14, pad=15)
1120 plt.xlabel('Predicted Tools', fontsize=12)
1121 plt.ylabel('Actual Tools', fontsize=12)
1122 plt.xticks(rotation=45, ha='right')
1123 plt.yticks(rotation=0)
1124 plt.tight_layout()
1125 plt.show()
1126
```



```
1127 print("\n Running t-SNE (this might take a minute)...")
1128 tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=1000)
1129 features_2d = tsne.fit_transform(all_features)
1130
1131
1132 dominant_labels = np.argmax(all_labels, axis=1)
1133
1134 plt.figure(figsize=(10, 8))
1135 scatter = plt.scatter(features_2d[:, 0], features_2d[:, 1], c=dominant_labels, cmap='tab10',
1136     ↪ alpha=0.7)
1137 plt.colorbar(scatter, ticks=range(10), format=plt.FuncFormatter(lambda val, loc:
1138     ↪ class_names[int(val)]))
1139 plt.title("t-SNE Visualization of Extracted Features (Cataract Tools)")
1140 plt.xlabel("t-SNE Component 1")
1141 plt.ylabel("t-SNE Component 2")
1142 plt.show()
1143
1144 import cv2
1145
1146 class GradCAM:
1147     def __init__(self, model, target_layer):
1148         self.model = model
1149         self.target_layer = target_layer
1150         self.gradients = None
1151         self.activations = None
1152
1153         self.target_layer.register_forward_hook(self.save_activation)
1154         self.target_layer.register_full_backward_hook(self.save_gradient)
1155
1156     def save_activation(self, module, input, output):
1157         self.activations = output.detach()
1158
1159     def save_gradient(self, module, grad_input, grad_output):
1160         self.gradients = grad_output[0].detach()
1161
1162     def generate_heatmap(self, input_image, class_idx):
1163         self.model.zero_grad()
1164         output = self.model(input_image)
```



```
۱۱۶۵
۱۱۶۶     target = output[0, class_idx]
۱۱۶۷     target.backward()
۱۱۶۸
۱۱۶۹
۱۱۷۰     weights = torch.mean(self.gradients, dim=[2, 3])[0]
۱۱۷۱
۱۱۷۲
۱۱۷۳     heatmap = torch.zeros(self.activations.shape[2:], dtype=torch.float32,
↪     device=input_image.device)
۱۱۷۴     for i, w in enumerate(weights):
۱۱۷۵         heatmap += w * self.activations[0, i, :, :]
۱۱۷۶
۱۱۷۷
۱۱۷۸     heatmap = torch.clamp(heatmap, min=0)
۱۱۷۹     heatmap /= torch.max(heatmap) if torch.max(heatmap) != 0 else 1
۱۱۸۰     return heatmap.cpu().numpy()
۱۱۸۱
۱۱۸۲
۱۱۸۳ target_layer = model.features[-1]
۱۱۸۴ cam = GradCAM(model, target_layer)
۱۱۸۵
۱۱۸۶
۱۱۸۷ plt.figure(figsize=(20, 10))
۱۱۸۸ cols = 5
۱۱۸۹ rows = 2
۱۱۹۰
۱۱۹۱ for class_idx, class_name in enumerate(class_names):
۱۱۹۲
۱۱۹۳     found_sample = False
۱۱۹۴     for inputs, labels in test_loader1:
۱۱۹۵         for idx in range(inputs.size(0)):
۱۱۹۶             if labels[idx][class_idx] == 1:
۱۱۹۷                 img_tensor = inputs[idx].unsqueeze(0).to(device)
۱۱۹۸                 found_sample = True
۱۱۹۹                 break
۱۲۰۰             if found_sample:
۱۲۰۱                 break
۱۲۰۲
۱۲۰۳     if not found_sample:
```



```
۱۲۰۴     print(f" No sample found containing class: {class_name} in test set.")
۱۲۰۵
۱۲۰۶     continue
۱۲۰۷
۱۲۰۸     heatmap = cam.generate_heatmap(img_tensor, class_idx)
۱۲۰۹
۱۲۱۰
۱۲۱۱     img_show = img_tensor.squeeze(0).cpu().numpy().transpose(1, 2, 0)
۱۲۱۲     mean = np.array([485.0, 456.0, 406.0])
۱۲۱۳     std = np.array([229.0, 224.0, 225.0])
۱۲۱۴     img_show = std * img_show + mean
۱۲۱۵     img_show = np.clip(img_show, 0, 1)
۱۲۱۶
۱۲۱۷
۱۲۱۸     heatmap_resized = cv2.resize(heatmap, (img_show.shape[1], img_show.shape[0]))
۱۲۱۹     heatmap_colored = cv2.applyColorMap(np.uint8(255 * heatmap_resized), cv2.COLORMAP_JET)
۱۲۲۰     heatmap_colored = cv2.cvtColor(heatmap_colored, cv2.COLOR_BGR2RGB) / 0.255
۱۲۲۱
۱۲۲۲
۱۲۲۳     overlapped_img = heatmap_colored * 4.0 + img_show * 6.0
۱۲۲۴
۱۲۲۵
۱۲۲۶     ax = plt.subplot(rows, cols, class_idx + 1)
۱۲۲۷     ax.imshow(overlapped_img)
۱۲۲۸     ax.set_title(f"Grad-CAM: {class_name}", fontsize=12)
۱۲۲۹     ax.axis('off')
۱۲۳۰
۱۲۳۱ plt.tight_layout()
۱۲۳۲ plt.show()
```




۳ پرسش سوم: اعتبارسنجی متقاطع، ارزیابی پایداری و یادگیری تجمعی

۱.۳ رویکردهای اعتبارسنجی متقاطع (Cross-Validation)

در استقرار مدل‌های هوش مصنوعی در سامانه‌های جراحی رباتیک، اطمینان از عملکرد پایدار (Robustness) در مواجهه با شرایط متغیر اتاق عمل (مانند تغییرات نور، خون‌ریزی و تداخل‌های ابزاری) فراتر از دقت آماری محض است.

۱.۱.۳ مقایسه تطبیقی K-Fold و Stratified K-Fold:

اعتبارسنجی متقاطع (Cross-Validation) ابزاری برای تخمین توانایی تعمیم مدل است. با این حال، انتخاب نوع تقسیم‌بندی داده‌ها تأثیر مستقیمی بر واقع‌گرایانه بودن نتایج دارد:

- **K-Fold Cross-Validation:** در این روش، داده‌ها به K بخش مساوی تقسیم می‌شوند و هر بار یک بخش به عنوان مجموعه تست و سایر بخش‌ها برای آموزش استفاده می‌شوند. این روش فرض می‌کند که داده‌ها به صورت تصادفی و یکنواخت توزیع شده‌اند.

- **Stratified K-Fold:** این رویکرد تضمین می‌کند که نسبت کلاس‌ها در هر "Fold" دقیقاً مشابه نسبت کل داده‌ها در مجموعه داده اصلی باشد. این موضوع در مسائل جراحی که با توزیع‌های نامتعادل (Imbalanced) روبرو هستیم، حیاتی است.

۲.۱.۳ ضرورت استفاده از روش Stratified در جراحی آب‌مروارید:

در دامنه جراحی چشم، فرکانس حضور ابزارها به شدت به فاز جراحی وابسته است. به عنوان مثال، ابزار Cannula در چندین مرحله حضور دارد، اما Lens Injector تنها در فاز نهایی و برای چند ثانیه ظاهر می‌شود.

تحلیل منطق انتخاب: اگر از K-Fold معمولی استفاده شود، این احتمال وجود دارد که در یک "Fold" خاص، هیچ نمونه‌ای از ابزارهای نادر (مانند Cap Forceps) قرار نگیرد یا برعکس، تمام نمونه‌های آن در یک بخش متمرکز شوند. این امر منجر به دو مشکل عمده می‌گردد:

۱. تخمین خوش‌بینانه یا بدبینانه: مدل ممکن است در یک مرحله دقت ۱۰۰٪ و در مرحله دیگر دقت ۰٪ داشته باشد که منجر به افزایش واریانس و عدم قطعیت در نتایج نهایی می‌شود.

۲. شکست در یادگیری ویژگی‌های اقلیت: در سیستم‌های جراحی رباتیک، ابزارهای اقلیت معمولاً ابزارهای بسیار حساسی هستند. روش Stratified به مدل اجازه می‌دهد در تمام مراحل آموزش و ارزیابی، بازنمایی درستی از این کلاس‌های حساس داشته باشد و از «سوگیری به سمت کلاس اکثریت» (Majority Class Bias) جلوگیری کند.

ارتقای قطعیت شبکه (Network Certainty): استفاده از این رویکرد باعث می‌شود که پارامترهای مدل (مانند Weights) بر اساس یک توزیع واقعی از ابزارها بهینه شوند. این پایداری آماری، پیش‌نیاز اصلی برای موتور استنتاج (Inference Engine) یک ربات جراح است تا در لحظات بحرانی جراحی، به دلیل کمبود داده‌های آموزشی در یک توزیع خاص، دچار خطای تشخیص نشود. در واقع، Stratified K-Fold ضامن این است که «مرزهای تصمیم‌گیری» (Decision Boundaries) مدل برای ابزارهای ظریف، با همان دقت ابزارهای اصلی ترسیم شود.

۲.۳ پیاده سازی و ریفکتور کد

۱.۲.۳ پیاده سازی اعتبارسنجی متقاطع لایه بندی شده (Multilabel Stratified K-Fold):

به منظور ارزیابی دقیق توانایی تعمیم دهی مدل و اطمینان از عدم بیش برآش (Overfitting) بر روی یک توزیع خاص، کل مجموعه داده شامل ۶۱۲۷ تصویر (ترکیب Valid و Train) وارد چرخه اعتبارسنجی گردید.

گام اول: تجمیع و لایه بندی هوشمند داده ها: با توجه به ماهیت چند برچسبی تصاویر جراحی، از الگوریتم MultilabelStratifiedKFold استفاده شد. این روش برخلاف K-Fold معمولی، تضمین می کند که نسبت حضور هر ۱۰ ابزار جراحی در هر یک از ۵ بخش (Folds) کاملاً مشابه توزیع کل داده ها باشد. این امر مانع از آن می شود که یک ابزار نادر (مانند Cap Forceps) تنها در بخش آموزش قرار گرفته و در بخش تست غایب باشد.

نکته بسیار مهم در این بخش این است که ما برای این سوال پیش پردازش های بیشتری را استفاده نمودیم. در بخش دوم پیش پردازش ها سبک تر بودند.

گام دوم: چرخه آموزش و بازنشانی مدل: در هر تکرار (Fold)، معماری EfficientNet-B3 به طور کامل بازنشانی (Reset) شده و با استفاده از بهینه ساز AdamW و تابع زیان Focal Loss تحت آموزش قرار گرفت. فرآیند Online Preprocessing شامل چرخش های تصادفی، تغییرات روشنایی و جابه جایی های هندسی به صورت پویا بر روی داده های آموزش هر فولد اعمال شد تا مدل نسبت به تغییرات فیزیکی در حین عمل جراحی مقاوم شود.

۲.۲.۳ تحلیل کمی و کیفی نتایج (Results Analysis):

خروجی های حاصل از ۵ مرحله اعتبارسنجی در جدول زیر خلاصه شده است:

جدول ۵: نتایج اعتبارسنجی متقاطع ۵-تایی برای مدل EfficientNet-B3

Evaluation Stage	Final Accuracy
Fold 1	0.9830
Fold 2	0.9819
Fold 3	0.9815
Fold 4	0.9833
Fold 5	0.9807
Mean Accuracy	0.9821
Standard Deviation (Std Dev)	0.00095

تحلیل پایداری و قطعیت (Robustness Analysis): نتایج فوق نشان دهنده سطح بسیار بالایی از اعتماد پذیری در مدل توسعه داده شده است:



- انحراف معیار بسیار پایین (0.00095): این عدد نشان می‌دهد که عملکرد مدل تقریباً مستقل از نحوه تقسیم‌بندی داده‌هاست. این پایداری برای یک سیستم جراحی رباتیک حیاتی است، چرا که نشان می‌دهد مدل در مواجهه با بیماران جدید و شرایط تصویربرداری متفاوت، رفتاری پیش‌بینی‌پذیر و ثابت خواهد داشت.
 - همگرایی سریع و دقیق: میانگین دقت 98.21 درصد در تمامی فولدها ثابت می‌کند که استراتژی Focal Loss و استفاده از وزن‌های پیش‌آموزش‌دیده ImageNet به خوبی توانسته‌اند چالش‌های نامتوازن کلاس‌ها در تصاویر جراحی آب‌مروارید را پوشش دهند.
 - تأیید معماری: واریانس نزدیک به صفر در نتایج، تأیید می‌کند که پیچیدگی مدل (Complexity) با حجم داده‌های موجود (۶۱۲۷ نمونه) در تعادل کامل است و مدل دچار پدیده‌های مخربی همچون High Variance نشده است.
- در نهایت، موفقیت در آزمون K-Fold با چنین دقت بالایی، چراغ سبزی برای عبور از فاز آزمایشگاهی و ورود به فاز تست‌های استنتاجی (Inference) بر روی بردهای پردازشی لبه (Edge Devices) صادر می‌کند.

کد ۳.۲.۳

```

1 # -*- coding: utf-8 -*-
2 """sec3.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/1qu9I4B_KtE4-h-QegU9Y6sksGT6Q0L2V
8
9 #Import Data
10 """
11
12 from google.colab import drive
13 drive.mount('/content/drive', force_remount = True)
14
15 !cp "/content/drive/MyDrive/Cataract_dataset_classification.v1i.multiclass.zip" .
16
17 !unzip Cataract_dataset_classification.v1i.multiclass.zip -d cataract_dataset1/
18
19 !pip install iterative-stratification
20
21 import os
22 import shutil
23 import pandas as pd
24 import numpy as np
25 import torch
26 import torch.nn as nn

```



```
۲۷ import torch.optim as optim
۲۸ import torch.nn.functional as F
۲۹ from torch.utils.data import Dataset, DataLoader, SubsetRandomSampler
۳۰ from torchvision import transforms, models
۳۱ from PIL import Image
۳۲ from iterstrat.ml_stratifiers import MultilabelStratifiedKfold
۳۳ from google.colab import drive
۳۴
۳۵ base_path = "/content/cataract_dataset1"
۳۶ output_folder = "/content/collected_csv"
۳۷ os.makedirs(output_folder, exist_ok=True)
۳۸
۳۹ """#Combine train and validation Dataset"""
۴۰
۴۱ base_path = "/content/cataract_dataset1"
۴۲ output_folder = "/content/collected_csv"
۴۳ os.makedirs(output_folder, exist_ok=True)
۴۴
۴۵ for split in ["train", "valid"]:
۴۶     split_path = os.path.join(base_path, split)
۴۷     if os.path.exists(split_path):
۴۸         for file in os.listdir(split_path):
۴۹             if file.endswith(".csv"):
۵۰                 shutil.copy(os.path.join(split_path, file), os.path.join(output_folder,
↳ f"{split}_{file}"))
۵۱
۵۲ df_train = pd.read_csv(os.path.join(output_folder, "train_classes.csv"))
۵۳ df_valid = pd.read_csv(os.path.join(output_folder, "valid_classes.csv"))
۵۴
۵۵ df_train.columns = df_train.columns.str.strip()
۵۶ df_valid.columns = df_valid.columns.str.strip()
۵۷
۵۸
۵۹ df_train['folder'] = os.path.join(base_path, "train")
۶۰ df_valid['folder'] = os.path.join(base_path, "valid")
۶۱
۶۲
۶۳ df_combined = pd.concat([df_train, df_valid], ignore_index=True)
۶۴ target_cols = [c for c in df_combined.columns if c not in ['filename', 'folder']]
۶۵ Y = df_combined[target_cols].values
```



```
۶۶
۶۷ print(f" Total samples for K-Fold (Train + Valid): {len(df_combined)}")
۶۸
۶۹ """#Preprocessing"""
۷۰
۷۱ train_transform = transforms.Compose([
۷۲     transforms.Resize((300, 300)),
۷۳     transforms.RandomHorizontalFlip(p=5.0),
۷۴     transforms.RandomVerticalFlip(p=2.0),
۷۵     transforms.RandomRotation(degrees=30),
۷۶     transforms.ColorJitter(brightness=3.0, contrast=3.0, hue=1.0, saturation=2.0),
۷۷     transforms.RandomAffine(degrees=15, translate=(1.0, 1.0), scale=(8.0, 2.1)),
۷۸     transforms.ToTensor(),
۷۹     transforms.Normalize(mean=[485.0, 456.0, 406.0], std=[229.0, 224.0, 225.0])
۸۰ ])
۸۱
۸۲ valid_transform = transforms.Compose([
۸۳     transforms.Resize((300, 300)),
۸۴     transforms.ToTensor(),
۸۵     transforms.Normalize(mean=[485.0, 456.0, 406.0], std=[229.0, 224.0, 225.0])
۸۶ ])
۸۷
۸۸ """#Focal loss class"""
۸۹
۹۰ device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
۹۱
۹۲ class FocalLoss(nn.Module):
۹۳     def __init__(self, alpha=1, gamma=2, reduction='mean'):
۹۴         super(FocalLoss, self).__init__()
۹۵         self.alpha = alpha
۹۶         self.gamma = gamma
۹۷         self.reduction = reduction
۹۸
۹۹     def forward(self, inputs, targets):
۱۰۰         probs = torch.sigmoid(inputs)
۱۰۱         bce_loss = F.binary_cross_entropy_with_logits(inputs, targets, reduction='none')
۱۰۲         pt = torch.exp(-bce_loss)
۱۰۳         focal_loss = self.alpha * (1 - pt)**self.gamma * bce_loss
۱۰۴         if self.reduction == 'mean': return focal_loss.mean()
۱۰۵         else: return focal_loss.sum()
```



```
class CataractDataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df = dataframe.reset_index(drop=True)
        self.transform = transform
        self.classes = [c for c in self.df.columns if c not in ['filename', 'folder']]

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        img_path = os.path.join(row['folder'], row['filename'])
        image = Image.open(img_path).convert('RGB')
        labels = torch.tensor(row[self.classes].values.astype(float), dtype=torch.float32)

        if self.transform:
            image = self.transform(image)
        return image, labels

train_dataset = CataractDataset(df_combined, transform=train_transform)
valid_dataset = CataractDataset(df_combined, transform=valid_transform)

def get_model():
    model = models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.IMAGENET1K_V1)
    num_fters = model.classifier[1].in_features
    model.classifier[1] = nn.Linear(num_fters, len(target_cols)) # 12 Classes
    return model.to(device)

"""#Training model"""

K_FOLDS = 5
EPOCHS_PER_FOLD = 5
BATCH_SIZE = 16
LEARNING_RATE = 1e-4

mskf = MultilabelStratifiedKFold(n_splits=K_FOLDS, shuffle=True, random_state=42)
fold_results = {}

print(f"\n Starting {K_FOLDS}-Fold Cross Validation with Online Augmentation...")
```



```
۱۴۶
۱۴۷ for fold, (train_idx, val_idx) in enumerate(mskf.split(np.zeros(len(df_combined)), Y)):
۱۴۸     print(f"\n{'-'*30}\n FOLD {fold + 1}/{K_FOLDS}\n{'-'*30}")
۱۴۹
۱۵۰
۱۵۱     train_subsampler = SubsetRandomSampler(train_idx)
۱۵۲     val_subsampler = SubsetRandomSampler(val_idx)
۱۵۳
۱۵۴     train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, sampler=train_subsampler,
    ↪     num_workers=2)
۱۵۵     val_loader = DataLoader(valid_dataset, batch_size=BATCH_SIZE, sampler=val_subsampler,
    ↪     num_workers=2)
۱۵۶
۱۵۷
۱۵۸     model = get_model()
۱۵۹     optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE, weight_decay=1e-4)
۱۶۰     criterion = FocalLoss(gamma=2)
۱۶۱
۱۶۲     best_fold_acc = 0.0
۱۶۳
۱۶۴     for epoch in range(EPOCHS_PER_FOLD):
۱۶۵
۱۶۶         model.train()
۱۶۷         train_loss, train_correct, train_total = 0.0, 0, 0
۱۶۸
۱۶۹         for inputs, labels in train_loader:
۱۷۰             inputs, labels = inputs.to(device), labels.to(device)
۱۷۱             optimizer.zero_grad()
۱۷۲             outputs = model(inputs)
۱۷۳             loss = criterion(outputs, labels)
۱۷۴             loss.backward()
۱۷۵             optimizer.step()
۱۷۶
۱۷۷             train_loss += loss.item() * inputs.size(0)
۱۷۸             preds = (torch.sigmoid(outputs) > 5.0).float()
۱۷۹             train_correct += (preds == labels).sum().item()
۱۸۰             train_total += labels.numel()
۱۸۱
۱۸۲         avg_train_loss = train_loss / len(train_idx)
۱۸۳         train_acc = train_correct / train_total
```



```
۱۸۴
۱۸۵
۱۸۶     model.eval()
۱۸۷     val_loss, val_correct, val_total = 0.0, 0, 0
۱۸۸
۱۸۹     with torch.no_grad():
۱۹۰         for inputs, labels in val_loader:
۱۹۱             inputs, labels = inputs.to(device), labels.to(device)
۱۹۲             outputs = model(inputs)
۱۹۳             loss = criterion(outputs, labels)
۱۹۴
۱۹۵             val_loss += loss.item() * inputs.size(0)
۱۹۶             preds = (torch.sigmoid(outputs) > 5.0).float()
۱۹۷             val_correct += (preds == labels).sum().item()
۱۹۸             val_total += labels.numel()
۱۹۹
۲۰۰     avg_val_loss = val_loss / len(val_idx)
۲۰۱     val_acc = val_correct / val_total
۲۰۲
۲۰۳     print(f"Epoch {epoch+1} | Train Loss: {avg_train_loss:.4f}, Acc: {train_acc:.4f} | Val
↳ Loss: {avg_val_loss:.4f}, Acc: {val_acc:.4f}")
۲۰۴
۲۰۵     if val_acc > best_fold_acc:
۲۰۶         best_fold_acc = val_acc
۲۰۷
۲۰۸     fold_results[f'Fold {fold+1}'] = best_fold_acc
۲۰۹
۲۱۰
۲۱۱     del model, optimizer, criterion
۲۱۲     torch.cuda.empty_cache()
۲۱۳
۲۱۴     """#K-Fold Validation Results"""
۲۱۵
۲۱۶     print("\n --- K-Fold Validation Results ---")
۲۱۷     accuracies = list(fold_results.values())
۲۱۸
۲۱۹     for key, value in fold_results.items():
۲۲۰         print(f"{key}: {value:4f}")
۲۲۱
۲۲۲     mean_acc = np.mean(accuracies)
```



```

۲۲۳ variance_acc = np.var(accuracies)
۲۲۴ std_acc = np.std(accuracies)
۲۲۵
۲۲۶ print("-" * 30)
۲۲۷ print(f" Mean Accuracy: {mean_acc:4f}")
۲۲۸ print(f" Variance: {variance_acc:6f}")
۲۲۹ print(f" Standard Deviation: {std_acc:6f}")
۲۳۰
۲۳۱ if std_acc < 05.0:
۲۳۲     print("\n The standard deviation is low. Our architecture is highly robust against changes in
        ↳ the training data.")
۲۳۳ else:
۲۳۴     print("\n The standard deviation is high. The model's stability relative to the data
        ↳ distribution is low.")
    
```

۳.۳ یادگیری تجمعی (Ensemble Learning) در برابر سرعت استنتاج

پس از آموزش K مدل در فرآیند اعتبارسنجی متقاطع، ترکیب این مدل‌ها برای دستیابی به بالاترین سطح از پایداری امری وسوسه‌انگیز است. با این حال، در محیط‌های حساس جراحی رباتیک که پاسخ‌دهی بلادرنگ حیاتی است، انتخاب استراتژی ترکیب مدل‌ها مستقیماً بر شاخص‌های کلیدی عملکرد (KPIs) استنتاج تأثیر می‌گذارد.

۱. رویکرد استنتاج تجمعی (Inference-time Ensemble): در این روش، تصویر ورودی به طور همزمان (یا متوالی) توسط هر K مدل پردازش شده و نتایج آن‌ها (مثلاً از طریق Voting یا Averaging) ترکیب می‌شود.

- چالش تأخیر (Latency): مطابق با استانداردهای کنترل رباتیک (نیاز به نرخ بروزرسانی ۵۰ هرتز یا تأخیر کمتر از ۲۰ میلی‌ثانیه)، این روش باعث افزایش خطی زمان استنتاج می‌شود. اگر پردازش یک مدل ۱۵ میلی‌ثانیه طول بکشد، یک انسمبل ۵ تایی زمانی معادل ۷۵ میلی‌ثانیه نیاز خواهد داشت که باعث خروج سیستم از فاز Real-time می‌گردد.

- کاهش نرخ عبور (Throughput): با اشغال منابع پردازشی (GPU/VRAM) توسط چندین مدل، تعداد فریم‌های قابل پردازش در ثانیه به شدت افت کرده و باعث ایجاد سکت (Lag) در فید ویدئویی جراح می‌شود.

۲. رویکرد سوپ مدل (Model Soups): این تکنیک نوین، به جای ترکیب خروجی مدل‌ها در زمان استنتاج، به میانگین‌گیری از وزن‌های (Parameters) چندین مدل که در فولدهای مختلف آموزش دیده‌اند، می‌پردازد. نتیجه این کار، یک مدل واحد است که دانش تمام مدل‌های قبلی را در خود جای داده است.

حل تضاد دقت و سرعت با Model Soups: رویکرد Model Soups تضاد میان دقت بالا و محدودیت‌های سخت‌افزاری (مانند حافظه محدود در Jetson Nano) را به شکل زیر برطرف می‌سازد:

- تأخیر ثابت: از آنجا که در نهایت تنها یک مدل (با وزن‌های بهینه‌شده) در زمان استنتاج بارگذاری می‌شود، Latency سیستم دقیقاً مشابه یک مدل واحد باقی می‌ماند و معیارهای تصویربرداری رباتیک (زیر ۲۰ میلی ثانیه) حفظ می‌شود.
 - بهینه‌سازی FLOPs و حافظه: برخلاف انسمبل سنتی که نیاز به حافظه K برابر دارد، در این روش بار حافظه (Memory Footprint) افزایش نمی‌یابد. این موضوع برای استقرار بر روی بردهای جراحی که محدودیت رم دارند حیاتی است.
 - تقویت پایداری (Robustness): تحقیقات نشان می‌دهد که میانگین‌گیری از وزن‌ها در فضای پارامتری اغلب منجر به یافتن «کمینه‌های پهن‌تر» (Flatter Minima) در تابع زیان می‌شود که به معنای تعمیم‌پذیری بهتر مدل در مواجهه با نویزهای تصویر جراحی است، بدون آنکه هزینه محاسباتی اضافی در زمان اجرا تحمیل کند.
- در نهایت، برای سیستم‌های جراحی که در آن‌ها هر میلی‌ثانیه تأخیر می‌تواند منجر به خطای ابزاری شود، استفاده از روش Model Soups بر انسمبل‌های سنتی ارجحیت مطلق دارد؛ چرا که دقت مدل تجمعی را با سرعت مدل تکی ترکیب می‌کند.